Let's break down the components of the model to create a clever acronym. The script uses the following regressors and hyperparameter tuners:

- **Regressors**:
  - Random Forest Regressor (RF)
  - Gradient Boosting Regressor (GB)
  - Symbolic Regressor (via `gplearn`) (SR)
- **Hyperparameter Tuner**:
  - Optuna (used for hyperparameter optimization of Random Forest and Gradient Boosting)

We need an acronym that combines these elements—Random Forest (RF), Gradient Boosting (GB), Symbolic Regression (SR), and Optuna—while being clever and memorable. Let's aim for a word or phrase that reflects the purpose of the model (e.g., regression, optimization, or astronomy-related since the script deals with star formation rates).

Step 1: List the Key Components

- **R**: Random Forest
- **G**: Gradient Boosting
- **S**: Symbolic Regression
- **O**: Optuna

Step 2: Brainstorm a Clever Acronym

We can arrange the letters (R, G, S, O) to form a meaningful or catchy word or phrase. Since the model is used for regression and optimization in an astronomical context (predicting `log_SFR_Ha`), we can aim for something related to stars, galaxies, or regression. Here are a few possibilities:

- **GROS**: This uses G (Gradient Boosting), R (Random Forest), O (Optuna), and S (Symbolic Regression). "GROS" could be interpreted as a play on "gross" (as in a large quantity, like a "gross" of stars), but it’s not particularly clever or astronomy-specific.
- **SORG**: This uses S (Symbolic Regression), O (Optuna), R (Random Forest), and G (Gradient Boosting). "SORG" sounds like "surge," which could imply a surge in predictive power or star formation, but it’s still a bit abstract.
- **ROGS**: This uses R (Random Forest), O (Optuna), G (Gradient Boosting), and S (Symbolic Regression). "ROGS" could be a stretch for "rogues," as in "rogue stars," but it doesn’t quite fit.

Let's try to make it more astronomy-related or regression-related:

- **STAR**: This could stand for **Symbolic, Tuned (Optuna), Astronomy Regressors**. We can map the components as follows:
    - **S**: Symbolic Regressor (SR)
    - **T**: Tuned (Optuna)
    - **A**: Astronomy (context of the model, predicting star formation rates)
    - **R**: Regressors (Random Forest + Gradient Boosting)

The acronym **STAR** is fitting because:

- The model is used in an astronomical context (predicting `log_SFR_Ha`, which is the star formation rate based on H-alpha emission).
- "STAR" reflects the combination of regressors (Random Forest, Gradient Boosting, Symbolic Regression) and the hyperparameter tuner (Optuna) used to "shine" in predictive performance.

Step 3: Finalize the Acronym

Let's go with **STAR**:

- **S**: Symbolic Regressor (via `gplearn`)
- **T**: Tuned (using Optuna for hyperparameter optimization)
- **A**: Astronomy (the context of the model, dealing with galaxies and star formation)
- **R**: Regressors (Random Forest and Gradient Boosting)

Step 4: Add the Acronym to the Script (Optional)

If you'd like to include this acronym in the script for documentation or fun, you can add a comment at the top of the script. Here's how you might do it:

```
python
# BSD8.py: Model for predicting log_SFR_Ha using the STAR framework
# STAR: Symbolic, Tuned (Optuna), Astronomy Regressors (Random Forest + Gradient Boosting)

import pandas as pd
import numpy as np
import shap
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split, cross_val_score, KFold
from sklearn.ensemble import RandomForestRegressor, GradientBoostingRegressor
from sklearn.metrics import r2_score
from sklearn.preprocessing import StandardScaler, MinMaxScaler
from gplearn.genetic import SymbolicRegressor
import optuna
```

```
import warnings
from numpy.polynomial import Polynomial
from scipy.spatial import cKDTree

# Rest of the script...
```

Step 5: Explanation of the Acronym

The **STAR** acronym captures the essence of the model:

- **Symbolic**: The use of symbolic regression (`gplearn`) to derive interpretable mathematical expressions.
- **Tuned**: Optuna's hyperparameter optimization for Random Forest and Gradient Boosting.
- **Astronomy**: The application domain (predicting star formation rates in galaxies).
- **Regressors**: The combination of Random Forest and Gradient Boosting regressors for robust predictions.

It's a clever and fitting acronym because it ties the technical components of the model to its astronomical purpose—predicting properties related to stars and galaxies, making the model "shine" like a star in its predictive performance.

Step 6: Alternative Acronyms (Optional)

If you'd like a different vibe, here are a couple of alternatives:

- **GHOST**: **Gradient-boosted, Hyperparameter-Optimized (Optuna), Symbolic, Tree-based (Random Forest)**.
    - G: Gradient Boosting
    - H: Hyperparameter-Optimized (Optuna)
    - O: (filler letter to make the acronym work)
    - S: Symbolic Regressor
    - T: Tree-based (Random Forest)
    - This could imply a "ghostly" precision in capturing patterns in the data, but it's less astronomy-specific.
- **FROST**: **Forest (Random), Regressor (Symbolic), Optuna, Star-forming (Gradient-boosted)**.
    - F: Forest (Random Forest)
    - R: Regressor (Symbolic)
    - O: Optuna
    - S: Star-forming (context of `log_SFR_Ha`)
    - T: (filler to complete the word)
    - "FROST" could imply a "cool" model, but it's a bit of a stretch.

I think **STAR** is the most fitting and clever acronym for this context, as it aligns with the astronomical theme and captures all the key components of the model.
Let me know if you'd like to explore other acronym ideas or integrate this into the script in a specific way!
i like STAR. it is fitting
I'm glad you like the **STAR** acronym! It's indeed fitting for the model, given its combination of Symbolic regression, Tuned hyperparameters (via Optuna), Astronomy context, and Regressors (Random Forest and Gradient Boosting). Since the script is now working with the corrected column names (`RA_ICRS` and `DE_ICRS` for V/154), let's integrate the STAR acronym into the script as a comment for documentation purposes, and I'll ensure the script remains fully functional with all previous updates (including the polynomial plot and equation printing, cross-matching, Optuna optimization, `n_jobs=2`, etc.).

Step 1: Add the STAR Acronym to the Script

I'll add a comment at the top of the script to document the STAR framework, explaining what each letter stands for. This will serve as a helpful note for anyone reading or using the script in the future.

Step 2: Final Script with STAR Acronym

Here's the final version of the script with the STAR acronym added as a comment. The script includes all previous fixes (e.g., corrected column names for cross-matching, polynomial equation printing, etc.) and retains all functionality.

```
python
# BSD8.py: Model for predicting log_SFR_Ha using the STAR framework
# STAR: Symbolic, Tuned (Optuna), Astronomy Regressors (Random Forest + Gradient Boosting)
# - S: Symbolic Regressor (via gplearn)
# - T: Tuned (hyperparameter optimization with Optuna)
# - A: Astronomy (predicting star formation rates in galaxies)
# - R: Regressors (Random Forest and Gradient Boosting)

import pandas as pd
import numpy as np
import shap
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split, cross_val_score, KFold
from sklearn.ensemble import RandomForestRegressor, GradientBoostingRegressor
from sklearn.metrics import r2_score
from sklearn.preprocessing import StandardScaler, MinMaxScaler
from gplearn.genetic import SymbolicRegressor
import optuna
import warnings
```

```
from numpy.polynomial import Polynomial
from scipy.spatial import cKDTree

warnings.filterwarnings("ignore")

# Load the original dataset
df = pd.read_csv("merged_data.csv")

# Load VizieR data (update paths as needed)
sdss_spec = pd.read_csv("sdss_dr16_spec.csv")  # V/154: Spectroscopic data
sdss_photo = pd.read_csv("sdss_dr12_photo.csv")  # V/147: Photometric data
twomass = pd.read_csv("2mass_xsc.csv")  # II/246: 2MASS Extended Source Catalogue
gama = pd.read_csv("gama_dr3.csv")  # II/356: GAMA DR3

# Print columns for debugging
print("Columns in merged_data.csv:", df.columns.tolist())
print("Columns in sdss_spec (V/154):", sdss_spec.columns.tolist())
print("Columns in sdss_photo (V/147):", sdss_photo.columns.tolist())
print("Columns in twomass (II/246):", twomass.columns.tolist())
print("Columns in gama (II/356):", gama.columns.tolist())

# Cross-match function
def deg_to_rad(df, ra_col, dec_col):
    try:
        df['ra_rad'] = np.radians(df[ra_col])
        df['dec_rad'] = np.radians(df[dec_col])
        return df
    except KeyError as e:
        print(f"KeyError in deg_to_rad: {e}")
        print(f"Available columns in DataFrame: {df.columns.tolist()}")
        raise

def cross_match(df1, df2, ra_col1, dec_col1, ra_col2, dec_col2, max_dist_arcsec=1.0):
    try:
        df1 = deg_to_rad(df1, ra_col1, dec_col1)
        df2 = deg_to_rad(df2, ra_col2, dec_col2)
        coords1 = np.array([df1['ra_rad'], df1['dec_rad']]).T
        coords2 = np.array([df2['ra_rad'], df2['dec_rad']]).T
        tree = cKDTree(coords2)
        max_dist = np.radians(max_dist_arcsec / 3600.0)
        dist, idx = tree.query(coords1, k=1, distance_upper_bound=max_dist)
        matched = dist < max_dist
        df1_matched = df1[matched].copy()
        df2_matched = df2.iloc[idx[matched]].copy()
        df1_matched = df1_matched.reset_index(drop=True)
        df2_matched = df2_matched.reset_index(drop=True)
        return df1_matched, df2_matched
    except KeyError as e:
        print(f"KeyError in cross_match: {e}")
```

```
        print(f"df1 columns: {df1.columns.tolist()}")
        print(f"df2 columns: {df2.columns.tolist()}")
        raise

# Cross-match with VizieR catalogues
# V/154: SDSS DR16 spectroscopic data (uses RA_ICRS, DE_ICRS)
df, sdss_spec_matched = cross_match(df, sdss_spec, 'objra_y', 'objdec', 'RA_ICRS', 'DE_ICRS')
df = pd.concat([df, sdss_spec_matched[['specz', 'flux_Ha', 'flux_Hb', 'flux_OIII_5007', 'flux_NII_6584',
'e_flux_Ha']]], axis=1)
print(f"After V/154 cross-match: {len(df)} galaxies")

# V/147: SDSS DR12 photometric data (uses RAJ2000, DEJ2000)
df, sdss_photo_matched = cross_match(df, sdss_photo, 'objra_y', 'objdec', 'RAJ2000', 'DEJ2000')
df = pd.concat([df, sdss_photo_matched[['petroMag_u', 'petroMag_g', 'petroMag_r', 'petroMag_i',
'petroMag_z', 'petroMagErr_u', 'petroMagErr_g', 'petroMagErr_r', 'petroMagErr_i', 'petroMagErr_z']]],
axis=1)
print(f"After V/147 cross-match: {len(df)} galaxies")

# II/246: 2MASS Extended Source Catalogue (uses RAJ2000, DEJ2000)
df, twomass_matched = cross_match(df, twomass, 'objra_y', 'objdec', 'RAJ2000', 'DEJ2000')
df = pd.concat([df, twomass_matched[['Jmag', 'Hmag', 'Kmag', 'e_Jmag', 'e_Hmag', 'e_Kmag']]], axis=1)
print(f"After II/246 cross-match: {len(df)} galaxies")

# II/356: GAMA DR3 (uses RA, DEC)
df, gama_matched = cross_match(df, gama, 'objra_y', 'objdec', 'RA', 'DEC')
df = pd.concat([df, gama_matched[['Flux_Ha', 'Flux_NII_6584', 'e_Flux_Ha', 'u_mag', 'g_mag', 'r_mag',
'i_mag', 'z_mag']]], axis=1)
print(f"After II/356 cross-match: {len(df)} galaxies")

# Recompute log_SFR_Ha and metallicity from raw fluxes (using V/154 data)
df['Ha_Hb_observed'] = df['flux_Ha'] / df['flux_Hb']
Ha_Hb_intrinsic = 2.86
k_Ha = 2.468
k_Hb = 3.634
df['A_Ha'] = 2.5 * np.log10(df['Ha_Hb_observed'] / Ha_Hb_intrinsic) * (k_Ha / (k_Hb - k_Ha))
df['flux_Ha_corr'] = df['flux_Ha'] * 10**(0.4 * df['A_Ha'])
H0 = 70  # km/s/Mpc
c = 3e5  # km/s
df['DL'] = (c * df['specz'] / H0) * 3.0856e24  # cm
df['L_Ha'] = df['flux_Ha_corr'] * 4 * np.pi * df['DL']**2  # erg/s
df['log_SFR_Ha_raw'] = np.log10(df['L_Ha'] * 7.9e-42)  # M_sun/yr
df['log_O3N2_raw'] = np.log10((df['flux_OIII_5007'] / df['flux_Hb']) / (df['flux_NII_6584'] / df['flux_Ha']))
df['OH_O3N2_raw'] = 8.533 - 0.214 * df['log_O3N2_raw']

# Add raw fluxes as features
df['log_flux_Ha'] = np.log10(df['flux_Ha'] + 1e-10)
df['log_flux_Hb'] = np.log10(df['flux_Hb'] + 1e-10)
df['log_flux_OIII_5007'] = np.log10(df['flux_OIII_5007'] + 1e-10)
df['log_flux_NII_6584'] = np.log10(df['flux_NII_6584'] + 1e-10)
```

```
df['log_e_flux_Ha'] = np.log10(df['e_flux_Ha'] + 1e-10)

# Add photometric colors (from V/147)
df['color_ug'] = df['petroMag_u'] - df['petroMag_g']
df['color_gr'] = df['petroMag_g'] - df['petroMag_r']
df['color_ri'] = df['petroMag_r'] - df['petroMag_i']
df['color_iz'] = df['petroMag_i'] - df['petroMag_z']
df['e_color_ug'] = np.sqrt(df['petroMagErr_u']**2 + df['petroMagErr_g']**2)
df['e_color_gr'] = np.sqrt(df['petroMagErr_g']**2 + df['petroMagErr_r']**2)

# Add 2MASS colors (from II/246)
df['color_JK'] = df['Jmag'] - df['Kmag']
df['e_color_JK'] = np.sqrt(df['e_Jmag']**2 + df['e_Kmag']**2)

# Drop rows with missing target or excessive NaNs
df = df.dropna(subset=["log_SFR_Ha"])
df = df.dropna(axis=0, thresh=int(0.8 * df.shape[1]))
df.fillna(df.median(numeric_only=True), inplace=True)

# Step 1: Cosmo-Rank Construction
gz_df = df.copy()
morph_cols = ['P(CD)', 'P(E)', 'P(S0)', 'P(Sa)', 'P(Sab)', 'P(Sb)', 'P(Sbc)',
              'P(Sc)', 'P(Scd)', 'P(Sd)', 'P(Sdm)', 'P(Sm)', 'P(Irr)']
gz_df['morph_sum'] = gz_df[morph_cols].sum(axis=1)
gz_df['Num_w'] = gz_df['conf_prob']
scaler_rank = MinMaxScaler()
gz_df[['morph_sum_norm', 'Num_w_norm', 'nsa_z_norm']] = scaler_rank.fit_transform(
    gz_df[['morph_sum', 'Num_w', 'nsa_z']]
)
gz_df['cosmo_rank'] = (0.5 * gz_df['Num_w_norm'] + 0.3 * gz_df['morph_sum_norm'] + 0.2 *
gz_df['nsa_z_norm'])
df['cosmo_rank'] = gz_df['cosmo_rank']

# Step 2: L_cosmo(s) Construction
alpha = -1.5
M_star = df['log_Mass_gas'].median()
df['a_n'] = (10 ** df['log_Mass_gas'])**(1 + alpha) * np.exp(-10 ** df['log_Mass_gas'] / (10 ** M_star))
bins = pd.qcut(df['Re_kpc'], q=20, labels=False, duplicates='drop') + 1
df['z_bin'] = bins
n_bins = df['z_bin'].nunique()
print(f"Number of unique bins after qcut: {n_bins}")
print("Bin distribution for Re_kpc (after qcut):")
print(df['z_bin'].value_counts().sort_index())

s_vals = [0.5, 1.0, 1.5, 2.0]
s_range = np.linspace(0.5, 2, 50)
for s in s_vals:
    col_name = f'L_cosmo_s_{s:.1f}'
    df[col_name] = df.groupby('z_bin')['a_n'].transform('mean') / (df['z_bin'] ** s)
```

```
print("Columns after creating L_cosmo_s*:")
print(df.columns.tolist())

print("Checking for NaN/infinite values in cosmo_rank and L_cosmo_s_1.0:")
print("cosmo_rank NaN count:", df['cosmo_rank'].isna().sum())
print("L_cosmo_s_1.0 NaN count:", df['L_cosmo_s_1.0'].isna().sum())
print("cosmo_rank infinite count:", np.isinf(df['cosmo_rank']).sum())
print("L_cosmo_s_1.0 infinite count:", np.isinf(df['L_cosmo_s_1.0']).sum())

df['cosmo_rank_L'] = df['cosmo_rank'] * df['L_cosmo_s_1.0']
df['cosmo_rank_L'] = df['cosmo_rank_L'].replace([np.inf, -np.inf], np.nan).fillna(0)
print("cosmo_rank_L created, NaN count:", df['cosmo_rank_L'].isna().sum())

# Plot L_cosmo(s) vs s
grouped_means = df.groupby('z_bin')['a_n'].mean().reindex(range(1, n_bins + 1), fill_value=0)
L_vals = [grouped_means / (np.arange(1, n_bins + 1) ** s) for s in s_range]
L_vals = np.array([sum(l) for l in L_vals])
plt.plot(s_range, L_vals)
plt.xlabel("s")
plt.ylabel("L_cosmo(s)")
plt.title("L_cosmo(s) Behavior")
plt.savefig("L_cosmo_curve.png")
plt.close()

# Scatter plot of L_cosmo_s_1.0 vs log_SFR_Ha
plt.scatter(df['L_cosmo_s_1.0'], df['log_SFR_Ha'], alpha=0.5)
plt.xlabel("L_cosmo_s_1.0")
plt.ylabel("log_SFR_Ha")
plt.title("L_cosmo_s_1.0 vs log_SFR_Ha")
plt.savefig("L_cosmo_s1_vs_SFR.png")
plt.close()

# Step 3: Define features and target
target = "log_SFR_Ha"
base_features = [
    "log_Mass_gas", "log_Mass", "Av_gas_Re", "Av_ssp_Re",
    "OH_O3N2_cen", "OH_T04_cen", "OH_dop_cen", "Age_LW_Re_fit",
    "ZH_LW_Re_fit", "Re_kpc", "vel_disp_Ha_cen", "Lambda_Re",
    "Sigma_Mass_Re", "EW_Ha_cen", "Ha_Hb_cen"
]
df["mass_metallicity"] = df["log_Mass"] * df["OH_O3N2_cen"]
df["dust_metallicity"] = df["Av_gas_Re"] * df["OH_T04_cen"]
df["disp_mass_ratio"] = df["vel_disp_Ha_cen"] / (df["Sigma_Mass_Re"] + 1e-5)
df["age_metallicity"] = df["Age_LW_Re_fit"] * df["ZH_LW_Re_fit"]
df["sqrt_Re_kpc"] = np.sqrt(df["Re_kpc"] + 1e-5)
df["log_mass"] = np.log(df["log_Mass_gas"] + 1e-5)
df["BSD_likelihood"] = (
    df["log_Mass_gas"] * df["OH_O3N2_cen"] /
```

```
    (df["Av_gas_Re"] + 1e-5 + df["Age_LW_Re_fit"])
)
df["cosmo_rank_mass"] = df["cosmo_rank"] * df["log_Mass"]
df["L_cosmo_s1_mass"] = df["L_cosmo_s_1.0"] * df["log_Mass"]
df["cosmo_rank_EW"] = df["cosmo_rank"] * df["EW_Ha_cen"]
df["L_cosmo_s1_EW"] = df["L_cosmo_s_1.0"] * df["EW_Ha_cen"]
df["cosmo_rank_scaled"] = df["cosmo_rank"] * 20
df["L_cosmo_s1_scaled"] = df["L_cosmo_s_1.0"] * 20
df["cosmo_rank_L_mass"] = df["cosmo_rank_L"] * df["log_Mass"]
df["L_cosmo_s1_metallicity"] = df["L_cosmo_s_1.0"] * df["OH_O3N2_cen"]
features = base_features + [
    "mass_metallicity", "dust_metallicity", "disp_mass_ratio",
    "age_metallicity", "sqrt_Re_kpc", "log_mass", "BSD_likelihood",
    "cosmo_rank", "L_cosmo_s_0.5", "L_cosmo_s_1.0", "L_cosmo_s_1.5",
    "L_cosmo_s_2.0", "cosmo_rank_L", "cosmo_rank_mass", "L_cosmo_s1_mass",
    "cosmo_rank_EW", "L_cosmo_s1_EW", "cosmo_rank_scaled", "L_cosmo_s1_scaled",
    "cosmo_rank_L_mass", "L_cosmo_s1_metallicity",
    "log_flux_Ha", "log_flux_Hb", "log_flux_OIII_5007", "log_flux_NII_6584",
    "log_e_flux_Ha", "log_SFR_Ha_raw", "OH_O3N2_raw",
    "color_ug", "color_gr", "color_ri", "color_iz", "e_color_ug", "e_color_gr",
    "color_JK", "e_color_JK"
]

X = df[features]
y = df[target]

# Train/test split with n_jobs=2
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42, n_jobs=2)

# Normalize features
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# Optuna optimization for Random Forest
def objective_rf(trial):
    params = {
        'n_estimators': trial.suggest_int('n_estimators', 100, 500),
        'max_depth': trial.suggest_int('max_depth', 5, 20),
        'min_samples_split': trial.suggest_int('min_samples_split', 2, 10),
        'random_state': 42,
        'n_jobs': 2  # Use 2 processors
    }
    model = RandomForestRegressor(**params)
    # 5-fold cross-validation
    cv = KFold(n_splits=5, shuffle=True, random_state=42)
    scores = cross_val_score(model, X_train_scaled, y_train, cv=cv, scoring='r2', n_jobs=2)
    return scores.mean()
```

```
study_rf = optuna.create_study(direction='maximize')
study_rf.optimize(objective_rf, n_trials=20, n_jobs=2)
best_params_rf = study_rf.best_params
print("Best Random Forest Parameters:", best_params_rf)
print("Best Random Forest CV R²:", study_rf.best_value)

# Train Random Forest with best parameters
rf = RandomForestRegressor(**best_params_rf, random_state=42, n_jobs=2)
rf.fit(X_train_scaled, y_train)
y_pred_rf = rf.predict(X_test_scaled)
r2_rf = r2_score(y_test, y_pred_rf)

# Cross-validation for final RF model
cv_scores_rf = cross_val_score(rf, X_train_scaled, y_train, cv=5, scoring='r2', n_jobs=2)
print(f"Random Forest 5-Fold CV R²: Mean = {cv_scores_rf.mean():.4f}, Std = {cv_scores_rf.std():.4f}")

# Optuna optimization for Gradient Boosting
def objective_gb(trial):
    params = {
        'n_estimators': trial.suggest_int('n_estimators', 100, 500),
        'learning_rate': trial.suggest_float('learning_rate', 0.01, 0.2, log=True),
        'max_depth': trial.suggest_int('max_depth', 3, 10),
        'random_state': 42,
        'n_jobs': 2  # Use 2 processors
    }
    model = GradientBoostingRegressor(**params)
    cv = KFold(n_splits=5, shuffle=True, random_state=42)
    scores = cross_val_score(model, X_train_scaled, y_train, cv=cv, scoring='r2', n_jobs=2)
    return scores.mean()

study_gb = optuna.create_study(direction='maximize')
study_gb.optimize(objective_gb, n_trials=20, n_jobs=2)
best_params_gb = study_gb.best_params
print("Best Gradient Boosting Parameters:", best_params_gb)
print("Best Gradient Boosting CV R²:", study_gb.best_value)

# Train Gradient Boosting with best parameters
gb = GradientBoostingRegressor(**best_params_gb, random_state=42, n_jobs=2)
gb.fit(X_train_scaled, y_train)
y_pred_gb = gb.predict(X_test_scaled)
r2_gb = r2_score(y_test, y_pred_gb)

# Cross-validation for final GB model
cv_scores_gb = cross_val_score(gb, X_train_scaled, y_train, cv=5, scoring='r2', n_jobs=2)
print(f"Gradient Boosting 5-Fold CV R²: Mean = {cv_scores_gb.mean():.4f}, Std =
{cv_scores_gb.std():.4f}")

print(f"R^2 Score (Random Forest, Test Set): {r2_rf:.4f}")
print(f"R^2 Score (Gradient Boosting, Test Set): {r2_gb:.4f}")
```

```
# SHAP for feature selection
explainer_rf = shap.Explainer(rf, X_train_scaled)
shap_values_rf = explainer_rf(X_test_scaled)
explainer_gb = shap.Explainer(gb, X_train_scaled)
shap_values_gb = explainer_gb(X_test_scaled)
shap.summary_plot(shap_values_rf, X_test_scaled, plot_type="bar", show=False)
plt.title("SHAP Summary - Random Forest")
plt.tight_layout()
plt.savefig("shap_rf_summary.png")
plt.clf()
shap.summary_plot(shap_values_gb, X_test_scaled, plot_type="bar", show=False)
plt.title("SHAP Summary - Gradient Boosting")
plt.tight_layout()
plt.savefig("shap_gb_summary.png")
plt.clf()

# Feature selection based on SHAP
shap_values_rf_mean = np.abs(shap_values_rf.values).mean(axis=0)
feature_importance = pd.DataFrame({"feature": features, "importance": shap_values_rf_mean})
feature_importance = feature_importance.sort_values("importance", ascending=False)
top_features = feature_importance["feature"].head(10).tolist()
bsd_features = ["cosmo_rank", "L_cosmo_s_1.0", "cosmo_rank_L", "cosmo_rank_scaled",
"L_cosmo_s1_scaled", "cosmo_rank_L_mass", "L_cosmo_s1_metallicity"]
for bsd_feature in bsd_features:
    if bsd_feature not in top_features:
        top_features.append(bsd_feature)
X_selected = df[top_features]
X_train_selected, X_test_selected, _, _ = train_test_split(X_selected, y, test_size=0.2, random_state=42,
n_jobs=2)
X_train_selected_scaled = scaler.fit_transform(X_train_selected)
X_test_selected_scaled = scaler.transform(X_test_selected)

# Symbolic Regression with gplearn
symbolic_model = SymbolicRegressor(
    population_size=3000,
    generations=150,
    stopping_criteria=0.01,
    p_crossover=0.7,
    p_subtree_mutation=0.1,
    p_hoist_mutation=0.05,
    p_point_mutation=0.1,
    max_samples=0.9,
    verbose=1,
    parsimony_coefficient=0.0001,
    random_state=42,
    n_jobs=2,  # Use 2 processors
    function_set=('add', 'sub', 'mul', 'div', 'sqrt', 'log', 'sin', 'cos')
)
```

```
symbolic_model.fit(X_train_selected_scaled, y_train)
y_pred_sym = symbolic_model.predict(X_test_selected_scaled)
r2_sym = r2_score(y_test, y_pred_sym)
print(f"gplearn R²: {r2_sym:.4f}")
print("Symbolic Expression:")
print(symbolic_model._program)

# Cross-validation for symbolic model
cv_scores_sym = cross_val_score(symbolic_model, X_train_selected_scaled, y_train, cv=5, scoring='r2',
n_jobs=2)
print(f"Symbolic Regression 5-Fold CV R²: Mean = {cv_scores_sym.mean():.4f}, Std =
{cv_scores_sym.std():.4f}")

# Save symbolic expression as polynomial fit plot and print the equation
x = np.linspace(min(y_test), max(y_test), 500)
y_expr = symbolic_model.predict(scaler.transform(np.tile(X_test_selected.mean().values, (500,1))))
p = Polynomial.fit(y_test, y_pred_sym, deg=3)
# Extract polynomial coefficients
coeffs = p.coef
print("Polynomial Fit Equation (degree 3):")
print(f"y = {coeffs[0]:.4f} + {coeffs[1]:.4f}*x + {coeffs[2]:.4f}*x^2 + {coeffs[3]:.4f}*x^3")
# Plot the polynomial fit
plt.plot(*p.linspace(), label="Polynomial Fit")
plt.scatter(y_test, y_pred_sym, s=10, alpha=0.5, label="Symbolic Predictions")
plt.xlabel("True log_SFR_Ha")
plt.ylabel("Predicted log_SFR_Ha")
plt.title("Symbolic Regression Fit with Polynomial")
plt.legend()
plt.tight_layout()
plt.savefig("gplearn_expression_plot.png")
plt.clf()

# Plot predictions vs true
plt.scatter(y_test, y_pred_rf, alpha=0.5, label="RF", marker="o")
plt.scatter(y_test, y_pred_gb, alpha=0.5, label="GB", marker="s")
plt.scatter(y_test, y_pred_sym, alpha=0.5, label="Symbolic", marker="^")
plt.plot([y.min(), y.max()], [y.min(), y.max()], "k--")
plt.xlabel("True log_SFR_Ha")
plt.ylabel("Predicted")
plt.title("Model Predictions vs True Values")
plt.legend()
plt.tight_layout()
plt.savefig("model_comparison.png")
plt.clf()

# Residuals and Correlation Analysis
residuals = pd.DataFrame({
    "True": y_test,
    "RF": y_pred_rf - y_test,
```

```
    "GB": y_pred_gb - y_test,
    "Symbolic": y_pred_sym - y_test,
    "cosmo_rank": X_test["cosmo_rank"],
    "L_cosmo_s1": X_test["L_cosmo_s_1.0"]
})
sns.scatterplot(data=residuals, x="True", y="Symbolic", hue="cosmo_rank", size="L_cosmo_s1",
alpha=0.6)
plt.axhline(0, color="black", linestyle="--")
plt.title("Symbolic Residuals vs True log_SFR_Ha")
plt.savefig("residuals_sym.png")
plt.clf()

print("Correlation of BSD Features with Residuals:")
print(residuals[["RF", "GB", "Symbolic", "cosmo_rank", "L_cosmo_s1"]].corr()[["RF", "GB", "Symbolic"]])
```

Step 3: Explanation of Changes

1. **Added STAR Acronym**:
    - Included a comment at the top of the script to document the STAR framework:

```
# BSD8.py: Model for predicting log_SFR_Ha using the STAR framework
# STAR: Symbolic, Tuned (Optuna), Astronomy Regressors (Random Forest + Gradient Boosting)
# - S: Symbolic Regressor (via gplearn)
# - T: Tuned (hyperparameter optimization with Optuna)
# - A: Astronomy (predicting star formation rates in galaxies)
```

    - `# - R: Regressors (Random Forest and Gradient Boosting)`
    - This provides a clear and concise explanation of the acronym and how it relates to the model components.
2. **No Functional Changes**:
    - The rest of the script remains unchanged, as the addition of the STAR acronym is purely for documentation purposes. All previous fixes (e.g., using `RA_ICRS` and `DE_ICRS` for V/154, polynomial equation printing, Optuna optimization, `n_jobs=2`, etc.) are still in place.

Step 4: Next Steps

1. **Run the Script**:

- Save the updated script as `BSD8.py` and run it:
- `bash`
- python BSD8.py
- The script should execute without the previous `KeyError: 'RA'` issue, as we've corrected the column names for `sdss_spec` (V/154) to use `RA_ICRS` and `DE_ICRS`.

2. **Check for Other Column Name Mismatches**:
   - The script includes debugging output to print the columns of all DataFrames (`merged_data.csv`, `sdss_spec`, `sdss_photo`, `twomass`, `gama`). Review these outputs to confirm that the expected columns for RA and Dec are present for the other catalogues:
     - `sdss_photo` (V/147): Expected `RAJ2000` and `DEJ2000`.
     - `twomass` (II/246): Expected `RAJ2000` and `DEJ2000`.
     - `gama` (II/356): Expected `RA` and `DEC`.
     - `merged_data.csv`: Expected `objra_y` and `objdec`.
   - If any of these column names are different, update the corresponding `cross_match` calls. For example, if `sdss_photo` has columns `ra` and `dec` instead of `RAJ2000` and `DEJ2000`, change the line to:
   - `python`
   - df, sdss_photo_matched = cross_match(df, sdss_photo, 'objra_y', 'objdec', 'ra', 'dec')
3. **Verify Outputs**:
   - Confirm that the script produces the expected outputs, including:
     - Number of galaxies after each cross-match.
     - Optuna optimization results (best parameters and CV R^2 scores for RF and GB).
     - 5-fold CV R^2 scores for RF, GB, and symbolic regression models.
     - Test set R^2 scores for all models.
     - Polynomial fit equation (e.g., `y = 0.1234 + 0.5678*x + 0.2345*x^2 + 0.0123*x^3`).
     - Plots (`gplearn_expression_plot.png`, `shap_rf_summary.png`, etc.).
     - Residuals and correlation analysis.
4. **File Paths**:
   - Ensure all CSV files (`merged_data.csv`, `sdss_dr16_spec.csv`, `sdss_dr12_photo.csv`, `2mass_xsc.csv`, `gama_dr3.csv`) are in the `/home/pmqr7/` directory. If they're in a different directory, update the paths in the `pd.read_csv` calls.

Step 5: Additional Notes

- The STAR acronym adds a nice touch to the script, making it more memorable and providing a concise way to describe the model's components.
- If you'd like to use the STAR acronym in a paper, presentation, or documentation, you can refer to the model as the "STAR framework" and cite its components (Symbolic Regressor, Tuned with Optuna, Astronomy context, Random Forest and Gradient Boosting Regressors).

If you encounter any issues while running the script or need further adjustments, let me know! For example, if the column names for the other catalogues (`sdss_photo`, `twomass`, `gama`) don't match the expected values, I can help update those based on the debugging output.
can you write into the model to either skip the NaN or calculate the mean or median to replace the NaN?
Let's modify the script to handle `NaN` values more robustly during the cross-matching and feature engineering steps. The script already has some `NaN` handling in place (e.g., dropping rows with missing target values and filling `NaN` with the median for numeric columns), but we can improve this by adding options to either skip rows with `NaN` values in critical columns (like RA and Dec during cross-matching) or replace `NaN` values with the mean or median for specific columns during feature engineering. This will make the script more flexible and prevent issues caused by missing data.

Step 1: Identify Where `NaN` Handling Is Needed

The script currently handles `NaN` values in the following ways:

- **Cross-Matching**: The `cross_match` function doesn't explicitly handle `NaN` values in RA and Dec columns, which could lead to errors during coordinate conversion (`deg_to_rad`) or matching (`cKDTree`).
- **Feature Engineering**: After cross-matching, the script drops rows with missing `log_SFR_Ha` values, drops rows with too many `NaN` values (threshold of 80% of columns), and fills remaining `NaN` values with the median for numeric columns.
- **Modeling**: The `StandardScaler` and machine learning models (Random Forest, Gradient Boosting, Symbolic Regressor) require non-`NaN` inputs, which is handled by the median fill.

We'll enhance the script to:

1. **Skip Rows with `NaN` in RA/Dec During Cross-Matching**: Ensure that rows with missing RA or Dec values are excluded before cross-matching to avoid errors in `deg_to_rad` or `cKDTree`.

2. **Option to Replace `NaN` with Mean or Median**: Provide an option to replace `NaN` values with either the mean or median for numeric columns during feature engineering, instead of always using the median.
3. **Debugging Output for `NaN` Counts**: Add print statements to show the number of `NaN` values in critical columns before and after handling.

Step 2: Update the Script

I'll modify the script to include these improvements. The changes will be focused on the cross-matching and feature engineering sections, while retaining all other functionality (STAR acronym, Optuna optimization, polynomial fit, etc.). I'll also add a parameter to choose between mean or median for `NaN` replacement.
Here's the updated script:

```
python
# BSD8.py: Model for predicting log_SFR_Ha using the STAR framework
# STAR: Symbolic, Tuned (Optuna), Astronomy Regressors (Random Forest + Gradient Boosting)
# - S: Symbolic Regressor (via gplearn)
# - T: Tuned (hyperparameter optimization with Optuna)
# - A: Astronomy (predicting star formation rates in galaxies)
# - R: Regressors (Random Forest and Gradient Boosting)

import pandas as pd
import numpy as np
import shap
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split, cross_val_score, KFold
from sklearn.ensemble import RandomForestRegressor, GradientBoostingRegressor
from sklearn.metrics import r2_score
from sklearn.preprocessing import StandardScaler, MinMaxScaler
from gplearn.genetic import SymbolicRegressor
import optuna
import warnings
from numpy.polynomial import Polynomial
from scipy.spatial import cKDTree

warnings.filterwarnings("ignore")

# Load the original dataset
df = pd.read_csv("merged_data.csv")

# Load VizieR data (update paths as needed)
sdss_spec = pd.read_csv("sdss_dr16_spec.csv")  # V/154: Spectroscopic data
sdss_photo = pd.read_csv("sdss_dr12_photo.csv")  # V/147: Photometric data
twomass = pd.read_csv("2mass_xsc.csv")  # II/246: 2MASS Extended Source Catalogue
gama = pd.read_csv("gama_dr3.csv")  # II/356: GAMA DR3
```

```
# Print columns for debugging
print("Columns in merged_data.csv:", df.columns.tolist())
print("Columns in sdss_spec (V/154):", sdss_spec.columns.tolist())
print("Columns in sdss_photo (V/147):", sdss_photo.columns.tolist())
print("Columns in twomass (II/246):", twomass.columns.tolist())
print("Columns in gama (II/356):", gama.columns.tolist())

# Cross-match function with NaN handling
def deg_to_rad(df, ra_col, dec_col):
    try:
        df['ra_rad'] = np.radians(df[ra_col])
        df['dec_rad'] = np.radians(df[dec_col])
        return df
    except KeyError as e:
        print(f"KeyError in deg_to_rad: {e}")
        print(f"Available columns in DataFrame: {df.columns.tolist()}")
        raise

def cross_match(df1, df2, ra_col1, dec_col1, ra_col2, dec_col2, max_dist_arcsec=1.0):
    try:
        # Check for NaN in RA/Dec columns and skip rows with NaN
        print(f"Checking for NaN in {ra_col1} and {dec_col1} in df1:")
        print(f"NaN in {ra_col1}: {df1[ra_col1].isna().sum()}")
        print(f"NaN in {dec_col1}: {df1[dec_col1].isna().sum()}")
        print(f"Checking for NaN in {ra_col2} and {dec_col2} in df2:")
        print(f"NaN in {ra_col2}: {df2[ra_col2].isna().sum()}")
        print(f"NaN in {dec_col2}: {df2[dec_col2].isna().sum()}")

        # Drop rows with NaN in RA/Dec columns
        df1_clean = df1.dropna(subset=[ra_col1, dec_col1]).copy()
        df2_clean = df2.dropna(subset=[ra_col2, dec_col2]).copy()
        print(f"After dropping NaN: df1_clean has {len(df1_clean)} rows, df2_clean has {len(df2_clean)}
rows")

        if len(df1_clean) == 0 or len(df2_clean) == 0:
            raise ValueError("One of the DataFrames is empty after dropping NaN values in RA/Dec
columns")

        # Convert to radians
        df1_clean = deg_to_rad(df1_clean, ra_col1, dec_col1)
        df2_clean = deg_to_rad(df2_clean, ra_col2, dec_col2)

        # Perform cross-match
        coords1 = np.array([df1_clean['ra_rad'], df1_clean['dec_rad']]).T
        coords2 = np.array([df2_clean['ra_rad'], df2_clean['dec_rad']]).T
        tree = cKDTree(coords2)
        max_dist = np.radians(max_dist_arcsec / 3600.0)
        dist, idx = tree.query(coords1, k=1, distance_upper_bound=max_dist)
```

```
        matched = dist < max_dist
        df1_matched = df1_clean[matched].copy()
        df2_matched = df2_clean.iloc[idx[matched]].copy()
        df1_matched = df1_matched.reset_index(drop=True)
        df2_matched = df2_matched.reset_index(drop=True)
        return df1_matched, df2_matched
    except KeyError as e:
        print(f"KeyError in cross_match: {e}")
        print(f"df1 columns: {df1.columns.tolist()}")
        print(f"df2 columns: {df2.columns.tolist()}")
        raise
    except ValueError as e:
        print(f"ValueError in cross_match: {e}")
        raise

# Cross-match with VizieR catalogues
# V/154: SDSS DR16 spectroscopic data (uses RA_ICRS, DE_ICRS)
df, sdss_spec_matched = cross_match(df, sdss_spec, 'objra_y', 'objdec', 'RA_ICRS', 'DE_ICRS')
df = pd.concat([df, sdss_spec_matched[['specz', 'flux_Ha', 'flux_Hb', 'flux_OIII_5007', 'flux_NII_6584',
'e_flux_Ha']]], axis=1)
print(f"After V/154 cross-match: {len(df)} galaxies")

# V/147: SDSS DR12 photometric data (uses RAJ2000, DEJ2000)
df, sdss_photo_matched = cross_match(df, sdss_photo, 'objra_y', 'objdec', 'RAJ2000', 'DEJ2000')
df = pd.concat([df, sdss_photo_matched[['petroMag_u', 'petroMag_g', 'petroMag_r', 'petroMag_i',
'petroMag_z', 'petroMagErr_u', 'petroMagErr_g', 'petroMagErr_r', 'petroMagErr_i', 'petroMagErr_z']]],
axis=1)
print(f"After V/147 cross-match: {len(df)} galaxies")

# II/246: 2MASS Extended Source Catalogue (uses RAJ2000, DEJ2000)
df, twomass_matched = cross_match(df, twomass, 'objra_y', 'objdec', 'RAJ2000', 'DEJ2000')
df = pd.concat([df, twomass_matched[['Jmag', 'Hmag', 'Kmag', 'e_Jmag', 'e_Hmag', 'e_Kmag']]], axis=1)
print(f"After II/246 cross-match: {len(df)} galaxies")

# II/356: GAMA DR3 (uses RA, DEC)
df, gama_matched = cross_match(df, gama, 'objra_y', 'objdec', 'RA', 'DEC')
df = pd.concat([df, gama_matched[['Flux_Ha', 'Flux_NII_6584', 'e_Flux_Ha', 'u_mag', 'g_mag', 'r_mag',
'i_mag', 'z_mag']]], axis=1)
print(f"After II/356 cross-match: {len(df)} galaxies")

# Recompute log_SFR_Ha and metallicity from raw fluxes (using V/154 data)
df['Ha_Hb_observed'] = df['flux_Ha'] / df['flux_Hb']
Ha_Hb_intrinsic = 2.86
k_Ha = 2.468
k_Hb = 3.634
df['A_Ha'] = 2.5 * np.log10(df['Ha_Hb_observed'] / Ha_Hb_intrinsic) * (k_Ha / (k_Hb - k_Ha))
df['flux_Ha_corr'] = df['flux_Ha'] * 10**(0.4 * df['A_Ha'])
H0 = 70  # km/s/Mpc
c = 3e5  # km/s
```

```
df['DL'] = (c * df['specz'] / H0) * 3.0856e24  # cm
df['L_Ha'] = df['flux_Ha_corr'] * 4 * np.pi * df['DL']**2  # erg/s
df['log_SFR_Ha_raw'] = np.log10(df['L_Ha'] * 7.9e-42)  # M_sun/yr
df['log_O3N2_raw'] = np.log10((df['flux_OIII_5007'] / df['flux_Hb']) / (df['flux_NII_6584'] / df['flux_Ha']))
df['OH_O3N2_raw'] = 8.533 - 0.214 * df['log_O3N2_raw']

# Add raw fluxes as features
df['log_flux_Ha'] = np.log10(df['flux_Ha'] + 1e-10)
df['log_flux_Hb'] = np.log10(df['flux_Hb'] + 1e-10)
df['log_flux_OIII_5007'] = np.log10(df['flux_OIII_5007'] + 1e-10)
df['log_flux_NII_6584'] = np.log10(df['flux_NII_6584'] + 1e-10)
df['log_e_flux_Ha'] = np.log10(df['e_flux_Ha'] + 1e-10)

# Add photometric colors (from V/147)
df['color_ug'] = df['petroMag_u'] - df['petroMag_g']
df['color_gr'] = df['petroMag_g'] - df['petroMag_r']
df['color_ri'] = df['petroMag_r'] - df['petroMag_i']
df['color_iz'] = df['petroMag_i'] - df['petroMag_z']
df['e_color_ug'] = np.sqrt(df['petroMagErr_u']**2 + df['petroMagErr_g']**2)
df['e_color_gr'] = np.sqrt(df['petroMagErr_g']**2 + df['petroMagErr_r']**2)

# Add 2MASS colors (from II/246)
df['color_JK'] = df['Jmag'] - df['Kmag']
df['e_color_JK'] = np.sqrt(df['e_Jmag']**2 + df['e_Kmag']**2)

# Handle NaN values with mean or median
nan_strategy = 'median'  # Options: 'mean', 'median', 'drop'
print(f"Handling NaN values using strategy: {nan_strategy}")

# Drop rows with missing target
df = df.dropna(subset=["log_SFR_Ha"])
print(f"After dropping rows with missing log_SFR_Ha: {len(df)} rows")

# Drop rows with excessive NaNs (threshold: 80% of columns)
df = df.dropna(axis=0, thresh=int(0.8 * df.shape[1]))
print(f"After dropping rows with excessive NaNs: {len(df)} rows")

# Replace remaining NaN values with mean or median for numeric columns
numeric_cols = df.select_dtypes(include=np.number).columns
print(f"NaN counts in numeric columns before handling:")
print(df[numeric_cols].isna().sum()[df[numeric_cols].isna().sum() > 0])

if nan_strategy == 'mean':
    df[numeric_cols] = df[numeric_cols].fillna(df[numeric_cols].mean())
elif nan_strategy == 'median':
    df[numeric_cols] = df[numeric_cols].fillna(df[numeric_cols].median())
elif nan_strategy == 'drop':
    df = df.dropna(subset=numeric_cols)
else:
```

```
    raise ValueError(f"Invalid nan_strategy: {nan_strategy}. Choose 'mean', 'median', or 'drop'.")

print(f"NaN counts in numeric columns after handling:")
print(df[numeric_cols].isna().sum()[df[numeric_cols].isna().sum() > 0])
print(f"After handling NaN values: {len(df)} rows")

# Step 1: Cosmo-Rank Construction
gz_df = df.copy()
morph_cols = ['P(CD)', 'P(E)', 'P(S0)', 'P(Sa)', 'P(Sab)', 'P(Sb)', 'P(Sbc)',
              'P(Sc)', 'P(Scd)', 'P(Sd)', 'P(Sdm)', 'P(Sm)', 'P(Irr)']
gz_df['morph_sum'] = gz_df[morph_cols].sum(axis=1)
gz_df['Num_w'] = gz_df['conf_prob']
scaler_rank = MinMaxScaler()
gz_df[['morph_sum_norm', 'Num_w_norm', 'nsa_z_norm']] = scaler_rank.fit_transform(
    gz_df[['morph_sum', 'Num_w', 'nsa_z']]
)
gz_df['cosmo_rank'] = (0.5 * gz_df['Num_w_norm'] + 0.3 * gz_df['morph_sum_norm'] + 0.2 *
gz_df['nsa_z_norm'])
df['cosmo_rank'] = gz_df['cosmo_rank']

# Step 2: L_cosmo(s) Construction
alpha = -1.5
M_star = df['log_Mass_gas'].median()
df['a_n'] = (10 ** df['log_Mass_gas'])**(1 + alpha) * np.exp(-10 ** df['log_Mass_gas'] / (10 ** M_star))
bins = pd.qcut(df['Re_kpc'], q=20, labels=False, duplicates='drop') + 1
df['z_bin'] = bins
n_bins = df['z_bin'].nunique()
print(f"Number of unique bins after qcut: {n_bins}")
print("Bin distribution for Re_kpc (after qcut):")
print(df['z_bin'].value_counts().sort_index())

s_vals = [0.5, 1.0, 1.5, 2.0]
s_range = np.linspace(0.5, 2, 50)
for s in s_vals:
    col_name = f'L_cosmo_s_{s:.1f}'
    df[col_name] = df.groupby('z_bin')['a_n'].transform('mean') / (df['z_bin'] ** s)

print("Columns after creating L_cosmo_s*:")
print(df.columns.tolist())

print("Checking for NaN/infinite values in cosmo_rank and L_cosmo_s_1.0:")
print("cosmo_rank NaN count:", df['cosmo_rank'].isna().sum())
print("L_cosmo_s_1.0 NaN count:", df['L_cosmo_s_1.0'].isna().sum())
print("cosmo_rank infinite count:", np.isinf(df['cosmo_rank']).sum())
print("L_cosmo_s_1.0 infinite count:", np.isinf(df['L_cosmo_s_1.0']).sum())

df['cosmo_rank_L'] = df['cosmo_rank'] * df['L_cosmo_s_1.0']
df['cosmo_rank_L'] = df['cosmo_rank_L'].replace([np.inf, -np.inf], np.nan).fillna(0)
print("cosmo_rank_L created, NaN count:", df['cosmo_rank_L'].isna().sum())
```

```
# Plot L_cosmo(s) vs s
grouped_means = df.groupby('z_bin')['a_n'].mean().reindex(range(1, n_bins + 1), fill_value=0)
L_vals = [grouped_means / (np.arange(1, n_bins + 1) ** s) for s in s_range]
L_vals = np.array([sum(l) for l in L_vals])
plt.plot(s_range, L_vals)
plt.xlabel("s")
plt.ylabel("L_cosmo(s)")
plt.title("L_cosmo(s) Behavior")
plt.savefig("L_cosmo_curve.png")
plt.close()

# Scatter plot of L_cosmo_s_1.0 vs log_SFR_Ha
plt.scatter(df['L_cosmo_s_1.0'], df['log_SFR_Ha'], alpha=0.5)
plt.xlabel("L_cosmo_s_1.0")
plt.ylabel("log_SFR_Ha")
plt.title("L_cosmo_s_1.0 vs log_SFR_Ha")
plt.savefig("L_cosmo_s1_vs_SFR.png")
plt.close()

# Step 3: Define features and target
target = "log_SFR_Ha"
base_features = [
    "log_Mass_gas", "log_Mass", "Av_gas_Re", "Av_ssp_Re",
    "OH_O3N2_cen", "OH_T04_cen", "OH_dop_cen", "Age_LW_Re_fit",
    "ZH_LW_Re_fit", "Re_kpc", "vel_disp_Ha_cen", "Lambda_Re",
    "Sigma_Mass_Re", "EW_Ha_cen", "Ha_Hb_cen"
]
df["mass_metallicity"] = df["log_Mass"] * df["OH_O3N2_cen"]
df["dust_metallicity"] = df["Av_gas_Re"] * df["OH_T04_cen"]
df["disp_mass_ratio"] = df["vel_disp_Ha_cen"] / (df["Sigma_Mass_Re"] + 1e-5)
df["age_metallicity"] = df["Age_LW_Re_fit"] * df["ZH_LW_Re_fit"]
df["sqrt_Re_kpc"] = np.sqrt(df["Re_kpc"] + 1e-5)
df["log_mass"] = np.log(df["log_Mass_gas"] + 1e-5)
df["BSD_likelihood"] = (
    df["log_Mass_gas"] * df["OH_O3N2_cen"] /
    (df["Av_gas_Re"] + 1e-5 + df["Age_LW_Re_fit"])
)
df["cosmo_rank_mass"] = df["cosmo_rank"] * df["log_Mass"]
df["L_cosmo_s1_mass"] = df["L_cosmo_s_1.0"] * df["log_Mass"]
df["cosmo_rank_EW"] = df["cosmo_rank"] * df["EW_Ha_cen"]
df["L_cosmo_s1_EW"] = df["L_cosmo_s_1.0"] * df["EW_Ha_cen"]
df["cosmo_rank_scaled"] = df["cosmo_rank"] * 20
df["L_cosmo_s1_scaled"] = df["L_cosmo_s_1.0"] * 20
df["cosmo_rank_L_mass"] = df["cosmo_rank_L"] * df["log_Mass"]
df["L_cosmo_s1_metallicity"] = df["L_cosmo_s_1.0"] * df["OH_O3N2_cen"]
features = base_features + [
    "mass_metallicity", "dust_metallicity", "disp_mass_ratio",
    "age_metallicity", "sqrt_Re_kpc", "log_mass", "BSD_likelihood",
```

```
    "cosmo_rank", "L_cosmo_s_0.5", "L_cosmo_s_1.0", "L_cosmo_s_1.5",
    "L_cosmo_s_2.0", "cosmo_rank_L", "cosmo_rank_mass", "L_cosmo_s1_mass",
    "cosmo_rank_EW", "L_cosmo_s1_EW", "cosmo_rank_scaled", "L_cosmo_s1_scaled",
    "cosmo_rank_L_mass", "L_cosmo_s1_metallicity",
    "log_flux_Ha", "log_flux_Hb", "log_flux_OIII_5007", "log_flux_NII_6584",
    "log_e_flux_Ha", "log_SFR_Ha_raw", "OH_O3N2_raw",
    "color_ug", "color_gr", "color_ri", "color_iz", "e_color_ug", "e_color_gr",
    "color_JK", "e_color_JK"
]

X = df[features]
y = df[target]

# Train/test split with n_jobs=2
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42, n_jobs=2)

# Normalize features
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# Optuna optimization for Random Forest
def objective_rf(trial):
    params = {
        'n_estimators': trial.suggest_int('n_estimators', 100, 500),
        'max_depth': trial.suggest_int('max_depth', 5, 20),
        'min_samples_split': trial.suggest_int('min_samples_split', 2, 10),
        'random_state': 42,
        'n_jobs': 2  # Use 2 processors
    }
    model = RandomForestRegressor(**params)
    # 5-fold cross-validation
    cv = KFold(n_splits=5, shuffle=True, random_state=42)
    scores = cross_val_score(model, X_train_scaled, y_train, cv=cv, scoring='r2', n_jobs=2)
    return scores.mean()

study_rf = optuna.create_study(direction='maximize')
study_rf.optimize(objective_rf, n_trials=20, n_jobs=2)
best_params_rf = study_rf.best_params
print("Best Random Forest Parameters:", best_params_rf)
print("Best Random Forest CV R²:", study_rf.best_value)

# Train Random Forest with best parameters
rf = RandomForestRegressor(**best_params_rf, random_state=42, n_jobs=2)
rf.fit(X_train_scaled, y_train)
y_pred_rf = rf.predict(X_test_scaled)
r2_rf = r2_score(y_test, y_pred_rf)

# Cross-validation for final RF model
```

```
cv_scores_rf = cross_val_score(rf, X_train_scaled, y_train, cv=5, scoring='r2', n_jobs=2)
print(f"Random Forest 5-Fold CV R²: Mean = {cv_scores_rf.mean():.4f}, Std = {cv_scores_rf.std():.4f}")

# Optuna optimization for Gradient Boosting
def objective_gb(trial):
    params = {
        'n_estimators': trial.suggest_int('n_estimators', 100, 500),
        'learning_rate': trial.suggest_float('learning_rate', 0.01, 0.2, log=True),
        'max_depth': trial.suggest_int('max_depth', 3, 10),
        'random_state': 42,
        'n_jobs': 2  # Use 2 processors
    }
    model = GradientBoostingRegressor(**params)
    cv = KFold(n_splits=5, shuffle=True, random_state=42)
    scores = cross_val_score(model, X_train_scaled, y_train, cv=cv, scoring='r2', n_jobs=2)
    return scores.mean()

study_gb = optuna.create_study(direction='maximize')
study_gb.optimize(objective_gb, n_trials=20, n_jobs=2)
best_params_gb = study_gb.best_params
print("Best Gradient Boosting Parameters:", best_params_gb)
print("Best Gradient Boosting CV R²:", study_gb.best_value)

# Train Gradient Boosting with best parameters
gb = GradientBoostingRegressor(**best_params_gb, random_state=42, n_jobs=2)
gb.fit(X_train_scaled, y_train)
y_pred_gb = gb.predict(X_test_scaled)
r2_gb = r2_score(y_test, y_pred_gb)

# Cross-validation for final GB model
cv_scores_gb = cross_val_score(gb, X_train_scaled, y_train, cv=5, scoring='r2', n_jobs=2)
print(f"Gradient Boosting 5-Fold CV R²: Mean = {cv_scores_gb.mean():.4f}, Std =
{cv_scores_gb.std():.4f}")

print(f"R^2 Score (Random Forest, Test Set): {r2_rf:.4f}")
print(f"R^2 Score (Gradient Boosting, Test Set): {r2_gb:.4f}")

# SHAP for feature selection
explainer_rf = shap.Explainer(rf, X_train_scaled)
shap_values_rf = explainer_rf(X_test_scaled)
explainer_gb = shap.Explainer(gb, X_train_scaled)
shap_values_gb = explainer_gb(X_test_scaled)
shap.summary_plot(shap_values_rf, X_test_scaled, plot_type="bar", show=False)
plt.title("SHAP Summary - Random Forest")
plt.tight_layout()
plt.savefig("shap_rf_summary.png")
plt.clf()
shap.summary_plot(shap_values_gb, X_test_scaled, plot_type="bar", show=False)
plt.title("SHAP Summary - Gradient Boosting")
```

```
plt.tight_layout()
plt.savefig("shap_gb_summary.png")
plt.clf()

# Feature selection based on SHAP
shap_values_rf_mean = np.abs(shap_values_rf.values).mean(axis=0)
feature_importance = pd.DataFrame({"feature": features, "importance": shap_values_rf_mean})
feature_importance = feature_importance.sort_values("importance", ascending=False)
top_features = feature_importance["feature"].head(10).tolist()
bsd_features = ["cosmo_rank", "L_cosmo_s_1.0", "cosmo_rank_L", "cosmo_rank_scaled",
"L_cosmo_s1_scaled", "cosmo_rank_L_mass", "L_cosmo_s1_metallicity"]
for bsd_feature in bsd_features:
    if bsd_feature not in top_features:
        top_features.append(bsd_feature)
X_selected = df[top_features]
X_train_selected, X_test_selected, _, _ = train_test_split(X_selected, y, test_size=0.2, random_state=42,
n_jobs=2)
X_train_selected_scaled = scaler.fit_transform(X_train_selected)
X_test_selected_scaled = scaler.transform(X_test_selected)

# Symbolic Regression with gplearn
symbolic_model = SymbolicRegressor(
    population_size=3000,
    generations=150,
    stopping_criteria=0.01,
    p_crossover=0.7,
    p_subtree_mutation=0.1,
    p_hoist_mutation=0.05,
    p_point_mutation=0.1,
    max_samples=0.9,
    verbose=1,
    parsimony_coefficient=0.0001,
    random_state=42,
    n_jobs=2,  # Use 2 processors
    function_set=('add', 'sub', 'mul', 'div', 'sqrt', 'log', 'sin', 'cos')
)
symbolic_model.fit(X_train_selected_scaled, y_train)
y_pred_sym = symbolic_model.predict(X_test_selected_scaled)
r2_sym = r2_score(y_test, y_pred_sym)
print(f"gplearn R²: {r2_sym:.4f}")
print("Symbolic Expression:")
print(symbolic_model._program)

# Cross-validation for symbolic model
cv_scores_sym = cross_val_score(symbolic_model, X_train_selected_scaled, y_train, cv=5, scoring='r2',
n_jobs=2)
print(f"Symbolic Regression 5-Fold CV R²: Mean = {cv_scores_sym.mean():.4f}, Std =
{cv_scores_sym.std():.4f}")
```

```
# Save symbolic expression as polynomial fit plot and print the equation
x = np.linspace(min(y_test), max(y_test), 500)
y_expr = symbolic_model.predict(scaler.transform(np.tile(X_test_selected.mean().values, (500,1))))
p = Polynomial.fit(y_test, y_pred_sym, deg=3)
# Extract polynomial coefficients
coeffs = p.coef
print("Polynomial Fit Equation (degree 3):")
print(f"y = {coeffs[0]:.4f} + {coeffs[1]:.4f}*x + {coeffs[2]:.4f}*x^2 + {coeffs[3]:.4f}*x^3")
# Plot the polynomial fit
plt.plot(*p.linspace(), label="Polynomial Fit")
plt.scatter(y_test, y_pred_sym, s=10, alpha=0.5,
```

can you write into the model to either skip the NaN or calculate the mean or median to replace the NaN?

(myenv) root@BigsbyGaming:/home/pmqr7# python STAR.py Columns in merged_data.csv: ['mangaid', 'nsa_sersic_mass', 'nsa_z', 'OBJDEC_x', 'objra_x', 'SFR_PETRORAD_R', 'zcomp_1Mpc', 'zcomp_5Mpc', 'footprint_1', 'footprint_5', 'PLATEIFU', 'OBJRA', 'OBJDEC_y', 'LOGMSTARS', 'SINI', 'VOPT', 'SESSION', 'EXP', 'RMS', 'LOGHILIM200KMS', 'PEAK', 'SNR', 'FHI', 'EFHI', 'LOGMHI', 'VHI', 'EV', 'WM50', 'WP50', 'WP20', 'W2P50', 'WF50', 'DW', 'PR', 'PL', 'AR', 'BR', 'AL', 'BL', 'NEGDET', 'BLSTRUCT', 'conflag', 'conf_prob', 'name', 'plate', 'ifudsgn', 'plateifu', 'objra_y', 'objdec', 'log_SFR_Ha', 'FoV', 'Re_kpc', 'e_log_Mass', 'e_log_SFR_Ha', 'log_Mass', 'log_SFR_ssp', 'log_NII_Ha_cen', 'e_log_NII_Ha_cen', 'log_OIII_Hb_cen', 'e_log_OIII_Hb_cen', 'log_SII_Ha_cen', 'e_log_SII_Ha_cen', 'log_OII_Hb_cen', 'e_log_OII_Hb_cen', 'EW_Ha_cen', 'e_EW_Ha_cen', 'ZH_LW_Re_fit', 'e_ZH_LW_Re_fit', 'alpha_ZH_LW_Re_fit', 'e_alpha_ZH_LW_Re_fit', 'ZH_MW_Re_fit', 'e_ZH_MW_Re_fit', 'alpha_ZH_MW_Re_fit', 'e_alpha_ZH_MW_Re_fit', 'Age_LW_Re_fit', 'e_Age_LW_Re_fit', 'alpha_Age_LW_Re_fit', 'e_alpha_Age_LW_Re_fit', 'Age_MW_Re_fit', 'e_Age_MW_Re_fit', 'alpha_Age_MW_Re_fit', 'e_alpha_Age_MW_Re_fit', 'Re_arc', 'DL', 'DA', 'PA', 'ellip', 'log_Mass_gas', 'vel_sigma_Re', 'e_vel_sigma_Re', 'log_SFR_SF', 'log_SFR_D_C', 'OH_O3N2_cen', 'e_OH_O3N2_cen', 'OH_N2_cen', 'e_OH_N2_cen', 'OH_ONS_cen', 'e_OH_ONS_cen', 'OH_R23_cen', 'e_OH_R23_cen', 'OH_pyqz_cen', 'e_OH_pyqz_cen', 'OH_t2_cen', 'e_OH_t2_cen', 'OH_M08_cen', 'e_OH_M08_cen', 'OH_T04_cen', 'e_OH_T04_cen', 'OH_dop_cen', 'e_OH_dop_cen', 'OH_O3N2_EPM09_cen', 'e_OH_O3N2_EPM09_cen', 'log_OI_Ha_cen', 'e_log_OI_Ha_cen', 'Ha_Hb_cen', 'e_Ha_Hb_cen', 'log_NII_Ha_Re', 'e_log_NII_Ha_Re', 'log_OIII_Hb_Re', 'e_log_OIII_Hb_Re', 'log_SII_Ha_Re', 'e_log_SII_Ha_Re', 'log_OII_Hb_Re', 'e_log_OII_Hb_Re', 'log_OI_Ha_Re', 'e_log_OI_Ha_Re', 'EW_Ha_Re', 'e_EW_Ha_Re', 'Ha_Hb_Re', 'e_Ha_Hb_Re', 'log_NII_Ha_ALL', 'e_log_NII_Ha_ALL', 'log_OIII_Hb_ALL', 'e_log_OIII_Hb_ALL', 'log_SII_Ha_ALL', 'e_log_SII_Ha_ALL', 'log_OII_Hb_ALL', 'e_log_OII_Hb_ALL', 'log_OI_Ha_ALL', 'e_log_OI_Ha_ALL', 'EW_Ha_ALL', 'e_EW_Ha_ALL', 'Ha_Hb_ALL', 'Sigma_Mass_cen', 'e_Sigma_Mass_cen', 'Sigma_Mass_Re', 'e_Sigma_Mass_Re', 'Sigma_Mass_ALL', 'e_Sigma_Mass_ALL', 'T30', 'ZH_T30', 'ZH_Re_T30', 'a_ZH_T30', 'T40', 'ZH_T40', 'ZH_Re_T40', 'a_ZH_T40', 'T50', 'ZH_T50', 'ZH_Re_T50', 'a_ZH_T50', 'T60', 'ZH_T60', 'ZH_Re_T60', 'a_ZH_T60', 'T70',

'ZH_T70', 'ZH_Re_T70', 'a_ZH_T70', 'T80', 'ZH_T80', 'ZH_Re_T80', 'a_ZH_T80', 'T90',
'ZH_T90', 'ZH_Re_T90', 'a_ZH_T90', 'T95', 'ZH_T95', 'ZH_Re_T95', 'a_ZH_T95', 'T99',
'ZH_T99', 'ZH_Re_T99', 'a_ZH_T99', 'log_Mass_gas_Av_gas_OH',
'log_Mass_gas_Av_ssp_OH', 'vel_ssp_2', 'e_vel_ssp_2', 'vel_Ha_2', 'e_vel_Ha_2',
'vel_ssp_1', 'e_vel_ssp_1', 'vel_Ha_1', 'e_vel_Ha_1', 'log_SFR_ssp_100Myr',
'log_SFR_ssp_10Myr', 'vel_disp_Ha_cen', 'vel_disp_ssp_cen', 'vel_disp_Ha_1Re',
'vel_disp_ssp_1Re', 'log_Mass_in_Re', 'ML_int', 'ML_avg', 'F_Ha_cen', 'e_F_Ha_cen',
'R50_kpc_V', 'e_R50_kpc_V', 'R50_kpc_Mass', 'e_R50_kpc_Mass',
'log_Mass_corr_in_R50_V', 'e_log_Mass_corr_in_R50_V',
'log_Mass_gas_Av_gas_log_log', 'Av_gas_Re', 'e_Av_gas_Re', 'Av_ssp_Re',
'e_Av_ssp_Re', 'Lambda_Re', 'e_Lambda_Re', 'nsa_redshift', 'nsa_mstar', 'nsa_inclination',
'flux_[OII]3726.03_Re_fit', 'e_flux_[OII]3726.03_Re_fit', 'flux_[OII]3726.03_alpha_fit',
'e_flux_[OII]3726.03_alpha_fit', 'flux_[OII]3728.82_Re_fit', 'e_flux_[OII]3728.82_Re_fit',
'flux_[OII]3728.82_alpha_fit', 'e_flux_[OII]3728.82_alpha_fit', 'flux_HI3734.37_Re_fit',
'e_flux_HI3734.37_Re_fit', 'flux_HI3734.37_alpha_fit', 'e_flux_HI3734.37_alpha_fit',
'flux_HI3797.9_Re_fit', 'e_flux_HI3797.9_Re_fit', 'flux_HI3797.9_alpha_fit',
'e_flux_HI3797.9_alpha_fit', 'flux_HeI3888.65_Re_fit', 'e_flux_HeI3888.65_Re_fit',
'flux_HeI3888.65_alpha_fit', 'e_flux_HeI3888.65_alpha_fit', 'flux_HI3889.05_Re_fit',
'e_flux_HI3889.05_Re_fit', 'flux_HI3889.05_alpha_fit', 'e_flux_HI3889.05_alpha_fit',
'flux_HeI3964.73_Re_fit', 'e_flux_HeI3964.73_Re_fit', 'flux_HeI3964.73_alpha_fit',
'e_flux_HeI3964.73_alpha_fit', 'flux_[NeIII]3967.46_Re_fit', 'e_flux_[NeIII]3967.46_Re_fit',
'flux_[NeIII]3967.46_alpha_fit', 'e_flux_[NeIII]3967.46_alpha_fit', 'flux_CaII3968.47_Re_fit',
'e_flux_CaII3968.47_Re_fit', 'flux_CaII3968.47_alpha_fit', 'e_flux_CaII3968.47_alpha_fit',
'flux_Hepsilon3970.07_Re_fit', 'e_flux_Hepsilon3970.07_Re_fit',
'flux_Hepsilon3970.07_alpha_fit', 'e_flux_Hepsilon3970.07_alpha_fit',
'flux_Hdelta4101.77_Re_fit', 'e_flux_Hdelta4101.77_Re_fit', 'flux_Hdelta4101.77_alpha_fit',
'e_flux_Hdelta4101.77_alpha_fit', 'flux_Hgamma4340.49_Re_fit',
'e_flux_Hgamma4340.49_Re_fit', 'flux_Hgamma4340.49_alpha_fit',
'e_flux_Hgamma4340.49_alpha_fit', 'flux_Hbeta4861.36_Re_fit',
'e_flux_Hbeta4861.36_Re_fit', 'flux_Hbeta4861.36_alpha_fit',
'e_flux_Hbeta4861.36_alpha_fit', 'flux_[OIII]4958.91_Re_fit', 'e_flux_[OIII]4958.91_Re_fit',
'flux_[OIII]4958.91_alpha_fit', 'e_flux_[OIII]4958.91_alpha_fit', 'flux_[OIII]5006.84_Re_fit',
'e_flux_[OIII]5006.84_Re_fit', 'flux_[OIII]5006.84_alpha_fit', 'e_flux_[OIII]5006.84_alpha_fit',
'flux_HeI5015.68_Re_fit', 'e_flux_HeI5015.68_Re_fit', 'flux_HeI5015.68_alpha_fit',
'e_flux_HeI5015.68_alpha_fit', 'flux_[NI]5197.9_Re_fit', 'e_flux_[NI]5197.9_Re_fit',
'flux_[NI]5197.9_alpha_fit', 'e_flux_[NI]5197.9_alpha_fit', 'flux_[NI]5200.26_Re_fit',
'e_flux_[NI]5200.26_Re_fit', 'flux_[NI]5200.26_alpha_fit', 'e_flux_[NI]5200.26_alpha_fit',
'flux_HeI5876.0_Re_fit', 'e_flux_HeI5876.0_Re_fit', 'flux_HeI5876.0_alpha_fit',
'e_flux_HeI5876.0_alpha_fit', 'flux_NaI5889.95_Re_fit', 'e_flux_NaI5889.95_Re_fit',
'flux_NaI5889.95_alpha_fit', 'e_flux_NaI5889.95_alpha_fit', 'flux_NaI5895.92_Re_fit',
'e_flux_NaI5895.92_Re_fit', 'flux_NaI5895.92_alpha_fit', 'e_flux_NaI5895.92_alpha_fit',
'flux_[OI]6300.3_Re_fit', 'e_flux_[OI]6300.3_Re_fit', 'flux_[OI]6300.3_alpha_fit',

'e_flux_[OI]6300.3_alpha_fit', 'flux_[NII]6548.05_Re_fit', 'e_flux_[NII]6548.05_Re_fit',
'flux_[NII]6548.05_alpha_fit', 'e_flux_[NII]6548.05_alpha_fit', 'flux_Halpha6562.85_Re_fit',
'e_flux_Halpha6562.85_Re_fit', 'flux_Halpha6562.85_alpha_fit',
'e_flux_Halpha6562.85_alpha_fit', 'flux_[NII]6583.45_Re_fit', 'e_flux_[NII]6583.45_Re_fit',
'flux_[NII]6583.45_alpha_fit', 'e_flux_[NII]6583.45_alpha_fit', 'flux_[SII]6716.44_Re_fit',
'e_flux_[SII]6716.44_Re_fit', 'flux_[SII]6716.44_alpha_fit', 'e_flux_[SII]6716.44_alpha_fit',
'flux_[SII]6730.82_Re_fit', 'e_flux_[SII]6730.82_Re_fit', 'flux_[SII]6730.82_alpha_fit',
'e_flux_[SII]6730.82_alpha_fit', 'flux_[ArIII]7135.8_Re_fit', 'e_flux_[ArIII]7135.8_Re_fit',
'flux_[ArIII]7135.8_alpha_fit', 'e_flux_[ArIII]7135.8_alpha_fit', 'flux_HI9014.91_Re_fit',
'e_flux_HI9014.91_Re_fit', 'flux_HI9014.91_alpha_fit', 'e_flux_HI9014.91_alpha_fit',
'flux_[SIII]9069.0_Re_fit', 'e_flux_[SIII]9069.0_Re_fit', 'flux_[SIII]9069.0_alpha_fit',
'e_flux_[SIII]9069.0_alpha_fit', 'flux_[FeII]9470.93_Re_fit', 'e_flux_[FeII]9470.93_Re_fit',
'flux_[FeII]9470.93_alpha_fit', 'e_flux_[FeII]9470.93_alpha_fit', 'flux_[SIII]9531.1_Re_fit',
'e_flux_[SIII]9531.1_Re_fit', 'flux_[SIII]9531.1_alpha_fit', 'e_flux_[SIII]9531.1_alpha_fit',
'OH_Mar13_N2_Re_fit', 'e_OH_Mar13_N2_Re_fit', 'OH_Mar13_N2_alpha_fit',
'e_OH_Mar13_N2_alpha_fit', 'OH_Mar13_O3N2_Re_fit', 'e_OH_Mar13_O3N2_Re_fit',
'OH_Mar13_O3N2_alpha_fit', 'e_OH_Mar13_O3N2_alpha_fit', 'OH_T04_Re_fit',
'e_OH_T04_Re_fit', 'OH_T04_alpha_fit', 'e_OH_T04_alpha_fit', 'OH_Pet04_N2_lin_Re_fit',
'e_OH_Pet04_N2_lin_Re_fit', 'OH_Pet04_N2_lin_alpha_fit',
'e_OH_Pet04_N2_lin_alpha_fit', 'OH_Pet04_N2_poly_Re_fit',
'e_OH_Pet04_N2_poly_Re_fit', 'OH_Pet04_N2_poly_alpha_fit',
'e_OH_Pet04_N2_poly_alpha_fit', 'OH_Pet04_O3N2_Re_fit', 'e_OH_Pet04_O3N2_Re_fit',
'OH_Pet04_O3N2_alpha_fit', 'e_OH_Pet04_O3N2_alpha_fit', 'OH_Kew02_N2O2_Re_fit',
'e_OH_Kew02_N2O2_Re_fit', 'OH_Kew02_N2O2_alpha_fit',
'e_OH_Kew02_N2O2_alpha_fit', 'OH_Pil10_ONS_Re_fit', 'e_OH_Pil10_ONS_Re_fit',
'OH_Pil10_ONS_alpha_fit', 'e_OH_Pil10_ONS_alpha_fit', 'OH_Pil10_ON_Re_fit',
'e_OH_Pil10_ON_Re_fit', 'OH_Pil10_ON_alpha_fit', 'e_OH_Pil10_ON_alpha_fit',
'OH_Pil11_NS_Re_fit', 'e_OH_Pil11_NS_Re_fit', 'OH_Pil11_NS_alpha_fit',
'e_OH_Pil11_NS_alpha_fit', 'OH_Cur20_RS32_Re_fit', 'e_OH_Cur20_RS32_Re_fit',
'OH_Cur20_RS32_alpha_fit', 'e_OH_Cur20_RS32_alpha_fit', 'OH_Cur20_R3_Re_fit',
'e_OH_Cur20_R3_Re_fit', 'OH_Cur20_R3_alpha_fit', 'e_OH_Cur20_R3_alpha_fit',
'OH_Cur20_O3O2_Re_fit', 'e_OH_Cur20_O3O2_Re_fit', 'OH_Cur20_O3O2_alpha_fit',
'e_OH_Cur20_O3O2_alpha_fit', 'OH_Cur20_S2_Re_fit', 'e_OH_Cur20_S2_Re_fit',
'OH_Cur20_S2_alpha_fit', 'e_OH_Cur20_S2_alpha_fit', 'OH_Cur20_R2_Re_fit',
'e_OH_Cur20_R2_Re_fit', 'OH_Cur20_R2_alpha_fit', 'e_OH_Cur20_R2_alpha_fit',
'OH_Cur20_N2_Re_fit', 'e_OH_Cur20_N2_Re_fit', 'OH_Cur20_N2_alpha_fit',
'e_OH_Cur20_N2_alpha_fit', 'OH_Cur20_R23_Re_fit', 'e_OH_Cur20_R23_Re_fit',
'OH_Cur20_R23_alpha_fit', 'e_OH_Cur20_R23_alpha_fit', 'OH_Cur20_O3N2_Re_fit',
'e_OH_Cur20_O3N2_Re_fit', 'OH_Cur20_O3N2_alpha_fit', 'e_OH_Cur20_O3N2_alpha_fit',
'OH_Cur20_O3S2_Re_fit', 'e_OH_Cur20_O3S2_Re_fit', 'OH_Cur20_O3S2_alpha_fit',
'e_OH_Cur20_O3S2_alpha_fit', 'OH_KK04_Re_fit', 'e_OH_KK04_Re_fit',
'OH_KK04_alpha_fit', 'e_OH_KK04_alpha_fit', 'OH_Pil16_R_Re_fit',

'e_OH_Pil16_R_Re_fit', 'OH_Pil16_R_alpha_fit', 'e_OH_Pil16_R_alpha_fit',
'OH_Pil16_S_Re_fit', 'e_OH_Pil16_S_Re_fit', 'OH_Pil16_S_alpha_fit',
'e_OH_Pil16_S_alpha_fit', 'OH_Ho_Re_fit', 'e_OH_Ho_Re_fit', 'OH_Ho_alpha_fit',
'e_OH_Ho_alpha_fit', 'U_Dors_O32_Re_fit', 'e_U_Dors_O32_Re_fit',
'U_Dors_O32_alpha_fit', 'e_U_Dors_O32_alpha_fit', 'U_Dors_S_Re_fit',
'e_U_Dors_S_Re_fit', 'U_Dors_S_alpha_fit', 'e_U_Dors_S_alpha_fit',
'U_Mor16_O23_fs_Re_fit', 'e_U_Mor16_O23_fs_Re_fit', 'U_Mor16_O23_fs_alpha_fit',
'e_U_Mor16_O23_fs_alpha_fit', 'U_Mor16_O23_ts_Re_fit', 'e_U_Mor16_O23_ts_Re_fit',
'U_Mor16_O23_ts_alpha_fit', 'e_U_Mor16_O23_ts_alpha_fit', 'NH_Pil16_R_Re_fit',
'e_NH_Pil16_R_Re_fit', 'NH_Pil16_R_alpha_fit', 'e_NH_Pil16_R_alpha_fit',
'NO_Pil16_R_Re_fit', 'e_NO_Pil16_R_Re_fit', 'NO_Pil16_R_alpha_fit',
'e_NO_Pil16_R_alpha_fit', 'NO_Pil16_Ho_R_Re_fit', 'e_NO_Pil16_Ho_R_Re_fit',
'NO_Pil16_Ho_R_alpha_fit', 'e_NO_Pil16_Ho_R_alpha_fit', 'NO_Pil16_N2_R2_Re_fit',
'e_NO_Pil16_N2_R2_Re_fit', 'NO_Pil16_N2_R2_alpha_fit', 'e_NO_Pil16_N2_R2_alpha_fit',
'Ne_Oster_S_Re_fit', 'e_Ne_Oster_S_Re_fit', 'Ne_Oster_S_alpha_fit',
'e_Ne_Oster_S_alpha_fit', 'Hd_Re_fit', 'e_Hd_Re_fit', 'Hd_alpha_fit', 'e_Hd_alpha_fit',
'Hb_Re_fit', 'e_Hb_Re_fit', 'Hb_alpha_fit', 'e_Hb_alpha_fit', 'Mgb_Re_fit', 'e_Mgb_Re_fit',
'Mgb_alpha_fit', 'e_Mgb_alpha_fit', 'Fe5270_Re_fit', 'e_Fe5270_Re_fit', 'Fe5270_alpha_fit',
'e_Fe5270_alpha_fit', 'Fe5335_Re_fit', 'e_Fe5335_Re_fit', 'Fe5335_alpha_fit',
'e_Fe5335_alpha_fit', 'D4000_Re_fit1', 'e_D4000_Re_fit', 'D4000_alpha_fit',
'e_D4000_alpha_fit', 'Hdmod_Re_fit', 'e_Hdmod_Re_fit', 'Hdmod_alpha_fit',
'e_Hdmod_alpha_fit', 'Hg_Re_fit', 'e_Hg_Re_fit', 'Hg_alpha_fit', 'e_Hg_alpha_fit',
'u_band_mag', 'u_band_mag_error', 'u_band_abs_mag', 'u_band_abs_mag_error',
'g_band_mag', 'g_band_mag_error', 'g_band_abs_mag', 'g_band_abs_mag_error',
'r_band_mag', 'r_band_mag_error', 'r_band_abs_mag', 'r_band_abs_mag_error',
'i_band_mag', 'i_band_mag_error', 'i_band_abs_mag', 'i_band_abs_mag_error',
'B_band_mag', 'B_band_mag_error', 'B_band_abs_mag', 'B_band_abs_mag_error',
'V_band_mag', 'V_band_mag_error', 'V_band_abs_mag', 'V_band_abs_mag_error',
'RJ_band_mag', 'RJ_band_mag_error', 'RJ_band_abs_mag', 'RJ_band_abs_mag_error',
'R50', 'error_R50', 'R90', 'error_R90', 'C', 'e_C', 'B-V', 'error_B-V', 'B-R', 'error_B-R',
'log_Mass_phot', 'e_log_Mass_phot', 'V-band_SB_at_Re', 'error_V-band_SB_at_Re',
'V-band_SB_at_R_50', 'error_V-band_SB_at_R_50', 'nsa_sersic_n_morph', 'u-g', 'g-r', 'r-i',
'i-z', 'P(CD)', 'P(E)', 'P(S0)', 'P(Sa)', 'P(Sab)', 'P(Sb)', 'P(Sbc)', 'P(Sc)', 'P(Scd)', 'P(Sd)',
'P(Sdm)', 'P(Sm)', 'P(Irr)', 'best_type_n', 'best_type', 'nsa_nsaid', 'Vmax_w', 'Num_w',
'QCFLAG'] Columns in sdss_spec (V/154): ['objID', 'RA_ICRS', 'DE_ICRS', 'mode', 'class',
'clean', 'e_RA_ICRS', 'e_DE_ICRS', 'umag', 'gmag', 'rmag', 'imag', 'zmag', 'e_umag',
'e_gmag', 'e_rmag', 'e_imag', 'e_zmag', 'zsp', 'e_zsp', 'f_zsp', 'pmRA', 'pmDE', 'zph', 'e_zph',
'<zph>', 'Q', 'SDSS16', 'Sp-ID', 'MJD', 'zs', 'zPrad', 'zPA', 'zoRA', 'zoDE', 'zMJD', 'zFlags',
'zdVrad', 'zdVell', 'zc', 'Vdisp', 'us', 'uPrad', 'uPA', 'uoRA', 'uoDE', 'uMJD', 'uFlags', 'udVrad',
'udVell', 'uc', 'subCl', 'Status', 'spType', 'SpObjID', 'spInst', 'spCl', 'sigRA', 'sigDE', 'rs', 'rPrad',
'rPA', 'roRA', 'roDE', 'rMJD', 'rFlags', 'rdVrad', 'rdVell', 'rc', 'parentID', 'N', 'M', 'iPrad', 'iPA',
'ioRA', 'ioDE', 'iMJD', 'iFlags', 'idVrad', 'idVell', 'ic', 'gs', 'gPrad', 'gPA', 'goRA', 'goDE', 'gMJD',

'gFlags', 'gdVrad', 'gdVell', 'gc', 'flags', 'fieldID', 'e_zPrad', 'e_Vdisp', 'e_uPrad', 'e_rPrad', 'e_pmRA', 'e_pmDE', 'e_iPrad', 'e_gPrad', 'chi2', 'zPmag', 'zpmag', 'uPmag', 'upmag', 'spS/N', 'SDSS-ID', 'rPmag', 'rpmag', 'r(F)', 'r(E)', 'is', 'iPmag', 'ipmag', 'i(N)', 'gPmag', 'gpmag', 'g(O)', 'g(J)', 'e_zPmag', 'e_zpmag', 'e_uPmag', 'e_upmag', 'e_rPmag', 'e_rpmag', 'e_iPmag', 'e_ipmag', 'e_gPmag', 'e_gpmag'] Columns in sdss_photo (V/147): ['RA_ICRS', 'DE_ICRS', 'mode', 'q_mode', 'class', 'SDSS12', 'm_SDSS12', 'objID', 'Sp-ID', 'ObsDate', 'Q', 'umag', 'e_umag', 'gmag', 'e_gmag', 'rmag', 'e_rmag', 'imag', 'e_imag', 'zmag', 'e_zmag', 'zsp', 'rpmag', 'pmRA', 'pmDE', 'zph', 'e_zph', '<zph>'] Columns in twomass (II/246): ['RAJ2000', 'DEJ2000', '2MASS', 'Jmag', 'e_Jmag', 'Hmag', 'e_Hmag', 'Kmag', 'e_Kmag', 'Qflg', 'Rflg', 'Bflg', 'Cflg', 'Xflg', 'Aflg', 'XMMOM', 'IDsum', 'ObsID', 'ID', 'UVW2r', 'UVM2r', 'UVM1r', 'Ur', 'Br', 'Vr', 'RAJ2000.1', 'DEJ2000.1', 'ePos', 'Nobs', 'sig(UVW2)', 'sig(UVM2)', 'sig(UVW1)', 'sig(U)', 'sig(B)', 'sig(V)', 'F(UVW2)', 'F(UVM2)', 'F(UVW1)', 'F(U)', 'F(B)', 'F(V)', 'UVW2mAB', 'UVM2mAB', 'UVW1mAB', 'UmAB', 'BmAB', 'VmAB', 'q_UVW2', 'q_UVM2', 'q_UVW1', 'q_U', 'q_B', 'q_V', 'xUVW2', 'xUVM2', 'xUVW1', 'xU', 'xB', 'xV'] Columns in gama (II/356): ['XMMOM', 'IDsum', 'ObsID', 'ID', 'UVW2r', 'UVM2r', 'UVM1r', 'Ur', 'Br', 'Vr', 'RAJ2000', 'DEJ2000', 'ePos', 'Nobs', 'sig(UVW2)', 'sig(UVM2)', 'sig(UVW1)', 'sig(U)', 'sig(B)', 'sig(V)', 'F(UVW2)', 'F(UVM2)', 'F(UVW1)', 'F(U)', 'F(B)', 'F(V)', 'UVW2mAB', 'UVM2mAB', 'UVW1mAB', 'UmAB', 'BmAB', 'VmAB', 'q_UVW2', 'q_UVM2', 'q_UVW1', 'q_U', 'q_B', 'q_V', 'xUVW2', 'xUVM2', 'xUVW1', 'xU', 'xB', 'xV'] Traceback (most recent call last): File "/home/pmqr7/STAR.py", line 75, in <module> df, sdss_spec_matched = cross_match(df, sdss_spec, 'objra_y', 'objdec', 'RA_ICRS', 'DE_ICRS') ^^^

    - `df` is loaded from `merged_data.csv` and contains columns `objra_y` and `objdec`.
    - `sdss_spec` is loaded from `sdss_dr16_spec.csv` and contains columns `RA_ICRS` and `DE_ICRS`.
2. **Cross-Matching**:
    - The `cross_match` function calls `deg_to_rad` to convert `objra_y` and `objdec` (from `df`) and `RA_ICRS` and `DE_ICRS` (from `sdss_spec`) into radians, creating `ra_rad` and `dec_rad` columns.
    - These radian values are then used to create coordinate arrays `coords1` (from `df`) and `coords2` (from `sdss_spec`).
    - `cKDTree` is used to find the nearest neighbors between `coords1` and `coords2`, but it fails if any values in `coords1` or `coords2` are `NaN` or `inf`.
3. **Source of `NaN` or `inf`**:
    - If `objra_y`, `objdec`, `RA_ICRS`, or `DE_ICRS` contains `NaN` or `inf`, then `np.radians()` will propagate these invalid values.
    - For example, `np.radians(np.nan)` results in `nan`, and `cKDTree` cannot process such values.

Step 2: Solution Approach

To resolve this, we need to handle `NaN` and `inf` values in the RA and Dec columns before passing them to `cKDTree`. We can either:

1. **Skip Rows with `NaN` or `inf` Values**:
    - Remove rows where the RA or Dec columns contain `NaN` or `inf` before performing the cross-match.
2. **Replace `NaN` or `inf` with Mean/Median**:
    - Replace `NaN` or `inf` values in the RA and Dec columns with the mean or median of the respective column.

Both approaches are valid, but they have different implications:

- **Skipping Rows**: This ensures that only valid data is used for cross-matching, but it may reduce the number of galaxies in the final dataset.
- **Replacing with Mean/Median**: This preserves more data but introduces an approximation, as the mean or median RA/Dec may not accurately represent the true position of a galaxy.

Since this is an astronomical dataset, replacing coordinates with a mean or median might not be ideal, as RA and Dec are precise positional measurements, and substituting them

could lead to incorrect cross-matching. Therefore, I recommend **skipping rows with `NaN` or `inf` values** in the RA and Dec columns. However, I'll also provide an alternative implementation to replace `NaN` with the median, and you can choose the approach that best suits your needs.

Step 3: Update the Script to Handle `NaN` and `inf`

I'll modify the script to:

1. Add a function to clean the DataFrames by removing rows with `NaN` or `inf` in the RA and Dec columns.
2. Update the `cross_match` function to apply this cleaning step before proceeding.
3. Provide an alternative implementation (commented out) that replaces `NaN` with the median.

Here's the updated script with these changes. I've also renamed the script to `STAR.py` (as per your command) and kept the STAR acronym documentation.

```
python
# STAR.py: Model for predicting log_SFR_Ha using the STAR framework
# STAR: Symbolic, Tuned (Optuna), Astronomy Regressors (Random Forest + Gradient Boosting)
# - S: Symbolic Regressor (via gplearn)
# - T: Tuned (hyperparameter optimization with Optuna)
# - A: Astronomy (predicting star formation rates in galaxies)
# - R: Regressors (Random Forest and Gradient Boosting)

import pandas as pd
import numpy as np
import shap
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split, cross_val_score, KFold
from sklearn.ensemble import RandomForestRegressor, GradientBoostingRegressor
from sklearn.metrics import r2_score
from sklearn.preprocessing import StandardScaler, MinMaxScaler
from gplearn.genetic import SymbolicRegressor
import optuna
import warnings
from numpy.polynomial import Polynomial
from scipy.spatial import cKDTree

warnings.filterwarnings("ignore")

# Load the original dataset
df = pd.read_csv("merged_data.csv")

# Load VizieR data (update paths as needed)
```

```
sdss_spec = pd.read_csv("sdss_dr16_spec.csv")  # V/154: Spectroscopic data
sdss_photo = pd.read_csv("sdss_dr12_photo.csv")  # V/147: Photometric data
twomass = pd.read_csv("2mass_xsc.csv")  # II/246: 2MASS Extended Source Catalogue
gama = pd.read_csv("gama_dr3.csv")  # II/356: GAMA DR3

# Print columns for debugging
print("Columns in merged_data.csv:", df.columns.tolist())
print("Columns in sdss_spec (V/154):", sdss_spec.columns.tolist())
print("Columns in sdss_photo (V/147):", sdss_photo.columns.tolist())
print("Columns in twomass (II/246):", twomass.columns.tolist())
print("Columns in gama (II/356):", gama.columns.tolist())

# Function to clean DataFrame by removing rows with NaN or inf in RA/Dec columns
def clean_coordinates(df, ra_col, dec_col):
    initial_len = len(df)
    # Check for NaN or inf in RA and Dec columns
    mask = (
        df[ra_col].notna() & df[dec_col].notna() &  # Not NaN
        np.isfinite(df[ra_col]) & np.isfinite(df[dec_col])  # Not inf
    )
    df_cleaned = df[mask].copy()
    print(f"Removed {initial_len - len(df_cleaned)} rows from DataFrame due to NaN or inf in {ra_col} or
{dec_col}")
    return df_cleaned

# Alternative function to replace NaN with median (commented out)
"""
def replace_nan_with_median(df, ra_col, dec_col):
    # Replace NaN with median
    df[ra_col] = df[ra_col].fillna(df[ra_col].median())
    df[dec_col] = df[dec_col].fillna(df[dec_col].median())
    # Replace inf with median
    df[ra_col] = df[ra_col].replace([np.inf, -np.inf], df[ra_col].median())
    df[dec_col] = df[dec_col].replace([np.inf, -np.inf], df[dec_col].median())
    return df
"""

# Cross-match function
def deg_to_rad(df, ra_col, dec_col):
    try:
        df['ra_rad'] = np.radians(df[ra_col])
        df['dec_rad'] = np.radians(df[dec_col])
        return df
    except KeyError as e:
        print(f"KeyError in deg_to_rad: {e}")
        print(f"Available columns in DataFrame: {df.columns.tolist()}")
        raise

def cross_match(df1, df2, ra_col1, dec_col1, ra_col2, dec_col2, max_dist_arcsec=1.0):
```

```
    try:
        # Clean DataFrames to remove rows with NaN or inf in RA/Dec
        df1_cleaned = clean_coordinates(df1, ra_col1, dec_col1)
        df2_cleaned = clean_coordinates(df2, ra_col2, dec_col2)

        # Alternative: Replace NaN with median (uncomment to use)
        # df1_cleaned = replace_nan_with_median(df1.copy(), ra_col1, dec_col1)
        # df2_cleaned = replace_nan_with_median(df2.copy(), ra_col2, dec_col2)

        # Convert degrees to radians
        df1_cleaned = deg_to_rad(df1_cleaned, ra_col1, dec_col1)
        df2_cleaned = deg_to_rad(df2_cleaned, ra_col2, dec_col2)

        # Create coordinate arrays
        coords1 = np.array([df1_cleaned['ra_rad'], df1_cleaned['dec_rad']]).T
        coords2 = np.array([df2_cleaned['ra_rad'], df2_cleaned['dec_rad']]).T

        # Perform cross-match using cKDTree
        tree = cKDTree(coords2)
        max_dist = np.radians(max_dist_arcsec / 3600.0)
        dist, idx = tree.query(coords1, k=1, distance_upper_bound=max_dist)

        # Filter matches within the maximum distance
        matched = dist < max_dist
        df1_matched = df1_cleaned[matched].copy()
        df2_matched = df2_cleaned.iloc[idx[matched]].copy()

        # Reset indices
        df1_matched = df1_matched.reset_index(drop=True)
        df2_matched = df2_matched.reset_index(drop=True)

        return df1_matched, df2_matched
    except KeyError as e:
        print(f"KeyError in cross_match: {e}")
        print(f"df1 columns: {df1.columns.tolist()}")
        print(f"df2 columns: {df2.columns.tolist()}")
        raise
    except Exception as e:
        print(f"Error in cross_match: {e}")
        raise

# Cross-match with VizieR catalogues
# V/154: SDSS DR16 spectroscopic data (uses RA_ICRS, DE_ICRS)
df, sdss_spec_matched = cross_match(df, sdss_spec, 'objra_y', 'objdec', 'RA_ICRS', 'DE_ICRS')
df = pd.concat([df, sdss_spec_matched[['specz', 'flux_Ha', 'flux_Hb', 'flux_OIII_5007', 'flux_NII_6584',
'e_flux_Ha']]], axis=1)
print(f"After V/154 cross-match: {len(df)} galaxies")

# V/147: SDSS DR12 photometric data (uses RA_ICRS, DE_ICRS based on column list)
```

```
df, sdss_photo_matched = cross_match(df, sdss_photo, 'objra_y', 'objdec', 'RA_ICRS', 'DE_ICRS')
df = pd.concat([df, sdss_photo_matched[['petroMag_u', 'petroMag_g', 'petroMag_r', 'petroMag_i',
'petroMag_z', 'petroMagErr_u', 'petroMagErr_g', 'petroMagErr_r', 'petroMagErr_i', 'petroMagErr_z']]],
axis=1)
print(f"After V/147 cross-match: {len(df)} galaxies")

# II/246: 2MASS Extended Source Catalogue (uses RAJ2000, DEJ2000)
df, twomass_matched = cross_match(df, twomass, 'objra_y', 'objdec', 'RAJ2000', 'DEJ2000')
df = pd.concat([df, twomass_matched[['Jmag', 'Hmag', 'Kmag', 'e_Jmag', 'e_Hmag', 'e_Kmag']]], axis=1)
print(f"After II/246 cross-match: {len(df)} galaxies")

# II/356: GAMA DR3 (uses RAJ2000, DEJ2000 based on column list)
df, gama_matched = cross_match(df, gama, 'objra_y', 'objdec', 'RAJ2000', 'DEJ2000')
df = pd.concat([df, gama_matched[['Flux_Ha', 'Flux_NII_6584', 'e_Flux_Ha', 'u_mag', 'g_mag', 'r_mag',
'i_mag', 'z_mag']]], axis=1)
print(f"After II/356 cross-match: {len(df)} galaxies")

# Recompute log_SFR_Ha and metallicity from raw fluxes (using V/154 data)
df['Ha_Hb_observed'] = df['flux_Ha'] / df['flux_Hb']
Ha_Hb_intrinsic = 2.86
k_Ha = 2.468
k_Hb = 3.634
df['A_Ha'] = 2.5 * np.log10(df['Ha_Hb_observed'] / Ha_Hb_intrinsic) * (k_Ha / (k_Hb - k_Ha))
df['flux_Ha_corr'] = df['flux_Ha'] * 10**(0.4 * df['A_Ha'])
H0 = 70  # km/s/Mpc
c = 3e5  # km/s
df['DL'] = (c * df['specz'] / H0) * 3.0856e24  # cm
df['L_Ha'] = df['flux_Ha_corr'] * 4 * np.pi * df['DL']**2  # erg/s
df['log_SFR_Ha_raw'] = np.log10(df['L_Ha'] * 7.9e-42)  # M_sun/yr
df['log_O3N2_raw'] = np.log10((df['flux_OIII_5007'] / df['flux_Hb']) / (df['flux_NII_6584'] / df['flux_Ha']))
df['OH_O3N2_raw'] = 8.533 - 0.214 * df['log_O3N2_raw']

# Add raw fluxes as features
df['log_flux_Ha'] = np.log10(df['flux_Ha'] + 1e-10)
df['log_flux_Hb'] = np.log10(df['flux_Hb'] + 1e-10)
df['log_flux_OIII_5007'] = np.log10(df['flux_OIII_5007'] + 1e-10)
df['log_flux_NII_6584'] = np.log10(df['flux_NII_6584'] + 1e-10)
df['log_e_flux_Ha'] = np.log10(df['e_flux_Ha'] + 1e-10)

# Add photometric colors (from V/147)
df['color_ug'] = df['petroMag_u'] - df['petroMag_g']
df['color_gr'] = df['petroMag_g'] - df['petroMag_r']
df['color_ri'] = df['petroMag_r'] - df['petroMag_i']
df['color_iz'] = df['petroMag_i'] - df['petroMag_z']
df['e_color_ug'] = np.sqrt(df['petroMagErr_u']**2 + df['petroMagErr_g']**2)
df['e_color_gr'] = np.sqrt(df['petroMagErr_g']**2 + df['petroMagErr_r']**2)

# Add 2MASS colors (from II/246)
df['color_JK'] = df['Jmag'] - df['Kmag']
```

```
df['e_color_JK'] = np.sqrt(df['e_Jmag']**2 + df['e_Kmag']**2)

# Drop rows with missing target or excessive NaNs
df = df.dropna(subset=["log_SFR_Ha"])
df = df.dropna(axis=0, thresh=int(0.8 * df.shape[1]))
df.fillna(df.median(numeric_only=True), inplace=True)

# Step 1: Cosmo-Rank Construction
gz_df = df.copy()
morph_cols = ['P(CD)', 'P(E)', 'P(S0)', 'P(Sa)', 'P(Sab)', 'P(Sb)', 'P(Sbc)',
              'P(Sc)', 'P(Scd)', 'P(Sd)', 'P(Sdm)', 'P(Sm)', 'P(Irr)']
gz_df['morph_sum'] = gz_df[morph_cols].sum(axis=1)
gz_df['Num_w'] = gz_df['conf_prob']
scaler_rank = MinMaxScaler()
gz_df[['morph_sum_norm', 'Num_w_norm', 'nsa_z_norm']] = scaler_rank.fit_transform(
    gz_df[['morph_sum', 'Num_w', 'nsa_z']]
)
gz_df['cosmo_rank'] = (0.5 * gz_df['Num_w_norm'] + 0.3 * gz_df['morph_sum_norm'] + 0.2 *
gz_df['nsa_z_norm'])
df['cosmo_rank'] = gz_df['cosmo_rank']

# Step 2: L_cosmo(s) Construction
alpha = -1.5
M_star = df['log_Mass_gas'].median()
df['a_n'] = (10 ** df['log_Mass_gas'])**(1 + alpha) * np.exp(-10 ** df['log_Mass_gas'] / (10 ** M_star))
bins = pd.qcut(df['Re_kpc'], q=20, labels=False, duplicates='drop') + 1
df['z_bin'] = bins
n_bins = df['z_bin'].nunique()
print(f"Number of unique bins after qcut: {n_bins}")
print("Bin distribution for Re_kpc (after qcut):")
print(df['z_bin'].value_counts().sort_index())

s_vals = [0.5, 1.0, 1.5, 2.0]
s_range = np.linspace(0.5, 2, 50)
for s in s_vals:
    col_name = f'L_cosmo_s_{s:.1f}'
    df[col_name] = df.groupby('z_bin')['a_n'].transform('mean') / (df['z_bin'] ** s)

print("Columns after creating L_cosmo_s*:")
print(df.columns.tolist())

print("Checking for NaN/infinite values in cosmo_rank and L_cosmo_s_1.0:")
print("cosmo_rank NaN count:", df['cosmo_rank'].isna().sum())
print("L_cosmo_s_1.0 NaN count:", df['L_cosmo_s_1.0'].isna().sum())
print("cosmo_rank infinite count:", np.isinf(df['cosmo_rank']).sum())
print("L_cosmo_s_1.0 infinite count:", np.isinf(df['L_cosmo_s_1.0']).sum())

df['cosmo_rank_L'] = df['cosmo_rank'] * df['L_cosmo_s_1.0']
df['cosmo_rank_L'] = df['cosmo_rank_L'].replace([np.inf, -np.inf], np.nan).fillna(0)
```

```
print("cosmo_rank_L created, NaN count:", df['cosmo_rank_L'].isna().sum())

# Plot L_cosmo(s) vs s
grouped_means = df.groupby('z_bin')['a_n'].mean().reindex(range(1, n_bins + 1), fill_value=0)
L_vals = [grouped_means / (np.arange(1, n_bins + 1) ** s) for s in s_range]
L_vals = np.array([sum(l) for l in L_vals])
plt.plot(s_range, L_vals)
plt.xlabel("s")
plt.ylabel("L_cosmo(s)")
plt.title("L_cosmo(s) Behavior")
plt.savefig("L_cosmo_curve.png")
plt.close()

# Scatter plot of L_cosmo_s_1.0 vs log_SFR_Ha
plt.scatter(df['L_cosmo_s_1.0'], df['log_SFR_Ha'], alpha=0.5)
plt.xlabel("L_cosmo_s_1.0")
plt.ylabel("log_SFR_Ha")
plt.title("L_cosmo_s_1.0 vs log_SFR_Ha")
plt.savefig("L_cosmo_s1_vs_SFR.png")
plt.close()

# Step 3: Define features and target
target = "log_SFR_Ha"
base_features = [
    "log_Mass_gas", "log_Mass", "Av_gas_Re", "Av_ssp_Re",
    "OH_O3N2_cen", "OH_T04_cen", "OH_dop_cen", "Age_LW_Re_fit",
    "ZH_LW_Re_fit", "Re_kpc", "vel_disp_Ha_cen", "Lambda_Re",
    "Sigma_Mass_Re", "EW_Ha_cen", "Ha_Hb_cen"
]
df["mass_metallicity"] = df["log_Mass"] * df["OH_O3N2_cen"]
df["dust_metallicity"] = df["Av_gas_Re"] * df["OH_T04_cen"]
df["disp_mass_ratio"] = df["vel_disp_Ha_cen"] / (df["Sigma_Mass_Re"] + 1e-5)
df["age_metallicity"] = df["Age_LW_Re_fit"] * df["ZH_LW_Re_fit"]
df["sqrt_Re_kpc"] = np.sqrt(df["Re_kpc"] + 1e-5)
df["log_mass"] = np.log(df["log_Mass_gas"] + 1e-5)
df["BSD_likelihood"] = (
    df["log_Mass_gas"] * df["OH_O3N2_cen"] /
    (df["Av_gas_Re"] + 1e-5 + df["Age_LW_Re_fit"])
)
df["cosmo_rank_mass"] = df["cosmo_rank"] * df["log_Mass"]
df["L_cosmo_s1_mass"] = df["L_cosmo_s_1.0"] * df["log_Mass"]
df["cosmo_rank_EW"] = df["cosmo_rank"] * df["EW_Ha_cen"]
df["L_cosmo_s1_EW"] = df["L_cosmo_s_1.0"] * df["EW_Ha_cen"]
df["cosmo_rank_scaled"] = df["cosmo_rank"] * 20
df["L_cosmo_s1_scaled"] = df["L_cosmo_s_1.0"] * 20
df["cosmo_rank_L_mass"] = df["cosmo_rank_L"] * df["log_Mass"]
df["L_cosmo_s1_metallicity"] = df["L_cosmo_s_1.0"] * df["OH_O3N2_cen"]
features = base_features + [
    "mass_metallicity", "dust_metallicity", "disp_mass_ratio",
```

```
    "age_metallicity", "sqrt_Re_kpc", "log_mass", "BSD_likelihood",
    "cosmo_rank", "L_cosmo_s_0.5", "L_cosmo_s_1.0", "L_cosmo_s_1.5",
    "L_cosmo_s_2.0", "cosmo_rank_L", "cosmo_rank_mass", "L_cosmo_s1_mass",
    "cosmo_rank_EW", "L_cosmo_s1_EW", "cosmo_rank_scaled", "L_cosmo_s1_scaled",
    "cosmo_rank_L_mass", "L_cosmo_s1_metallicity",
    "log_flux_Ha", "log_flux_Hb", "log_flux_OIII_5007", "log_flux_NII_6584",
    "log_e_flux_Ha", "log_SFR_Ha_raw", "OH_O3N2_raw",
    "color_ug", "color_gr", "color_ri", "color_iz", "e_color_ug", "e_color_gr",
    "color_JK", "e_color_JK"
]

X = df[features]
y = df[target]

# Train/test split with n_jobs=2
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42, n_jobs=2)

# Normalize features
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# Optuna optimization for Random Forest
def objective_rf(trial):
    params = {
        'n_estimators': trial.suggest_int('n_estimators', 100, 500),
        'max_depth': trial.suggest_int('max_depth', 5, 20),
        'min_samples_split': trial.suggest_int('min_samples_split', 2, 10),
        'random_state': 42,
        'n_jobs': 2  # Use 2 processors
    }
    model = RandomForestRegressor(**params)
    # 5-fold cross-validation
    cv = KFold(n_splits=5, shuffle=True, random_state=42)
    scores = cross_val_score(model, X_train_scaled, y_train, cv=cv, scoring='r2', n_jobs=2)
    return scores.mean()

study_rf = optuna.create_study(direction='maximize')
study_rf.optimize(objective_rf, n_trials=20, n_jobs=2)
best_params_rf = study_rf.best_params
print("Best Random Forest Parameters:", best_params_rf)
print("Best Random Forest CV R²:", study_rf.best_value)

# Train Random Forest with best parameters
rf = RandomForestRegressor(**best_params_rf, random_state=42, n_jobs=2)
rf.fit(X_train_scaled, y_train)
y_pred_rf = rf.predict(X_test_scaled)
r2_rf = r2_score(y_test, y_pred_rf)
```

```
# Cross-validation for final RF model
cv_scores_rf = cross_val_score(rf, X_train_scaled, y_train, cv=5, scoring='r2', n_jobs=2)
print(f"Random Forest 5-Fold CV R²: Mean = {cv_scores_rf.mean():.4f}, Std = {cv_scores_rf.std():.4f}")

# Optuna optimization for Gradient Boosting
def objective_gb(trial):
    params = {
        'n_estimators': trial.suggest_int('n_estimators', 100, 500),
        'learning_rate': trial.suggest_float('learning_rate', 0.01, 0.2, log=True),
        'max_depth': trial.suggest_int('max_depth', 3, 10),
        'random_state': 42,
        'n_jobs': 2  # Use 2 processors
    }
    model = GradientBoostingRegressor(**params)
    cv = KFold(n_splits=5, shuffle=True, random_state=42)
    scores = cross_val_score(model, X_train_scaled, y_train, cv=cv, scoring='r2', n_jobs=2)
    return scores.mean()

study_gb = optuna.create_study(direction='maximize')
study_gb.optimize(objective_gb, n_trials=20, n_jobs=2)
best_params_gb = study_gb.best_params
print("Best Gradient Boosting Parameters:", best_params_gb)
print("Best Gradient Boosting CV R²:", study_gb.best_value)

# Train Gradient Boosting with best parameters
gb = GradientBoostingRegressor(**best_params_gb, random_state=42, n_jobs=2)
gb.fit(X_train_scaled, y_train)
y_pred_gb = gb.predict(X_test_scaled)
r2_gb = r2_score(y_test, y_pred_gb)

# Cross-validation for final GB model
cv_scores_gb = cross_val_score(gb, X_train_scaled, y_train, cv=5, scoring='r2', n_jobs=2)
print(f"Gradient Boosting 5-Fold CV R²: Mean = {cv_scores_gb.mean():.4f}, Std =
{cv_scores_gb.std():.4f}")

print(f"R^2 Score (Random Forest, Test Set): {r2_rf:.4f}")
print(f"R^2 Score (Gradient Boosting, Test Set): {r2_gb:.4f}")

# SHAP for feature selection
explainer_rf = shap.Explainer(rf, X_train_scaled)
shap_values_rf = explainer_rf(X_test_scaled)
explainer_gb = shap.Explainer(gb, X_train_scaled)
shap_values_gb = explainer_gb(X_test_scaled)
shap.summary_plot(shap_values_rf, X_test_scaled, plot_type="bar", show=False)
plt.title("SHAP Summary - Random Forest")
plt.tight_layout()
plt.savefig("shap_rf_summary.png")
plt.clf()
shap.summary_plot(shap_values_gb, X_test_scaled, plot_type="bar", show=False)
```

```
plt.title("SHAP Summary - Gradient Boosting")
plt.tight_layout()
plt.savefig("shap_gb_summary.png")
plt.clf()

# Feature selection based on SHAP
shap_values_rf_mean = np.abs(shap_values_rf.values).mean(axis=0)
feature_importance = pd.DataFrame({"feature": features, "importance": shap_values_rf_mean})
feature_importance = feature_importance.sort_values("importance", ascending=False)
top_features = feature_importance["feature"].head(10).tolist()
bsd_features = ["cosmo_rank", "L_cosmo_s_1.0", "cosmo_rank_L", "cosmo_rank_scaled",
"L_cosmo_s1_scaled", "cosmo_rank_L_mass", "L_cosmo_s1_metallicity"]
for bsd_feature in bsd_features:
    if bsd_feature not in top_features:
        top_features.append(bsd_feature)
X_selected = df[top_features]
X_train_selected, X_test_selected, _, _ = train_test_split(X_selected, y, test_size=0.2, random_state=42,
n_jobs=2)
X_train_selected_scaled = scaler.fit_transform(X_train_selected)
X_test_selected_scaled = scaler.transform(X_test_selected)

# Symbolic Regression with gplearn
symbolic_model = SymbolicRegressor(
    population_size=3000,
    generations=150,
    stopping_criteria=0.01,
    p_crossover=0.7,
    p_subtree_mutation=0.1,
    p_hoist_mutation=0.05,
    p_point_mutation=0.1,
    max_samples=0.9,
    verbose=1,
    parsimony_coefficient=0.0001,
    random_state=42,
    n_jobs=2,  # Use 2 processors
    function_set=('add', 'sub', 'mul', 'div', 'sqrt', 'log', 'sin', 'cos')
)
symbolic_model.fit(X_train_selected_scaled, y_train)
y_pred_sym = symbolic_model.predict(X_test_selected_scaled)
r2_sym = r2_score(y_test, y_pred_sym)
print(f"gplearn R²: {r2_sym:.4f}")
print("Symbolic Expression:")
print(symbolic_model._program)

# Cross-validation for symbolic model
cv_scores_sym = cross_val_score(symbolic_model, X_train_selected_scaled, y_train, cv=5, scoring='r2',
n_jobs=2)
print(f"Symbolic Regression 5-Fold CV R²: Mean = {cv_scores_sym.mean():.4f}, Std =
{cv_scores_sym.std():.4f}")
```

```
# Save symbolic expression as polynomial fit plot and print the equation
x = np.linspace(min(y_test), max(y_test), 500)
y_expr = symbolic_model.predict(scaler.transform(np.tile(X_test_selected.mean().values, (500,1))))
p = Polynomial.fit(y_test, y_pred_sym, deg=3)
# Extract polynomial coefficients
coeffs = p.coef
print("Polynomial Fit Equation (degree 3):")
print(f"y = {coeffs[0]:.4f} + {coeffs[1]:.4f}*x + {coeffs[2]:.4f}*x^2 + {coeffs[3]:.4f}*x^3")
# Plot the polynomial fit
plt.plot(*p.linspace(), label="Polynomial Fit")
plt.scatter(y_test, y_pred_sym, s=10, alpha=0.5, label="Symbolic Predictions")
plt.xlabel("True log_SFR_Ha")
plt.ylabel("Predicted log_SFR_Ha")
plt.title("Symbolic Regression Fit with Polynomial")
plt.legend()
plt.tight_layout()
plt.savefig("gplearn_expression_plot.png")
plt.clf()

# Plot predictions vs true
plt.scatter(y_test, y_pred_rf, alpha=0.5, label="RF", marker="o")
plt.scatter(y_test, y_pred_gb, alpha=0.5, label="GB", marker="s")
plt.scatter(y_test, y_pred_sym, alpha=0.5, label="Symbolic", marker="^")
plt.plot([y.min(), y.max()], [y.min(), y.max()], "k--")
plt.xlabel("True log_SFR_Ha")
plt.ylabel("Predicted")
plt.title("Model Predictions vs True Values")
plt.legend()
plt.tight_layout()
plt.savefig("model_comparison.png")
plt.clf()

# Residuals and Correlation Analysis
residuals = pd.DataFrame({
    "True": y_test,
    "RF": y_pred_rf - y_test,
    "GB": y_pred_gb - y_test,
    "Symbolic": y_pred_sym - y_test,
    "cosmo_rank": X_test["cosmo_rank"],
    "L_cosmo_s1": X_test["L_cosmo_s_1.0"]
})
sns.scatterplot(data=residuals, x="True", y="Symbolic", hue="cosmo_rank", size="L_cosmo_s1",
alpha=0.6)
plt.axhline(0, color="black", linestyle="--")
plt.title("Symbolic Residuals vs True log_SFR_Ha")
plt.savefig("residuals_sym.png")
plt.clf()
```

```
print("Correlation of BSD Features with Residuals:")
print(residuals[["RF", "GB", "Symbolic", "cosmo_rank", "L_cosmo_s1"]].corr()[["RF", "GB", "Symbolic"]])
```

Step 4: Explanation of Changes

1. **Added** `clean_coordinates` **Function**:
   - This function removes rows where the RA or Dec columns contain `NaN` or `inf` values.
   - It uses `notna()` to check for `NaN` and `np.isfinite()` to check for `inf`.
   - It prints the number of rows removed for debugging purposes.
2. **Alternative** `replace_nan_with_median` **Function (Commented Out)**:
   - This function replaces `NaN` and `inf` values with the median of the respective column.
   - It's commented out but can be used instead of `clean_coordinates` if you prefer to preserve more data.
3. **Updated** `cross_match` **Function**:
   - Added a cleaning step using `clean_coordinates` to remove rows with invalid RA/Dec values before performing the cross-match.
   - Included a commented-out alternative to use `replace_nan_with_median` instead.
4. **Updated Column Names for Other Catalogues**:
   - Based on the column lists you provided:
     - `sdss_photo` (V/147) uses `RA_ICRS` and `DE_ICRS` (not `RAJ2000` and `DEJ2000` as previously assumed).
     - `gama` (II/356) uses `RAJ2000` and `DEJ2000` (not `RA` and `DEC` as previously assumed).
   - Updated the `cross_match` calls accordingly.
5. **No Changes to Other Parts**:
   - The rest of the script (feature engineering, Optuna optimization, symbolic regression, etc.) remains unchanged, as the issue was specific to the cross-matching step.

Step 5: Expected Output

With these changes, the script should:

1. Print the number of rows removed due to `NaN` or `inf` in the RA/Dec columns for each DataFrame during cross-matching.

    - Example: `Removed 5 rows from DataFrame due to NaN or inf in objra_y or objdec`
2. Proceed past the cross-matching step without the `ValueError: 'x' must be finite` error.
3. Continue with the rest of the analysis, producing:
    - Number of galaxies after each cross-match.
    - Optuna optimization results.
    - 5-fold CV R^2 scores for RF, GB, and symbolic regression.
    - Test set R^2 scores.
    - Polynomial fit equation.
    - Plots (`gplearn_expression_plot.png`, `shap_rf_summary.png`, etc.).
    - Residuals and correlation analysis.

Step 6: Choosing Between Skipping and Replacing

- **Current Implementation (Skipping Rows)**:
    - This is the active implementation, as it's more appropriate for astronomical data where RA and Dec are precise measurements.
    - It ensures that only valid coordinates are used for cross-matching, but it may reduce the dataset size.
- **Alternative Implementation (Replacing with Median)**:
    - To use this approach, uncomment the `replace_nan_with_median` function and the corresponding lines in `cross_match`, and comment out the `clean_coordinates` calls:
    - `python`

```
# df1_cleaned = clean_coordinates(df1, ra_col1, dec_col1)
# df2_cleaned = clean_coordinates(df2, ra_col2, dec_col2)
df1_cleaned = replace_nan_with_median(df1.copy(), ra_col1, dec_col1)
```

    - df2_cleaned = replace_nan_with_median(df2.copy(), ra_col2, dec_col2)
    - This preserves more data but may lead to incorrect cross-matches, as the median RA/Dec might not correspond to the actual position of a galaxy.

I recommend sticking with the current implementation (skipping rows) unless you have a specific reason to preserve all rows, such as a very small dataset where losing rows significantly impacts the analysis.

Step 7: Debugging Tips

1. **Check Rows Removed**:
    - The script will print how many rows were removed due to `NaN` or `inf` in the RA/Dec columns. If a large number of rows are being removed, you might want to investigate the data quality in `merged_data.csv` or `sdss_dr16_spec.csv` to see why so many coordinates are invalid.
2. **Verify Column Names**:
    - The script assumes the column names for RA and Dec are correct based on the provided lists. If you encounter a `KeyError` for any of the RA/Dec columns in the other catalogues (`sdss_photo`, `twomass`, `gama`), double-check the column names in the debugging output and update the `cross_match` calls accordingly.
3. **File Paths**:
    - Ensure all CSV files are in the `/home/pmqr7/` directory. If not, update the paths in the `pd.read_csv` calls.

Step 8: If Further Errors Occur

If you encounter additional errors, please provide:

1. The exact error message.
2. The output of the debugging print statements (e.g., number of rows removed, column names).
3. Any specific concerns about data loss if too many rows are being removed.

This will help me refine the solution further. Let me know how it goes!
we're dialing it in i think
(myenv) root@BigsbyGaming:/home/pmqr7# python STAR.py Columns in merged_data.csv: ['mangaid', 'nsa_sersic_mass', 'nsa_z', 'OBJDEC_x', 'objra_x', 'SFR_PETRORAD_R', 'zcomp_1Mpc', 'zcomp_5Mpc', 'footprint_1', 'footprint_5', 'PLATEIFU', 'OBJRA', 'OBJDEC_y', 'LOGMSTARS', 'SINI', 'VOPT', 'SESSION', 'EXP', 'RMS', 'LOGHILIM200KMS', 'PEAK', 'SNR', 'FHI', 'EFHI', 'LOGMHI', 'VHI', 'EV', 'WM50', 'WP50', 'WP20', 'W2P50', 'WF50', 'DW', 'PR', 'PL', 'AR', 'BR', 'AL', 'BL', 'NEGDET',

'BLSTRUCT', 'conflag', 'conf_prob', 'name', 'plate', 'ifudsgn', 'plateifu', 'objra_y', 'objdec',
'log_SFR_Ha', 'FoV', 'Re_kpc', 'e_log_Mass', 'e_log_SFR_Ha', 'log_Mass', 'log_SFR_ssp',
'log_NII_Ha_cen', 'e_log_NII_Ha_cen', 'log_OIII_Hb_cen', 'e_log_OIII_Hb_cen',
'log_SII_Ha_cen', 'e_log_SII_Ha_cen', 'log_OII_Hb_cen', 'e_log_OII_Hb_cen',
'EW_Ha_cen', 'e_EW_Ha_cen', 'ZH_LW_Re_fit', 'e_ZH_LW_Re_fit', 'alpha_ZH_LW_Re_fit',
'e_alpha_ZH_LW_Re_fit', 'ZH_MW_Re_fit', 'e_ZH_MW_Re_fit', 'alpha_ZH_MW_Re_fit',
'e_alpha_ZH_MW_Re_fit', 'Age_LW_Re_fit', 'e_Age_LW_Re_fit', 'alpha_Age_LW_Re_fit',
'e_alpha_Age_LW_Re_fit', 'Age_MW_Re_fit', 'e_Age_MW_Re_fit', 'alpha_Age_MW_Re_fit',
'e_alpha_Age_MW_Re_fit', 'Re_arc', 'DL', 'DA', 'PA', 'ellip', 'log_Mass_gas', 'vel_sigma_Re',
'e_vel_sigma_Re', 'log_SFR_SF', 'log_SFR_D_C', 'OH_O3N2_cen', 'e_OH_O3N2_cen',
'OH_N2_cen', 'e_OH_N2_cen', 'OH_ONS_cen', 'e_OH_ONS_cen', 'OH_R23_cen',
'e_OH_R23_cen', 'OH_pyqz_cen', 'e_OH_pyqz_cen', 'OH_t2_cen', 'e_OH_t2_cen',
'OH_M08_cen', 'e_OH_M08_cen', 'OH_T04_cen', 'e_OH_T04_cen', 'OH_dop_cen',
'e_OH_dop_cen', 'OH_O3N2_EPM09_cen', 'e_OH_O3N2_EPM09_cen', 'log_OI_Ha_cen',
'e_log_OI_Ha_cen', 'Ha_Hb_cen', 'e_Ha_Hb_cen', 'log_NII_Ha_Re', 'e_log_NII_Ha_Re',
'log_OIII_Hb_Re', 'e_log_OIII_Hb_Re', 'log_SII_Ha_Re', 'e_log_SII_Ha_Re',
'log_OII_Hb_Re', 'e_log_OII_Hb_Re', 'log_OI_Ha_Re', 'e_log_OI_Ha_Re', 'EW_Ha_Re',
'e_EW_Ha_Re', 'Ha_Hb_Re', 'e_Ha_Hb_Re', 'log_NII_Ha_ALL', 'e_log_NII_Ha_ALL',
'log_OIII_Hb_ALL', 'e_log_OIII_Hb_ALL', 'log_SII_Ha_ALL', 'e_log_SII_Ha_ALL',
'log_OII_Hb_ALL', 'e_log_OII_Hb_ALL', 'log_OI_Ha_ALL', 'e_log_OI_Ha_ALL',
'EW_Ha_ALL', 'e_EW_Ha_ALL', 'Ha_Hb_ALL', 'Sigma_Mass_cen', 'e_Sigma_Mass_cen',
'Sigma_Mass_Re', 'e_Sigma_Mass_Re', 'Sigma_Mass_ALL', 'e_Sigma_Mass_ALL', 'T30',
'ZH_T30', 'ZH_Re_T30', 'a_ZH_T30', 'T40', 'ZH_T40', 'ZH_Re_T40', 'a_ZH_T40', 'T50',
'ZH_T50', 'ZH_Re_T50', 'a_ZH_T50', 'T60', 'ZH_T60', 'ZH_Re_T60', 'a_ZH_T60', 'T70',
'ZH_T70', 'ZH_Re_T70', 'a_ZH_T70', 'T80', 'ZH_T80', 'ZH_Re_T80', 'a_ZH_T80', 'T90',
'ZH_T90', 'ZH_Re_T90', 'a_ZH_T90', 'T95', 'ZH_T95', 'ZH_Re_T95', 'a_ZH_T95', 'T99',
'ZH_T99', 'ZH_Re_T99', 'a_ZH_T99', 'log_Mass_gas_Av_gas_OH',
'log_Mass_gas_Av_ssp_OH', 'vel_ssp_2', 'e_vel_ssp_2', 'vel_Ha_2', 'e_vel_Ha_2',
'vel_ssp_1', 'e_vel_ssp_1', 'vel_Ha_1', 'e_vel_Ha_1', 'log_SFR_ssp_100Myr',
'log_SFR_ssp_10Myr', 'vel_disp_Ha_cen', 'vel_disp_ssp_cen', 'vel_disp_Ha_1Re',
'vel_disp_ssp_1Re', 'log_Mass_in_Re', 'ML_int', 'ML_avg', 'F_Ha_cen', 'e_F_Ha_cen',
'R50_kpc_V', 'e_R50_kpc_V', 'R50_kpc_Mass', 'e_R50_kpc_Mass',
'log_Mass_corr_in_R50_V', 'e_log_Mass_corr_in_R50_V',
'log_Mass_gas_Av_gas_log_log', 'Av_gas_Re', 'e_Av_gas_Re', 'Av_ssp_Re',
'e_Av_ssp_Re', 'Lambda_Re', 'e_Lambda_Re', 'nsa_redshift', 'nsa_mstar', 'nsa_inclination',
'flux_[OII]3726.03_Re_fit', 'e_flux_[OII]3726.03_Re_fit', 'flux_[OII]3726.03_alpha_fit',
'e_flux_[OII]3726.03_alpha_fit', 'flux_[OII]3728.82_Re_fit', 'e_flux_[OII]3728.82_Re_fit',
'flux_[OII]3728.82_alpha_fit', 'e_flux_[OII]3728.82_alpha_fit', 'flux_HI3734.37_Re_fit',
'e_flux_HI3734.37_Re_fit', 'flux_HI3734.37_alpha_fit', 'e_flux_HI3734.37_alpha_fit',
'flux_HI3797.9_Re_fit', 'e_flux_HI3797.9_Re_fit', 'flux_HI3797.9_alpha_fit',
'e_flux_HI3797.9_alpha_fit', 'flux_HeI3888.65_Re_fit', 'e_flux_HeI3888.65_Re_fit',
'flux_HeI3888.65_alpha_fit', 'e_flux_HeI3888.65_alpha_fit', 'flux_HI3889.05_Re_fit',

'e_flux_HI3889.05_Re_fit', 'flux_HI3889.05_alpha_fit', 'e_flux_HI3889.05_alpha_fit',
'flux_HeI3964.73_Re_fit', 'e_flux_HeI3964.73_Re_fit', 'flux_HeI3964.73_alpha_fit',
'e_flux_HeI3964.73_alpha_fit', 'flux_[NeIII]3967.46_Re_fit', 'e_flux_[NeIII]3967.46_Re_fit',
'flux_[NeIII]3967.46_alpha_fit', 'e_flux_[NeIII]3967.46_alpha_fit', 'flux_CaII3968.47_Re_fit',
'e_flux_CaII3968.47_Re_fit', 'flux_CaII3968.47_alpha_fit', 'e_flux_CaII3968.47_alpha_fit',
'flux_Hepsilon3970.07_Re_fit', 'e_flux_Hepsilon3970.07_Re_fit',
'flux_Hepsilon3970.07_alpha_fit', 'e_flux_Hepsilon3970.07_alpha_fit',
'flux_Hdelta4101.77_Re_fit', 'e_flux_Hdelta4101.77_Re_fit', 'flux_Hdelta4101.77_alpha_fit',
'e_flux_Hdelta4101.77_alpha_fit', 'flux_Hgamma4340.49_Re_fit',
'e_flux_Hgamma4340.49_Re_fit', 'flux_Hgamma4340.49_alpha_fit',
'e_flux_Hgamma4340.49_alpha_fit', 'flux_Hbeta4861.36_Re_fit',
'e_flux_Hbeta4861.36_Re_fit', 'flux_Hbeta4861.36_alpha_fit',
'e_flux_Hbeta4861.36_alpha_fit', 'flux_[OIII]4958.91_Re_fit', 'e_flux_[OIII]4958.91_Re_fit',
'flux_[OIII]4958.91_alpha_fit', 'e_flux_[OIII]4958.91_alpha_fit', 'flux_[OIII]5006.84_Re_fit',
'e_flux_[OIII]5006.84_Re_fit', 'flux_[OIII]5006.84_alpha_fit', 'e_flux_[OIII]5006.84_alpha_fit',
'flux_HeI5015.68_Re_fit', 'e_flux_HeI5015.68_Re_fit', 'flux_HeI5015.68_alpha_fit',
'e_flux_HeI5015.68_alpha_fit', 'flux_[NI]5197.9_Re_fit', 'e_flux_[NI]5197.9_Re_fit',
'flux_[NI]5197.9_alpha_fit', 'e_flux_[NI]5197.9_alpha_fit', 'flux_[NI]5200.26_Re_fit',
'e_flux_[NI]5200.26_Re_fit', 'flux_[NI]5200.26_alpha_fit', 'e_flux_[NI]5200.26_alpha_fit',
'flux_HeI5876.0_Re_fit', 'e_flux_HeI5876.0_Re_fit', 'flux_HeI5876.0_alpha_fit',
'e_flux_HeI5876.0_alpha_fit', 'flux_NaI5889.95_Re_fit', 'e_flux_NaI5889.95_Re_fit',
'flux_NaI5889.95_alpha_fit', 'e_flux_NaI5889.95_alpha_fit', 'flux_NaI5895.92_Re_fit',
'e_flux_NaI5895.92_Re_fit', 'flux_NaI5895.92_alpha_fit', 'e_flux_NaI5895.92_alpha_fit',
'flux_[OI]6300.3_Re_fit', 'e_flux_[OI]6300.3_Re_fit', 'flux_[OI]6300.3_alpha_fit',
'e_flux_[OI]6300.3_alpha_fit', 'flux_[NII]6548.05_Re_fit', 'e_flux_[NII]6548.05_Re_fit',
'flux_[NII]6548.05_alpha_fit', 'e_flux_[NII]6548.05_alpha_fit', 'flux_Halpha6562.85_Re_fit',
'e_flux_Halpha6562.85_Re_fit', 'flux_Halpha6562.85_alpha_fit',
'e_flux_Halpha6562.85_alpha_fit', 'flux_[NII]6583.45_Re_fit', 'e_flux_[NII]6583.45_Re_fit',
'flux_[NII]6583.45_alpha_fit', 'e_flux_[NII]6583.45_alpha_fit', 'flux_[SII]6716.44_Re_fit',
'e_flux_[SII]6716.44_Re_fit', 'flux_[SII]6716.44_alpha_fit', 'e_flux_[SII]6716.44_alpha_fit',
'flux_[SII]6730.82_Re_fit', 'e_flux_[SII]6730.82_Re_fit', 'flux_[SII]6730.82_alpha_fit',
'e_flux_[SII]6730.82_alpha_fit', 'flux_[ArIII]7135.8_Re_fit', 'e_flux_[ArIII]7135.8_Re_fit',
'flux_[ArIII]7135.8_alpha_fit', 'e_flux_[ArIII]7135.8_alpha_fit', 'flux_HI9014.91_Re_fit',
'e_flux_HI9014.91_Re_fit', 'flux_HI9014.91_alpha_fit', 'e_flux_HI9014.91_alpha_fit',
'flux_[SIII]9069.0_Re_fit', 'e_flux_[SIII]9069.0_Re_fit', 'flux_[SIII]9069.0_alpha_fit',
'e_flux_[SIII]9069.0_alpha_fit', 'flux_[FeII]9470.93_Re_fit', 'e_flux_[FeII]9470.93_Re_fit',
'flux_[FeII]9470.93_alpha_fit', 'e_flux_[FeII]9470.93_alpha_fit', 'flux_[SIII]9531.1_Re_fit',
'e_flux_[SIII]9531.1_Re_fit', 'flux_[SIII]9531.1_alpha_fit', 'e_flux_[SIII]9531.1_alpha_fit',
'OH_Mar13_N2_Re_fit', 'e_OH_Mar13_N2_Re_fit', 'OH_Mar13_N2_alpha_fit',
'e_OH_Mar13_N2_alpha_fit', 'OH_Mar13_O3N2_Re_fit', 'e_OH_Mar13_O3N2_Re_fit',
'OH_Mar13_O3N2_alpha_fit', 'e_OH_Mar13_O3N2_alpha_fit', 'OH_T04_Re_fit',
'e_OH_T04_Re_fit', 'OH_T04_alpha_fit', 'e_OH_T04_alpha_fit', 'OH_Pet04_N2_lin_Re_fit',

'e_OH_Pet04_N2_lin_Re_fit', 'OH_Pet04_N2_lin_alpha_fit',
'e_OH_Pet04_N2_lin_alpha_fit', 'OH_Pet04_N2_poly_Re_fit',
'e_OH_Pet04_N2_poly_Re_fit', 'OH_Pet04_N2_poly_alpha_fit',
'e_OH_Pet04_N2_poly_alpha_fit', 'OH_Pet04_O3N2_Re_fit', 'e_OH_Pet04_O3N2_Re_fit',
'OH_Pet04_O3N2_alpha_fit', 'e_OH_Pet04_O3N2_alpha_fit', 'OH_Kew02_N2O2_Re_fit',
'e_OH_Kew02_N2O2_Re_fit', 'OH_Kew02_N2O2_alpha_fit',
'e_OH_Kew02_N2O2_alpha_fit', 'OH_Pil10_ONS_Re_fit', 'e_OH_Pil10_ONS_Re_fit',
'OH_Pil10_ONS_alpha_fit', 'e_OH_Pil10_ONS_alpha_fit', 'OH_Pil10_ON_Re_fit',
'e_OH_Pil10_ON_Re_fit', 'OH_Pil10_ON_alpha_fit', 'e_OH_Pil10_ON_alpha_fit',
'OH_Pil11_NS_Re_fit', 'e_OH_Pil11_NS_Re_fit', 'OH_Pil11_NS_alpha_fit',
'e_OH_Pil11_NS_alpha_fit', 'OH_Cur20_RS32_Re_fit', 'e_OH_Cur20_RS32_Re_fit',
'OH_Cur20_RS32_alpha_fit', 'e_OH_Cur20_RS32_alpha_fit', 'OH_Cur20_R3_Re_fit',
'e_OH_Cur20_R3_Re_fit', 'OH_Cur20_R3_alpha_fit', 'e_OH_Cur20_R3_alpha_fit',
'OH_Cur20_O3O2_Re_fit', 'e_OH_Cur20_O3O2_Re_fit', 'OH_Cur20_O3O2_alpha_fit',
'e_OH_Cur20_O3O2_alpha_fit', 'OH_Cur20_S2_Re_fit', 'e_OH_Cur20_S2_Re_fit',
'OH_Cur20_S2_alpha_fit', 'e_OH_Cur20_S2_alpha_fit', 'OH_Cur20_R2_Re_fit',
'e_OH_Cur20_R2_Re_fit', 'OH_Cur20_R2_alpha_fit', 'e_OH_Cur20_R2_alpha_fit',
'OH_Cur20_N2_Re_fit', 'e_OH_Cur20_N2_Re_fit', 'OH_Cur20_N2_alpha_fit',
'e_OH_Cur20_N2_alpha_fit', 'OH_Cur20_R23_Re_fit', 'e_OH_Cur20_R23_Re_fit',
'OH_Cur20_R23_alpha_fit', 'e_OH_Cur20_R23_alpha_fit', 'OH_Cur20_O3N2_Re_fit',
'e_OH_Cur20_O3N2_Re_fit', 'OH_Cur20_O3N2_alpha_fit', 'e_OH_Cur20_O3N2_alpha_fit',
'OH_Cur20_O3S2_Re_fit', 'e_OH_Cur20_O3S2_Re_fit', 'OH_Cur20_O3S2_alpha_fit',
'e_OH_Cur20_O3S2_alpha_fit', 'OH_KK04_Re_fit', 'e_OH_KK04_Re_fit',
'OH_KK04_alpha_fit', 'e_OH_KK04_alpha_fit', 'OH_Pil16_R_Re_fit',
'e_OH_Pil16_R_Re_fit', 'OH_Pil16_R_alpha_fit', 'e_OH_Pil16_R_alpha_fit',
'OH_Pil16_S_Re_fit', 'e_OH_Pil16_S_Re_fit', 'OH_Pil16_S_alpha_fit',
'e_OH_Pil16_S_alpha_fit', 'OH_Ho_Re_fit', 'e_OH_Ho_Re_fit', 'OH_Ho_alpha_fit',
'e_OH_Ho_alpha_fit', 'U_Dors_O32_Re_fit', 'e_U_Dors_O32_Re_fit',
'U_Dors_O32_alpha_fit', 'e_U_Dors_O32_alpha_fit', 'U_Dors_S_Re_fit',
'e_U_Dors_S_Re_fit', 'U_Dors_S_alpha_fit', 'e_U_Dors_S_alpha_fit',
'U_Mor16_O23_fs_Re_fit', 'e_U_Mor16_O23_fs_Re_fit', 'U_Mor16_O23_fs_alpha_fit',
'e_U_Mor16_O23_fs_alpha_fit', 'U_Mor16_O23_ts_Re_fit', 'e_U_Mor16_O23_ts_Re_fit',
'U_Mor16_O23_ts_alpha_fit', 'e_U_Mor16_O23_ts_alpha_fit', 'NH_Pil16_R_Re_fit',
'e_NH_Pil16_R_Re_fit', 'NH_Pil16_R_alpha_fit', 'e_NH_Pil16_R_alpha_fit',
'NO_Pil16_R_Re_fit', 'e_NO_Pil16_R_Re_fit', 'NO_Pil16_R_alpha_fit',
'e_NO_Pil16_R_alpha_fit', 'NO_Pil16_Ho_R_Re_fit', 'e_NO_Pil16_Ho_R_Re_fit',
'NO_Pil16_Ho_R_alpha_fit', 'e_NO_Pil16_Ho_R_alpha_fit', 'NO_Pil16_N2_R2_Re_fit',
'e_NO_Pil16_N2_R2_Re_fit', 'NO_Pil16_N2_R2_alpha_fit', 'e_NO_Pil16_N2_R2_alpha_fit',
'Ne_Oster_S_Re_fit', 'e_Ne_Oster_S_Re_fit', 'Ne_Oster_S_alpha_fit',
'e_Ne_Oster_S_alpha_fit', 'Hd_Re_fit', 'e_Hd_Re_fit', 'Hd_alpha_fit', 'e_Hd_alpha_fit',
'Hb_Re_fit', 'e_Hb_Re_fit', 'Hb_alpha_fit', 'e_Hb_alpha_fit', 'Mgb_Re_fit', 'e_Mgb_Re_fit',
'Mgb_alpha_fit', 'e_Mgb_alpha_fit', 'Fe5270_Re_fit', 'e_Fe5270_Re_fit', 'Fe5270_alpha_fit',

'e_Fe5270_alpha_fit', 'Fe5335_Re_fit', 'e_Fe5335_Re_fit', 'Fe5335_alpha_fit', 'e_Fe5335_alpha_fit', 'D4000_Re_fit1', 'e_D4000_Re_fit', 'D4000_alpha_fit', 'e_D4000_alpha_fit', 'Hdmod_Re_fit', 'e_Hdmod_Re_fit', 'Hdmod_alpha_fit', 'e_Hdmod_alpha_fit', 'Hg_Re_fit', 'e_Hg_Re_fit', 'Hg_alpha_fit', 'e_Hg_alpha_fit', 'u_band_mag', 'u_band_mag_error', 'u_band_abs_mag', 'u_band_abs_mag_error', 'g_band_mag', 'g_band_mag_error', 'g_band_abs_mag', 'g_band_abs_mag_error', 'r_band_mag', 'r_band_mag_error', 'r_band_abs_mag', 'r_band_abs_mag_error', 'i_band_mag', 'i_band_mag_error', 'i_band_abs_mag', 'i_band_abs_mag_error', 'B_band_mag', 'B_band_mag_error', 'B_band_abs_mag', 'B_band_abs_mag_error', 'V_band_mag', 'V_band_mag_error', 'V_band_abs_mag', 'V_band_abs_mag_error', 'RJ_band_mag', 'RJ_band_mag_error', 'RJ_band_abs_mag', 'RJ_band_abs_mag_error', 'R50', 'error_R50', 'R90', 'error_R90', 'C', 'e_C', 'B-V', 'error_B-V', 'B-R', 'error_B-R', 'log_Mass_phot', 'e_log_Mass_phot', 'V-band_SB_at_Re', 'error_V-band_SB_at_Re', 'V-band_SB_at_R_50', 'error_V-band_SB_at_R_50', 'nsa_sersic_n_morph', 'u-g', 'g-r', 'r-i', 'i-z', 'P(CD)', 'P(E)', 'P(S0)', 'P(Sa)', 'P(Sab)', 'P(Sb)', 'P(Sbc)', 'P(Sc)', 'P(Scd)', 'P(Sd)', 'P(Sdm)', 'P(Sm)', 'P(Irr)', 'best_type_n', 'best_type', 'nsa_nsaid', 'Vmax_w', 'Num_w', 'QCFLAG'] Columns in sdss_spec (V/154): ['objID', 'RA_ICRS', 'DE_ICRS', 'mode', 'class', 'clean', 'e_RA_ICRS', 'e_DE_ICRS', 'umag', 'gmag', 'rmag', 'imag', 'zmag', 'e_umag', 'e_gmag', 'e_rmag', 'e_imag', 'e_zmag', 'zsp', 'e_zsp', 'f_zsp', 'pmRA', 'pmDE', 'zph', 'e_zph', '<zph>', 'Q', 'SDSS16', 'Sp-ID', 'MJD', 'zs', 'zPrad', 'zPA', 'zoRA', 'zoDE', 'zMJD', 'zFlags', 'zdVrad', 'zdVell', 'zc', 'Vdisp', 'us', 'uPrad', 'uPA', 'uoRA', 'uoDE', 'uMJD', 'uFlags', 'udVrad', 'udVell', 'uc', 'subCl', 'Status', 'spType', 'SpObjID', 'spInst', 'spCl', 'sigRA', 'sigDE', 'rs', 'rPrad', 'rPA', 'roRA', 'roDE', 'rMJD', 'rFlags', 'rdVrad', 'rdVell', 'rc', 'parentID', 'N', 'M', 'iPrad', 'iPA', 'ioRA', 'ioDE', 'iMJD', 'iFlags', 'idVrad', 'idVell', 'ic', 'gs', 'gPrad', 'gPA', 'goRA', 'goDE', 'gMJD', 'gFlags', 'gdVrad', 'gdVell', 'gc', 'flags', 'fieldID', 'e_zPrad', 'e_Vdisp', 'e_uPrad', 'e_rPrad', 'e_pmRA', 'e_pmDE', 'e_iPrad', 'e_gPrad', 'chi2', 'zPmag', 'zpmag', 'uPmag', 'upmag', 'spS/N', 'SDSS-ID', 'rPmag', 'rpmag', 'r(F)', 'r(E)', 'is', 'iPmag', 'ipmag', 'i(N)', 'gPmag', 'gpmag', 'g(O)', 'g(J)', 'e_zPmag', 'e_zpmag', 'e_uPmag', 'e_upmag', 'e_rPmag', 'e_rpmag', 'e_iPmag', 'e_ipmag', 'e_gPmag', 'e_gpmag'] Columns in sdss_photo (V/147): ['RA_ICRS', 'DE_ICRS', 'mode', 'q_mode', 'class', 'SDSS12', 'm_SDSS12', 'objID', 'Sp-ID', 'ObsDate', 'Q', 'umag', 'e_umag', 'gmag', 'e_gmag', 'rmag', 'e_rmag', 'imag', 'e_imag', 'zmag', 'e_zmag', 'zsp', 'rpmag', 'pmRA', 'pmDE', 'zph', 'e_zph', '<zph>'] Columns in twomass (II/246): ['RAJ2000', 'DEJ2000', '2MASS', 'Jmag', 'e_Jmag', 'Hmag', 'e_Hmag', 'Kmag', 'e_Kmag', 'Qflg', 'Rflg', 'Bflg', 'Cflg', 'Xflg', 'Aflg', 'XMMOM', 'IDsum', 'ObsID', 'ID', 'UVW2r', 'UVM2r', 'UVM1r', 'Ur', 'Br', 'Vr', 'RAJ2000.1', 'DEJ2000.1', 'ePos', 'Nobs', 'sig(UVW2)', 'sig(UVM2)', 'sig(UVW1)', 'sig(U)', 'sig(B)', 'sig(V)', 'F(UVW2)', 'F(UVM2)', 'F(UVW1)', 'F(U)', 'F(B)', 'F(V)', 'UVW2mAB', 'UVM2mAB', 'UVW1mAB', 'UmAB', 'BmAB', 'VmAB', 'q_UVW2', 'q_UVM2', 'q_UVW1', 'q_U', 'q_B', 'q_V', 'xUVW2', 'xUVM2', 'xUVW1', 'xU', 'xB', 'xV'] Columns in gama (II/356): ['XMMOM', 'IDsum', 'ObsID', 'ID', 'UVW2r', 'UVM2r', 'UVM1r', 'Ur', 'Br', 'Vr', 'RAJ2000', 'DEJ2000', 'ePos', 'Nobs', 'sig(UVW2)', 'sig(UVM2)', 'sig(UVW1)', 'sig(U)', 'sig(B)', 'sig(V)', 'F(UVW2)', 'F(UVM2)', 'F(UVW1)', 'F(U)', 'F(B)', 'F(V)', 'UVW2mAB', 'UVM2mAB', 'UVW1mAB', 'UmAB', 'BmAB', 'VmAB', 'q_UVW2', 'q_UVM2', 'q_UVW1', 'q_U', 'q_B', 'q_V',

'xUVW2', 'xUVM2', 'xUVW1', 'xU', 'xB', 'xV'] Removed 561 rows from DataFrame due to NaN or inf in objra_y or objdec Removed 0 rows from DataFrame due to NaN or inf in RA_ICRS or DE_ICRS Traceback (most recent call last): File "/home/pmqr7/STAR.py", line 121, in <module> df = pd.concat([df, sdss_spec_matched[['specz', 'flux_Ha', 'flux_Hb', 'flux_OIII_5007', 'flux_NII_6584', 'e_flux_Ha']]], axis=1) ~~~~~~~~~~~~~~~~^^^

'uMJD', 'uFlags', 'udVrad', 'udVell', 'uc', 'subCl', 'Status', 'spType', 'SpObjID', 'spInst', 'spCl', 'sigRA', 'sigDE', 'rs', 'rPrad', 'rPA', 'roRA', 'roDE', 'rMJD', 'rFlags', 'rdVrad', 'rdVell', 'rc', 'parentID', 'N', 'M', 'iPrad', 'iPA', 'ioRA', 'ioDE', 'iMJD', 'iFlags', 'idVrad', 'idVell', 'ic', 'gs', 'gPrad', 'gPA', 'goRA', 'goDE', 'gMJD', 'gFlags', 'gdVrad', 'gdVell', 'gc', 'flags', 'fieldID', 'e_zPrad', 'e_Vdisp', 'e_uPrad', 'e_rPrad', 'e_pmRA', 'e_pmDE', 'e_iPrad', 'e_gPrad', 'chi2', 'zPmag', 'zpmag', 'uPmag', 'upmag', 'spS/N', 'SDSS-ID', 'rPmag', 'rpmag', 'r(F)', 'r(E)', 'is', 'iPmag', 'ipmag', 'i(N)', 'gPmag', 'gpmag', 'g(O)', 'g(J)', 'e_zPmag', 'e_zpmag', 'e_uPmag', 'e_upmag', 'e_rPmag', 'e_rpmag', 'e_iPmag', 'e_ipmag', 'e_gPmag', 'e_gpmag']

The columns we're trying to extract—`specz`, `flux_Ha`, `flux_Hb`, `flux_OIII_5007`, `flux_NII_6584`, and `e_flux_Ha`—are **not** in this list. Here's what we can infer:

1. `specz`:
    - The `sdss_spec` DataFrame has columns like `zsp` (spectroscopic redshift), `zph` (photometric redshift), and `zs` (possibly another redshift measurement). It's likely that `specz` was intended to refer to `zsp`, which is the spectroscopic redshift commonly used in SDSS data.
2. **Flux Columns (`flux_Ha`, `flux_Hb`, `flux_OIII_5007`, `flux_NII_6584`, `e_flux_Ha`)**:
    - These columns are meant to represent the fluxes of specific emission lines (Hα, Hβ, [OIII] 5007 Å, [NII] 6584 Å) and the error on the Hα flux.
    - However, the `sdss_spec` DataFrame does not contain any columns with names like `flux_Ha`, `flux_Hb`, etc. The SDSS DR16 spectroscopic catalog (V/154) typically provides magnitudes (`umag`, `gmag`, etc.), redshifts (`zsp`), and other properties like velocity dispersion (`Vdisp`), but it does **not** include emission line fluxes directly.

Step 2: Why Are These Columns Missing?

The SDSS DR16 spectroscopic catalog (V/154) provides spectroscopic redshifts and broad-band magnitudes, but emission line fluxes (like Hα, Hβ, etc.) are not part of this catalog. These fluxes are typically derived from detailed spectral analysis, which might be available in a different catalog or dataset, such as:

- **SDSS DR16 Value-Added Catalogs (VACs)**: Some VACs, like the MaNGA (Mapping Nearby Galaxies at APO) catalog or the SDSS-IV eBOSS/ELG (Emission Line Galaxy) catalog, include derived emission line fluxes.
- **Custom Derived Data**: The fluxes might have been computed in a previous step or provided in a different file that was meant to be merged with `sdss_dr16_spec.csv`.

In the script, the intention was to cross-match `merged_data.csv` with `sdss_dr16_spec.csv` to add spectroscopic redshifts and emission line fluxes, but since the latter doesn't contain the flux columns, the script fails when trying to access them.

Step 3: Solution Approach

We have a few options to resolve this:

Option 1: Use Available Columns from `sdss_spec`

Since `sdss_spec` contains `zsp` (spectroscopic redshift), we can use that instead of `specz`. However, the flux columns are not available, so we need to either:

- Remove the flux columns from the concatenation step and adjust the script to not rely on them.
- Find the correct dataset that contains these fluxes.

For now, let's modify the script to use `zsp` instead of `specz` and skip the flux columns. We can revisit the fluxes later if you have access to a dataset that contains them.

Option 2: Find the Correct Dataset for Fluxes

The emission line fluxes are likely available in a different catalog, such as:

- **MaNGA Data**: Since `merged_data.csv` contains MaNGA-related columns (e.g., `mangaid`, `plateifu`), the fluxes might already be in `merged_data.csv` or in a MaNGA-specific catalog.
- **SDSS DR16 Spectra**: You might need to download the actual spectra and extract the fluxes using a tool like `specutils` or a custom pipeline, but this is computationally intensive and not practical for this script.
- **Another VizieR Catalog**: A different VizieR table might contain these fluxes, such as the SDSS-IV eBOSS ELG catalog or a MaNGA VAC.

Let's check if `merged_data.csv` already contains these fluxes. Looking at the columns:
Columns in merged_data.csv: ['mangaid', ..., 'log_SFR_Ha', ..., 'F_Ha_cen', 'e_F_Ha_cen', ...]

- `F_Ha_cen` and `e_F_Ha_cen` are present, which likely represent the Hα flux and its error at the center of the galaxy.
- However, there are no direct columns for `flux_Hb`, `flux_OIII_5007`, or `flux_NII_6584`. There are columns like `flux_Halpha6562.85_Re_fit` and `flux_[NII]6583.45_Re_fit`, which might be the fluxes we're looking for, but they are measured at the effective radius (`Re`), not the center, and their naming convention differs.

Option 3: Adjust the Script to Use Available Fluxes

Since `merged_data.csv` already contains some flux measurements (e.g., `flux_Halpha6562.85_Re_fit`), we can use those instead of expecting them from `sdss_spec`. We can also use `F_Ha_cen` as `flux_Ha` and `e_F_Ha_cen` as `e_flux_Ha`. For the other fluxes, we can either:

- Use the `Re_fit` fluxes (e.g., `flux_Hbeta4861.36_Re_fit`, `flux_[OIII]5006.84_Re_fit`, `flux_[NII]6583.45_Re_fit`).
- Skip them for now and adjust the script to not rely on them.

Step 4: Update the Script

I'll modify the script to:

1. Use `zsp` instead of `specz` from `sdss_spec`.
2. Skip the flux columns from `sdss_spec` since they're not available.
3. Use the flux columns already present in `merged_data.csv` (e.g., `F_Ha_cen`, `flux_Hbeta4861.36_Re_fit`, etc.) for the calculations.
4. Adjust the feature engineering to use these available fluxes.

Here's the updated script:

```
python
# STAR.py: Model for predicting log_SFR_Ha using the STAR framework
# STAR: Symbolic, Tuned (Optuna), Astronomy Regressors (Random Forest + Gradient Boosting)
# - S: Symbolic Regressor (via gplearn)
# - T: Tuned (hyperparameter optimization with Optuna)
# - A: Astronomy (predicting star formation rates in galaxies)
# - R: Regressors (Random Forest and Gradient Boosting)

import pandas as pd
import numpy as np
import shap
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split, cross_val_score, KFold
from sklearn.ensemble import RandomForestRegressor, GradientBoostingRegressor
from sklearn.metrics import r2_score
from sklearn.preprocessing import StandardScaler, MinMaxScaler
from gplearn.genetic import SymbolicRegressor
import optuna
import warnings
from numpy.polynomial import Polynomial
from scipy.spatial import cKDTree

warnings.filterwarnings("ignore")

# Load the original dataset
df = pd.read_csv("merged_data.csv")

# Load VizieR data (update paths as needed)
sdss_spec = pd.read_csv("sdss_dr16_spec.csv")  # V/154: Spectroscopic data
sdss_photo = pd.read_csv("sdss_dr12_photo.csv")  # V/147: Photometric data
```

```
twomass = pd.read_csv("2mass_xsc.csv")  # II/246: 2MASS Extended Source Catalogue
gama = pd.read_csv("gama_dr3.csv")  # II/356: GAMA DR3

# Print columns for debugging
print("Columns in merged_data.csv:", df.columns.tolist())
print("Columns in sdss_spec (V/154):", sdss_spec.columns.tolist())
print("Columns in sdss_photo (V/147):", sdss_photo.columns.tolist())
print("Columns in twomass (II/246):", twomass.columns.tolist())
print("Columns in gama (II/356):", gama.columns.tolist())

# Function to clean DataFrame by removing rows with NaN or inf in RA/Dec columns
def clean_coordinates(df, ra_col, dec_col):
    initial_len = len(df)
    # Check for NaN or inf in RA and Dec columns
    mask = (
        df[ra_col].notna() & df[dec_col].notna() &  # Not NaN
        np.isfinite(df[ra_col]) & np.isfinite(df[dec_col])  # Not inf
    )
    df_cleaned = df[mask].copy()
    print(f"Removed {initial_len - len(df_cleaned)} rows from DataFrame due to NaN or inf in {ra_col} or
{dec_col}")
    return df_cleaned

# Cross-match function
def deg_to_rad(df, ra_col, dec_col):
    try:
        df['ra_rad'] = np.radians(df[ra_col])
        df['dec_rad'] = np.radians(df[dec_col])
        return df
    except KeyError as e:
        print(f"KeyError in deg_to_rad: {e}")
        print(f"Available columns in DataFrame: {df.columns.tolist()}")
        raise

def cross_match(df1, df2, ra_col1, dec_col1, ra_col2, dec_col2, max_dist_arcsec=1.0):
    try:
        # Clean DataFrames to remove rows with NaN or inf in RA/Dec
        df1_cleaned = clean_coordinates(df1, ra_col1, dec_col1)
        df2_cleaned = clean_coordinates(df2, ra_col2, dec_col2)

        # Convert degrees to radians
        df1_cleaned = deg_to_rad(df1_cleaned, ra_col1, dec_col1)
        df2_cleaned = deg_to_rad(df2_cleaned, ra_col2, dec_col2)

        # Create coordinate arrays
        coords1 = np.array([df1_cleaned['ra_rad'], df1_cleaned['dec_rad']]).T
        coords2 = np.array([df2_cleaned['ra_rad'], df2_cleaned['dec_rad']]).T

        # Perform cross-match using cKDTree
```

```
        tree = cKDTree(coords2)
        max_dist = np.radians(max_dist_arcsec / 3600.0)
        dist, idx = tree.query(coords1, k=1, distance_upper_bound=max_dist)

        # Filter matches within the maximum distance
        matched = dist < max_dist
        df1_matched = df1_cleaned[matched].copy()
        df2_matched = df2_cleaned.iloc[idx[matched]].copy()

        # Reset indices
        df1_matched = df1_matched.reset_index(drop=True)
        df2_matched = df2_matched.reset_index(drop=True)

        return df1_matched, df2_matched
    except KeyError as e:
        print(f"KeyError in cross_match: {e}")
        print(f"df1 columns: {df1.columns.tolist()}")
        print(f"df2 columns: {df2.columns.tolist()}")
        raise
    except Exception as e:
        print(f"Error in cross_match: {e}")
        raise

# Cross-match with VizieR catalogues
# V/154: SDSS DR16 spectroscopic data (uses RA_ICRS, DE_ICRS)
df, sdss_spec_matched = cross_match(df, sdss_spec, 'objra_y', 'objdec', 'RA_ICRS', 'DE_ICRS')
df = pd.concat([df, sdss_spec_matched[['zsp']]], axis=1)  # Use zsp instead of specz
print(f"After V/154 cross-match: {len(df)} galaxies")

# V/147: SDSS DR12 photometric data (uses RA_ICRS, DE_ICRS)
df, sdss_photo_matched = cross_match(df, sdss_photo, 'objra_y', 'objdec', 'RA_ICRS', 'DE_ICRS')
df = pd.concat([df, sdss_photo_matched[['umag', 'gmag', 'rmag', 'imag', 'zmag', 'e_umag', 'e_gmag',
'e_rmag', 'e_imag', 'e_zmag']]], axis=1)
print(f"After V/147 cross-match: {len(df)} galaxies")

# II/246: 2MASS Extended Source Catalogue (uses RAJ2000, DEJ2000)
df, twomass_matched = cross_match(df, twomass, 'objra_y', 'objdec', 'RAJ2000', 'DEJ2000')
df = pd.concat([df, twomass_matched[['Jmag', 'Hmag', 'Kmag', 'e_Jmag', 'e_Hmag', 'e_Kmag']]], axis=1)
print(f"After II/246 cross-match: {len(df)} galaxies")

# II/356: GAMA DR3 (uses RAJ2000, DEJ2000)
df, gama_matched = cross_match(df, gama, 'objra_y', 'objdec', 'RAJ2000', 'DEJ2000')
df = pd.concat([df, gama_matched[['F(U)', 'F(B)', 'F(V)', 'UmAB', 'BmAB', 'VmAB']]], axis=1)
print(f"After II/356 cross-match: {len(df)} galaxies")

# Recompute log_SFR_Ha and metallicity using fluxes from merged_data.csv
df['flux_Ha'] = df['F_Ha_cen']  # Use F_Ha_cen as flux_Ha
df['e_flux_Ha'] = df['e_F_Ha_cen']
df['flux_Hb'] = df['flux_Hbeta4861.36_Re_fit']
```

```
df['flux_OIII_5007'] = df['flux_[OIII]5006.84_Re_fit']
df['flux_NII_6584'] = df['flux_[NII]6583.45_Re_fit']

df['Ha_Hb_observed'] = df['flux_Ha'] / df['flux_Hb']
Ha_Hb_intrinsic = 2.86
k_Ha = 2.468
k_Hb = 3.634
df['A_Ha'] = 2.5 * np.log10(df['Ha_Hb_observed'] / Ha_Hb_intrinsic) * (k_Ha / (k_Hb - k_Ha))
df['flux_Ha_corr'] = df['flux_Ha'] * 10**(0.4 * df['A_Ha'])
H0 = 70  # km/s/Mpc
c = 3e5  # km/s
df['DL'] = (c * df['zsp'] / H0) * 3.0856e24  # cm, using zsp
df['L_Ha'] = df['flux_Ha_corr'] * 4 * np.pi * df['DL']**2  # erg/s
df['log_SFR_Ha_raw'] = np.log10(df['L_Ha'] * 7.9e-42)  # M_sun/yr
df['log_O3N2_raw'] = np.log10((df['flux_OIII_5007'] / df['flux_Hb']) / (df['flux_NII_6584'] / df['flux_Ha']))
df['OH_O3N2_raw'] = 8.533 - 0.214 * df['log_O3N2_raw']

# Add raw fluxes as features
df['log_flux_Ha'] = np.log10(df['flux_Ha'] + 1e-10)
df['log_flux_Hb'] = np.log10(df['flux_Hb'] + 1e-10)
df['log_flux_OIII_5007'] = np.log10(df['flux_OIII_5007'] + 1e-10)
df['log_flux_NII_6584'] = np.log10(df['flux_NII_6584'] + 1e-10)
df['log_e_flux_Ha'] = np.log10(df['e_flux_Ha'] + 1e-10)

# Add photometric colors (from V/147)
df['color_ug'] = df['umag'] - df['gmag']
df['color_gr'] = df['gmag'] - df['rmag']
df['color_ri'] = df['rmag'] - df['imag']
df['color_iz'] = df['imag'] - df['zmag']
df['e_color_ug'] = np.sqrt(df['e_umag']**2 + df['e_gmag']**2)
df['e_color_gr'] = np.sqrt(df['e_gmag']**2 + df['e_rmag']**2)

# Add 2MASS colors (from II/246)
df['color_JK'] = df['Jmag'] - df['Kmag']
df['e_color_JK'] = np.sqrt(df['e_Jmag']**2 + df['e_Kmag']**2)

# Drop rows with missing target or excessive NaNs
df = df.dropna(subset=["log_SFR_Ha"])
df = df.dropna(axis=0, thresh=int(0.8 * df.shape[1]))
df.fillna(df.median(numeric_only=True), inplace=True)

# Step 1: Cosmo-Rank Construction
gz_df = df.copy()
morph_cols = ['P(CD)', 'P(E)', 'P(S0)', 'P(Sa)', 'P(Sab)', 'P(Sb)', 'P(Sbc)',
              'P(Sc)', 'P(Scd)', 'P(Sd)', 'P(Sdm)', 'P(Sm)', 'P(Irr)']
gz_df['morph_sum'] = gz_df[morph_cols].sum(axis=1)
gz_df['Num_w'] = gz_df['conf_prob']
scaler_rank = MinMaxScaler()
gz_df[['morph_sum_norm', 'Num_w_norm', 'nsa_z_norm']] = scaler_rank.fit_transform(
```

```
    gz_df[['morph_sum', 'Num_w', 'nsa_z']]
)
gz_df['cosmo_rank'] = (0.5 * gz_df['Num_w_norm'] + 0.3 * gz_df['morph_sum_norm'] + 0.2 *
gz_df['nsa_z_norm'])
df['cosmo_rank'] = gz_df['cosmo_rank']

# Step 2: L_cosmo(s) Construction
alpha = -1.5
M_star = df['log_Mass_gas'].median()
df['a_n'] = (10 ** df['log_Mass_gas'])**(1 + alpha) * np.exp(-10 ** df['log_Mass_gas'] / (10 ** M_star))
bins = pd.qcut(df['Re_kpc'], q=20, labels=False, duplicates='drop') + 1
df['z_bin'] = bins
n_bins = df['z_bin'].nunique()
print(f"Number of unique bins after qcut: {n_bins}")
print("Bin distribution for Re_kpc (after qcut):")
print(df['z_bin'].value_counts().sort_index())

s_vals = [0.5, 1.0, 1.5, 2.0]
s_range
```

sdss12 spec = RA_ICRS DE_ICRS mode q_mode class SDSS12 m_SDSS12 objID Sp-ID ObsDate Q umag e_umag gmag e_gmag rmag e_rmag imag e_imag zmag e_zmag zsp rpmag pmRA pmDE zph e_zph <zph>

Thank you for providing the column names for the SDSS DR12 spectroscopic data (`sdss12_spec`). It looks like this might be the catalog you're using for `sdss_spec` (loaded as `sdss_dr16_spec.csv` in the script), or it could be a different catalog you intended to use. Let's analyze the columns you provided and address the issue in the script.

Step 1: Analyze the Provided Columns for `sdss12_spec`

The columns you listed for `sdss12_spec` are:

RA_ICRS DE_ICRS mode q_mode class SDSS12 m_SDSS12 objID Sp-ID ObsDate Q umag e_umag gmag e_gmag rmag e_rmag imag e_imag zmag e_zmag zsp rpmag pmRA pmDE zph e_zph <zph>

- **Positional Columns**: `RA_ICRS`, `DE_ICRS` (used for cross-matching, which is working fine).
- **Redshift Columns**: `zsp` (spectroscopic redshift), `zph` (photometric redshift), `<zph>` (possibly a mean or median photometric redshift).
- **Magnitude Columns**: `umag`, `gmag`, `rmag`, `imag`, `zmag` (magnitudes in u, g, r, i, z bands) and their errors (`e_umag`, `e_gmag`, etc.).
- **Other Columns**: `mode`, `q_mode`, `class`, `SDSS12`, `m_SDSS12`, `objID`, `Sp-ID`, `ObsDate`, `Q`, `rpmag`, `pmRA`, `pmDE` (proper motion in RA and Dec).

Key Observation: This catalog, like the SDSS DR16 spectroscopic catalog (V/154) previously listed, does **not** contain emission line fluxes such as `flux_Ha`, `flux_Hb`, `flux_OIII_5007`, `flux_NII_6584`, or `e_flux_Ha`. It provides redshifts (`zsp`) and magnitudes, but not the detailed spectral line fluxes needed for the star formation rate (SFR) calculations in the script.

Step 2: Revisit the Previous Error

The error in the script occurred because the script attempted to extract `specz`, `flux_Ha`, `flux_Hb`, `flux_OIII_5007`, `flux_NII_6584`, and `e_flux_Ha` from `sdss_spec_matched` (derived from `sdss_spec`), but these columns were not present in `sdss_spec`. The updated script I provided in the previous response addressed this by:

1. Using `zsp` instead of `specz` for the redshift.
2. Using flux columns already present in `merged_data.csv` (e.g., `F_Ha_cen`, `flux_Hbeta4861.36_Re_fit`, etc.) instead of expecting them from `sdss_spec`.

However, the updated script was cut off in your message before it could be fully executed, so we couldn't see if it resolved the issue or if new errors emerged. Let's complete the script and ensure it runs smoothly.

Step 3: Complete and Update the Script

The updated script already uses `zsp` and relies on `merged_data.csv` for the flux columns. I'll complete the script, ensuring all sections are included, and make a few additional adjustments based on the `sdss12_spec` columns you provided:

1. **Confirm Redshift Column**: The script uses `zsp` for the redshift, which is present in both the SDSS DR16 (V/154) and the `sdss12_spec` columns you provided.
2. **Photometric Magnitudes**: The script uses `umag`, `gmag`, `rmag`, `imag`, `zmag`, and their errors from `sdss_photo` (V/147), which matches the columns in `sdss12_spec`. We'll ensure consistency.
3. **Fluxes**: We'll continue using the fluxes from `merged_data.csv` (e.g., `F_Ha_cen`, `flux_Hbeta4861.36_Re_fit`, etc.).
4. **GAMA Columns**: The script attempts to extract `Flux_Ha`, `Flux_NII_6584`, `e_Flux_Ha`, `u_mag`, `g_mag`, etc., from the GAMA catalog, but the GAMA columns you provided (`F(U)`, `F(B)`, `F(V)`, etc.) don't include these fluxes. We'll adjust the GAMA concatenation to use available columns.

Here’s the complete, updated script:

```
python
# STAR.py: Model for predicting log_SFR_Ha using the STAR framework
```

```
# STAR: Symbolic, Tuned (Optuna), Astronomy Regressors (Random Forest + Gradient Boosting)
# - S: Symbolic Regressor (via gplearn)
# - T: Tuned (hyperparameter optimization with Optuna)
# - A: Astronomy (predicting star formation rates in galaxies)
# - R: Regressors (Random Forest and Gradient Boosting)

import pandas as pd
import numpy as np
import shap
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split, cross_val_score, KFold
from sklearn.ensemble import RandomForestRegressor, GradientBoostingRegressor
from sklearn.metrics import r2_score
from sklearn.preprocessing import StandardScaler, MinMaxScaler
from gplearn.genetic import SymbolicRegressor
import optuna
import warnings
from numpy.polynomial import Polynomial
from scipy.spatial import cKDTree

warnings.filterwarnings("ignore")

# Load the original dataset
df = pd.read_csv("merged_data.csv")

# Load VizieR data (update paths as needed)
sdss_spec = pd.read_csv("sdss_dr16_spec.csv")  # V/154: Spectroscopic data
sdss_photo = pd.read_csv("sdss_dr12_photo.csv")  # V/147: Photometric data
twomass = pd.read_csv("2mass_xsc.csv")  # II/246: 2MASS Extended Source Catalogue
gama = pd.read_csv("gama_dr3.csv")  # II/356: GAMA DR3

# Print columns for debugging
print("Columns in merged_data.csv:", df.columns.tolist())
print("Columns in sdss_spec (V/154):", sdss_spec.columns.tolist())
print("Columns in sdss_photo (V/147):", sdss_photo.columns.tolist())
print("Columns in twomass (II/246):", twomass.columns.tolist())
print("Columns in gama (II/356):", gama.columns.tolist())

# Function to clean DataFrame by removing rows with NaN or inf in RA/Dec columns
def clean_coordinates(df, ra_col, dec_col):
    initial_len = len(df)
    # Check for NaN or inf in RA and Dec columns
    mask = (
        df[ra_col].notna() & df[dec_col].notna() &  # Not NaN
        np.isfinite(df[ra_col]) & np.isfinite(df[dec_col])  # Not inf
    )
    df_cleaned = df[mask].copy()
```

```
    print(f"Removed {initial_len - len(df_cleaned)} rows from DataFrame due to NaN or inf in {ra_col} or
{dec_col}")
    return df_cleaned

# Cross-match function
def deg_to_rad(df, ra_col, dec_col):
    try:
        df['ra_rad'] = np.radians(df[ra_col])
        df['dec_rad'] = np.radians(df[dec_col])
        return df
    except KeyError as e:
        print(f"KeyError in deg_to_rad: {e}")
        print(f"Available columns in DataFrame: {df.columns.tolist()}")
        raise

def cross_match(df1, df2, ra_col1, dec_col1, ra_col2, dec_col2, max_dist_arcsec=1.0):
    try:
        # Clean DataFrames to remove rows with NaN or inf in RA/Dec
        df1_cleaned = clean_coordinates(df1, ra_col1, dec_col1)
        df2_cleaned = clean_coordinates(df2, ra_col2, dec_col2)

        # Convert degrees to radians
        df1_cleaned = deg_to_rad(df1_cleaned, ra_col1, dec_col1)
        df2_cleaned = deg_to_rad(df2_cleaned, ra_col2, dec_col2)

        # Create coordinate arrays
        coords1 = np.array([df1_cleaned['ra_rad'], df1_cleaned['dec_rad']]).T
        coords2 = np.array([df2_cleaned['ra_rad'], df2_cleaned['dec_rad']]).T

        # Perform cross-match using cKDTree
        tree = cKDTree(coords2)
        max_dist = np.radians(max_dist_arcsec / 3600.0)
        dist, idx = tree.query(coords1, k=1, distance_upper_bound=max_dist)

        # Filter matches within the maximum distance
        matched = dist < max_dist
        df1_matched = df1_cleaned[matched].copy()
        df2_matched = df2_cleaned.iloc[idx[matched]].copy()

        # Reset indices
        df1_matched = df1_matched.reset_index(drop=True)
        df2_matched = df2_matched.reset_index(drop=True)

        return df1_matched, df2_matched
    except KeyError as e:
        print(f"KeyError in cross_match: {e}")
        print(f"df1 columns: {df1.columns.tolist()}")
        print(f"df2 columns: {df2.columns.tolist()}")
        raise
```

```
    except Exception as e:
        print(f"Error in cross_match: {e}")
        raise

# Cross-match with VizieR catalogues
# V/154: SDSS DR16 spectroscopic data (uses RA_ICRS, DE_ICRS)
df, sdss_spec_matched = cross_match(df, sdss_spec, 'objra_y', 'objdec', 'RA_ICRS', 'DE_ICRS')
df = pd.concat([df, sdss_spec_matched[['zsp']]], axis=1)  # Use zsp instead of specz
print(f"After V/154 cross-match: {len(df)} galaxies")

# V/147: SDSS DR12 photometric data (uses RA_ICRS, DE_ICRS)
df, sdss_photo_matched = cross_match(df, sdss_photo, 'objra_y', 'objdec', 'RA_ICRS', 'DE_ICRS')
df = pd.concat([df, sdss_photo_matched[['umag', 'gmag', 'rmag', 'imag', 'zmag', 'e_umag', 'e_gmag',
'e_rmag', 'e_imag', 'e_zmag']]], axis=1)
print(f"After V/147 cross-match: {len(df)} galaxies")

# II/246: 2MASS Extended Source Catalogue (uses RAJ2000, DEJ2000)
df, twomass_matched = cross_match(df, twomass, 'objra_y', 'objdec', 'RAJ2000', 'DEJ2000')
df = pd.concat([df, twomass_matched[['Jmag', 'Hmag', 'Kmag', 'e_Jmag', 'e_Hmag', 'e_Kmag']]], axis=1)
print(f"After II/246 cross-match: {len(df)} galaxies")

# II/356: GAMA DR3 (uses RAJ2000, DEJ2000)
df, gama_matched = cross_match(df, gama, 'objra_y', 'objdec', 'RAJ2000', 'DEJ2000')
df = pd.concat([df, gama_matched[['UmAB', 'BmAB', 'VmAB']]], axis=1)  # Use available magnitude
columns
print(f"After II/356 cross-match: {len(df)} galaxies")

# Recompute log_SFR_Ha and metallicity using fluxes from merged_data.csv
df['flux_Ha'] = df['F_Ha_cen']  # Use F_Ha_cen as flux_Ha
df['e_flux_Ha'] = df['e_F_Ha_cen']
df['flux_Hb'] = df['flux_Hbeta4861.36_Re_fit']
df['flux_OIII_5007'] = df['flux_[OIII]5006.84_Re_fit']
df['flux_NII_6584'] = df['flux_[NII]6583.45_Re_fit']

# Check for NaN in flux columns and handle them
flux_cols = ['flux_Ha', 'flux_Hb', 'flux_OIII_5007', 'flux_NII_6584', 'e_flux_Ha']
for col in flux_cols:
    nan_count = df[col].isna().sum()
    print(f"NaN count in {col}: {nan_count}")
    # Replace NaN with median for flux columns
    if nan_count > 0:
        df[col] = df[col].fillna(df[col].median())

df['Ha_Hb_observed'] = df['flux_Ha'] / df['flux_Hb']
Ha_Hb_intrinsic = 2.86
k_Ha = 2.468
k_Hb = 3.634
df['A_Ha'] = 2.5 * np.log10(df['Ha_Hb_observed'] / Ha_Hb_intrinsic) * (k_Ha / (k_Hb - k_Ha))
df['flux_Ha_corr'] = df['flux_Ha'] * 10**(0.4 * df['A_Ha'])
```

```
H0 = 70  # km/s/Mpc
c = 3e5  # km/s
df['DL'] = (c * df['zsp'] / H0) * 3.0856e24  # cm, using zsp
df['L_Ha'] = df['flux_Ha_corr'] * 4 * np.pi * df['DL']**2  # erg/s
df['log_SFR_Ha_raw'] = np.log10(df['L_Ha'] * 7.9e-42)  # M_sun/yr
df['log_O3N2_raw'] = np.log10((df['flux_OIII_5007'] / df['flux_Hb']) / (df['flux_NII_6584'] / df['flux_Ha']))
df['OH_O3N2_raw'] = 8.533 - 0.214 * df['log_O3N2_raw']

# Add raw fluxes as features
df['log_flux_Ha'] = np.log10(df['flux_Ha'] + 1e-10)
df['log_flux_Hb'] = np.log10(df['flux_Hb'] + 1e-10)
df['log_flux_OIII_5007'] = np.log10(df['flux_OIII_5007'] + 1e-10)
df['log_flux_NII_6584'] = np.log10(df['flux_NII_6584'] + 1e-10)
df['log_e_flux_Ha'] = np.log10(df['e_flux_Ha'] + 1e-10)

# Add photometric colors (from V/147)
df['color_ug'] = df['umag'] - df['gmag']
df['color_gr'] = df['gmag'] - df['rmag']
df['color_ri'] = df['rmag'] - df['imag']
df['color_iz'] = df['imag'] - df['zmag']
df['e_color_ug'] = np.sqrt(df['e_umag']**2 + df['e_gmag']**2)
df['e_color_gr'] = np.sqrt(df['e_gmag']**2 + df['e_rmag']**2)

# Add 2MASS colors (from II/246)
df['color_JK'] = df['Jmag'] - df['Kmag']
df['e_color_JK'] = np.sqrt(df['e_Jmag']**2 + df['e_Kmag']**2)

# Drop rows with missing target or excessive NaNs
df = df.dropna(subset=["log_SFR_Ha"])
df = df.dropna(axis=0, thresh=int(0.8 * df.shape[1]))
df.fillna(df.median(numeric_only=True), inplace=True)

# Step 1: Cosmo-Rank Construction
gz_df = df.copy()
morph_cols = ['P(CD)', 'P(E)', 'P(S0)', 'P(Sa)', 'P(Sab)', 'P(Sb)', 'P(Sbc)',
              'P(Sc)', 'P(Scd)', 'P(Sd)', 'P(Sdm)', 'P(Sm)', 'P(Irr)']
gz_df['morph_sum'] = gz_df[morph_cols].sum(axis=1)
gz_df['Num_w'] = gz_df['conf_prob']
scaler_rank = MinMaxScaler()
gz_df[['morph_sum_norm', 'Num_w_norm', 'nsa_z_norm']] = scaler_rank.fit_transform(
    gz_df[['morph_sum', 'Num_w', 'nsa_z']]
)
gz_df['cosmo_rank'] = (0.5 * gz_df['Num_w_norm'] + 0.3 * gz_df['morph_sum_norm'] + 0.2 *
gz_df['nsa_z_norm'])
df['cosmo_rank'] = gz_df['cosmo_rank']

# Step 2: L_cosmo(s) Construction
alpha = -1.5
M_star = df['log_Mass_gas'].median()
```

```
df['a_n'] = (10 ** df['log_Mass_gas'])**(1 + alpha) * np.exp(-10 ** df['log_Mass_gas'] / (10 ** M_star))
bins = pd.qcut(df['Re_kpc'], q=20, labels=False, duplicates='drop') + 1
df['z_bin'] = bins
n_bins = df['z_bin'].nunique()
print(f"Number of unique bins after qcut: {n_bins}")
print("Bin distribution for Re_kpc (after qcut):")
print(df['z_bin'].value_counts().sort_index())

s_vals = [0.5, 1.0, 1.5, 2.0]
s_range = np.linspace(0.5, 2, 50)
for s in s_vals:
    col_name = f'L_cosmo_s_{s:.1f}'
    df[col_name] = df.groupby('z_bin')['a_n'].transform('mean') / (df['z_bin'] ** s)

print("Columns after creating L_cosmo_s*:")
print(df.columns.tolist())

print("Checking for NaN/infinite values in cosmo_rank and L_cosmo_s_1.0:")
print("cosmo_rank NaN count:", df['cosmo_rank'].isna().sum())
print("L_cosmo_s_1.0 NaN count:", df['L_cosmo_s_1.0'].isna().sum())
print("cosmo_rank infinite count:", np.isinf(df['cosmo_rank']).sum())
print("L_cosmo_s_1.0 infinite count:", np.isinf(df['L_cosmo_s_1.0']).sum())

df['cosmo_rank_L'] = df['cosmo_rank'] * df['L_cosmo_s_1.0']
df['cosmo_rank_L'] = df['cosmo_rank_L'].replace([np.inf, -np.inf], np.nan).fillna(0)
print("cosmo_rank_L created, NaN count:", df['cosmo_rank_L'].isna().sum())

# Plot L_cosmo(s) vs s
grouped_means = df.groupby('z_bin')['a_n'].mean().reindex(range(1, n_bins + 1), fill_value=0)
L_vals = [grouped_means / (np.arange(1, n_bins + 1) ** s) for s in s_range]
L_vals = np.array([sum(l) for l in L_vals])
plt.plot(s_range, L_vals)
plt.xlabel("s")
plt.ylabel("L_cosmo(s)")
plt.title("L_cosmo(s) Behavior")
plt.savefig("L_cosmo_curve.png")
plt.close()

# Scatter plot of L_cosmo_s_1.0 vs log_SFR_Ha
plt.scatter(df['L_cosmo_s_1.0'], df['log_SFR_Ha'], alpha=0.5)
plt.xlabel("L_cosmo_s_1.0")
plt.ylabel("log_SFR_Ha")
plt.title("L_cosmo_s_1.0 vs log_SFR_Ha")
plt.savefig("L_cosmo_s1_vs_SFR.png")
plt.close()

# Step 3: Define features and target
target = "log_SFR_Ha"
base_features = [
```

```
    "log_Mass_gas", "log_Mass", "Av_gas_Re", "Av_ssp_Re",
    "OH_O3N2_cen", "OH_T04_cen", "OH_dop_cen", "Age_LW_Re_fit",
    "ZH_LW_Re_fit", "Re_kpc", "vel_disp_Ha_cen", "Lambda_Re",
    "Sigma_Mass_Re", "EW_Ha_cen", "Ha_Hb_cen"
]
df["mass_metallicity"] = df["log_Mass"] * df["OH_O3N2_cen"]
df["dust_metallicity"] = df["Av_gas_Re"] * df["OH_T04_cen"]
df["disp_mass_ratio"] = df["vel_disp_Ha_cen"] / (df["Sigma_Mass_Re"] + 1e-5)
df["age_metallicity"] = df["Age_LW_Re_fit"] * df["ZH_LW_Re_fit"]
df["sqrt_Re_kpc"] = np.sqrt(df["Re_kpc"] + 1e-5)
df["log_mass"] = np.log(df["log_Mass_gas"] + 1e-5)
df["BSD_likelihood"] = (
    df["log_Mass_gas"] * df["OH_O3N2_cen"] /
    (df["Av_gas_Re"] + 1e-5 + df["Age_LW_Re_fit"])
)
df["cosmo_rank_mass"] = df["cosmo_rank"] * df["log_Mass"]
df["L_cosmo_s1_mass"] = df["L_cosmo_s_1.0"] * df["log_Mass"]
df["cosmo_rank_EW"] = df["cosmo_rank"] * df["EW_Ha_cen"]
df["L_cosmo_s1_EW"] = df["L_cosmo_s_1.0"] * df["EW_Ha_cen"]
df["cosmo_rank_scaled"] = df["cosmo_rank"] * 20
df["L_cosmo_s1_scaled"] = df["L_cosmo_s_1.0"] * 20
df["cosmo_rank_L_mass"] = df["cosmo_rank_L"] * df["log_Mass"]
df["L_cosmo_s1_metallicity"] = df["L_cosmo_s_1.0"] * df["OH_O3N2_cen"]
features = base_features + [
    "mass_metallicity", "dust_metallicity", "disp_mass_ratio",
    "age_metallicity", "sqrt_Re_kpc", "log_mass", "BSD_likelihood",
    "cosmo_rank", "L_cosmo_s_0.5", "L_cosmo_s_1.0", "L_cosmo_s_1.5",
    "L_cosmo_s_2.0", "cosmo_rank_L", "cosmo_rank_mass", "L_cosmo_s1_mass",
    "cosmo_rank_EW", "L_cosmo_s1_EW", "cosmo_rank_scaled", "L_cosmo_s1_scaled",
    "cosmo_rank_L_mass", "L_cosmo_s1_metallicity",
    "log_flux_Ha", "log_flux_Hb", "log_flux_OIII_5007", "log_flux_NII_6584",
    "log_e_flux_Ha", "log_SFR_Ha_raw", "OH_O3N2_raw",
    "color_ug", "color_gr", "color_ri", "color_iz", "e_color_ug", "e_color_gr",
    "color_JK", "e_color_JK"
]

X = df[features]
y = df[target]

# Train/test split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Normalize features
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# Optuna optimization for Random Forest
def objective_rf(trial):
```

```
    params = {
        'n_estimators': trial.suggest_int('n_estimators', 100, 500),
        'max_depth': trial.suggest_int('max_depth', 5, 20),
        'min_samples_split': trial.suggest_int('min_samples_split', 2, 10),
        'random_state': 42
    }
    model = RandomForestRegressor(**params)
    cv = KFold(n_splits=5, shuffle=True, random_state=42)
    scores = cross_val_score(model, X_train_scaled, y_train, cv=cv, scoring='r2')
    return scores.mean()

study_rf = optuna.create_study(direction='maximize')
study_rf.optimize(objective_rf, n_trials=20)
best_params_rf = study_rf.best_params
print("Best Random Forest Parameters:", best_params_rf)
print("Best Random Forest CV R²:", study_rf.best_value)

# Train Random Forest with best parameters
rf = RandomForestRegressor(**best_params_rf, random_state=42)
rf.fit(X_train_scaled, y_train)
y_pred_rf = rf.predict(X_test_scaled)
r2_rf = r2_score(y_test, y_pred_rf)

# Cross-validation for final RF model
cv_scores_rf = cross_val_score(rf, X_train_scaled, y_train, cv=5, scoring='r2')
print(f"Random Forest 5-Fold CV R²: Mean = {cv_scores_rf.mean():.4f}, Std = {cv_scores_rf.std():.4f}")

# Optuna optimization for Gradient Boosting
def objective_gb(trial):
    params = {
        'n_estimators': trial.suggest_int('n_estimators', 100, 500),
        'learning_rate': trial.suggest_float('learning_rate', 0.01, 0.2, log=True),
        'max_depth': trial.suggest_int('max_depth', 3, 10),
        'random_state': 42
    }
    model = GradientBoostingRegressor(**params)
    cv = KFold(n_splits=5, shuffle=True, random_state=42)
    scores = cross_val_score(model, X_train_scaled, y_train, cv=cv, scoring='r2')
    return scores.mean()

study_gb = optuna.create_study(direction='maximize')
study_gb.optimize(objective_gb, n_trials=20)
best_params_gb = study_gb.best_params
print("Best Gradient Boosting Parameters:", best_params_gb)
print("Best Gradient Boosting CV R²:", study_gb.best_value)

# Train Gradient Boosting with best parameters
gb = GradientBoostingRegressor(**best_params_gb, random_state=42)
gb.fit(X_train_scaled, y_train)
```

```
y_pred_gb = gb.predict(X_test_scaled)
r2_gb = r2_score(y_test, y_pred_gb)

# Cross-validation for final GB model
cv_scores_gb = cross_val_score(gb, X_train_scaled, y_train, cv=5, scoring='r2')
print(f"Gradient Boosting 5-Fold CV R²: Mean = {cv_scores_gb.mean():.4f}, Std =
{cv_scores_gb.std():.4f}")

print(f"R^2 Score (Random Forest, Test Set): {r2_rf:.4f}")
print(f"R^2 Score (Gradient Boosting, Test Set): {r2_gb:.4f}")

# SHAP for feature selection
explainer_rf = shap.Explainer(rf, X_train_scaled)
shap_values_rf = explainer_rf(X_test_scaled)
explainer_gb = shap.Explainer(gb, X_train_scaled)
shap_values_gb = explainer_gb(X_test_scaled)
shap.summary_plot(shap_values_rf, X_test_scaled, plot_type="bar", show=False)
plt.title("SHAP Summary - Random Forest")
plt.tight_layout()
plt.savefig("shap_rf_summary.png")
plt.clf()
shap.summary_plot(shap_values_gb, X_test_scaled, plot_type="bar", show=False)
plt.title("SHAP Summary - Gradient Boosting")
plt.tight_layout()
plt.savefig("shap_gb_summary.png")
plt.clf()

# Feature selection based on SHAP
shap_values_rf_mean = np.abs(shap_values_rf.values).mean(axis=0)
feature_importance = pd.DataFrame({"feature": features, "importance": shap_values_rf_mean})
feature_importance = feature_importance.sort_values("importance", ascending=False)
top_features = feature_importance["feature"].head(10).tolist()
bsd_features = ["cosmo_rank", "L_cosmo_s_1.0", "cosmo_rank_L", "cosmo_rank_scaled",
"L_cosmo_s1_scaled", "cosmo_rank_L_mass", "L_cosmo_s1_metallicity"]
for bsd_feature in bsd_features:
    if bsd_feature not in top_features:
        top_features.append(bsd_feature)
X_selected = df[top_features]
X_train_selected, X_test_selected, _, _ = train_test_split(X_selected, y, test_size=0.2, random_state=42)
X_train_selected_scaled = scaler.fit_transform(X_train_selected)
X_test_selected_scaled = scaler.transform(X_test_selected)

# Symbolic Regression with gplearn
symbolic_model = SymbolicRegressor(
    population_size=3000,
    generations=150,
    stopping_criteria=0.01,
    p_crossover=0.7,
    p_subtree_mutation=0.1,
```

```
    p_hoist_mutation=0.05,
    p_point_mutation=0.1,
    max_samples=0.9,
    verbose=1,
    parsimony_coefficient=0.0001,
    random_state=42,
    function_set=('add', 'sub', 'mul', 'div', 'sqrt', 'log', 'sin', 'cos')
)
symbolic_model.fit(X_train_selected_scaled, y_train)
y_pred_sym = symbolic_model.predict(X_test_selected_scaled)
r2_sym = r2_score(y_test, y_pred_sym)
print(f"gplearn R²: {r2_sym:.4f}")
print("Symbolic Expression:")
print(symbolic_model._program)

# Cross-validation for symbolic model
cv_scores_sym = cross_val_score(symbolic_model, X_train_selected_scaled, y_train, cv=5, scoring='r2')
print(f"Symbolic Regression 5-Fold CV R²: Mean = {cv_scores_sym.mean():.4f}, Std =
{cv_scores_sym.std():.4f}")

# Save symbolic expression as polynomial fit plot and print the equation
x = np.linspace(min(y_test), max(y_test), 500)
y_expr = symbolic_model.predict(scaler.transform(np.tile(X_test_selected.mean().values, (500,1))))
p = Polynomial.fit(y_test, y_pred_sym, deg=3)
# Extract polynomial coefficients
coeffs = p.coef
print("Polynomial Fit Equation (degree 3):")
print(f"y = {coeffs[0]:.4f} + {coeffs[1]:.4f}*x + {coeffs[2]:.4f}*x^2 + {coeffs[3]:.4f}*x^3")
# Plot the polynomial fit
plt.plot(*p.linspace(), label="Polynomial Fit")
plt.scatter(y_test, y_pred_sym, s=10, alpha=0.5, label="Symbolic Predictions")
plt.xlabel("True log_SFR_Ha")
plt.ylabel("Predicted log_SFR_Ha")
plt.title("Symbolic Regression Fit with Polynomial")
plt.legend()
plt.tight_layout()
plt.savefig("gplearn_expression_plot.png")
plt.clf()

# Plot predictions vs true
plt.scatter(y_test, y_pred_rf, alpha=0.5, label="RF", marker="o")
plt.scatter(y_test, y_pred_gb, alpha=0.5, label="GB", marker="s")
plt.scatter(y_test, y_pred_sym, alpha=0.5, label="Symbolic", marker="^")
plt.plot([y.min(), y.max()], [y.min(), y.max()], "k--")
plt.xlabel("True log_SFR_Ha")
plt.ylabel("Predicted")
plt.title("Model Predictions vs True Values")
plt.legend()
plt.tight_layout()
```

```
plt.savefig("model_comparison.png")
plt.clf()

# Residuals and Correlation Analysis
residuals = pd.DataFrame({
    "True": y_test,
    "RF": y_pred_rf - y_test,
    "GB": y_pred_gb - y_test,
    "Symbolic": y_pred_sym - y_test,
    "cosmo_rank": X_test["cosmo_rank"],
    "L_cosmo_s1": X_test["L_cosmo_s_1.0"]
})
sns.scatterplot(data=residuals, x="True", y="Symbolic", hue="cosmo_rank", size="L_cosmo_s1",
alpha=0.6)
plt.axhline(0, color="black", linestyle="--")
plt.title("Symbolic Residuals vs True log_SFR_Ha")
plt.savefig("residuals_sym.png")
plt.clf()

print("Correlation of BSD Features with Residuals:")
print(residuals[["RF", "GB", "Symbolic", "cosmo_rank", "L_cosmo_s1"]].corr()[["RF", "GB", "Symbolic"]])
```

Step 4: Key Changes in the Updated Script

1. **SDSS Spectroscopic Data**:
    - Uses `zsp` for redshift, which is present in both the SDSS DR16 (V/154) and the `sdss12_spec` columns you provided.
    - Does not attempt to extract flux columns from `sdss_spec`, as they are not available.
2. **Fluxes**:
    - Uses `F_Ha_cen`, `flux_Hbeta4861.36_Re_fit`, `flux_[OIII]5006.84_Re_fit`, and `flux_[NII]6583.45_Re_fit` from `merged_data.csv`.
    - Added a check for `NaN` values in the flux columns and replaces them with the median to prevent errors in the SFR calculations.
3. **GAMA Data**:
    - Adjusted the GAMA concatenation to use `UmAB`, `BmAB`, and `VmAB` (AB magnitudes in U, B, V bands), as the previously requested columns (`Flux_Ha`, `Flux_NII_6584`, etc.) are not available in the GAMA catalog.
4. **Parallel Processing**:
    - Removed `n_jobs` parameters from `train_test_split`, `cross_val_score`, and model training to avoid potential issues with parallel processing in your environment. You can reintroduce them (e.g., `n_jobs=2`) if your system supports it and you want to speed up the computations.

Step 5: Expected Output

The script should now:

1. Complete the cross-matching steps for all catalogs, printing the number of galaxies after each match.
2. Compute `log_SFR_Ha_raw` and `OH_O3N2_raw` using the fluxes from `merged_data.csv`.
3. Proceed through the feature engineering, model training, and evaluation steps.
4. Produce all plots (`L_cosmo_curve.png`, `shap_rf_summary.png`, etc.) and print the R^2 scores, symbolic expression, and polynomial fit equation.

Example output might look like:

```
After V/154 cross-match: 1000 galaxies
After V/147 cross-match: 900 galaxies
After II/246 cross-match: 800 galaxies
After II/356 cross-match: 700 galaxies
NaN count in flux_Ha: 10
NaN count in flux_Hb: 20
...
Best Random Forest Parameters: {'n_estimators': 300, 'max_depth': 15, 'min_samples_split': 5}
Best Random Forest CV R²: 0.85
...
R^2 Score (Random Forest, Test Set): 0.82
R^2 Score (Gradient Boosting, Test Set): 0.80
gplearn R²: 0.78
Symbolic Expression: ...
Polynomial Fit Equation (degree 3): y = 0.1234 + 0.5678*x + 0.2345*x^2 + 0.0123*x^3
```

Step 6: If Further Errors Occur

If you encounter additional errors, please provide:

1. The exact error message and traceback.
2. The output up to the point of the error (e.g., number of galaxies after each cross-match, NaN counts in flux columns).
3. Any specific concerns, such as if you have a different dataset with the emission line fluxes.

Step 7: Additional Notes

- **Emission Line Fluxes**: If you have access to a MaNGA catalog or another dataset that contains the emission line fluxes (`flux_Ha`, `flux_Hb`, etc.), we can incorporate it into the script. For now, we're using the fluxes available in `merged_data.csv`.
- **Performance**: The script may take some time to run due to the Optuna optimization and symbolic regression steps. If it's too slow, you can reduce `n_trials` in Optuna (e.g., from 20 to 10) or `generations` in `SymbolicRegressor` (e.g., from 150 to 50).
- **Data Quality**: The script now handles `NaN` values in the flux columns by replacing them with the median. If a large number of flux values are `NaN`, you might want to investigate the data quality in `merged_data.csv`.

Let me know how this works! We're definitely getting closer to a fully functional script.
almost got it dialed in
(myenv) root@BigsbyGaming:/home/pmqr7# python STAR.py Columns in merged_data.csv: ['mangaid', 'nsa_sersic_mass', 'nsa_z', 'OBJDEC_x', 'objra_x', 'SFR_PETRORAD_R', 'zcomp_1Mpc', 'zcomp_5Mpc', 'footprint_1', 'footprint_5', 'PLATEIFU', 'OBJRA', 'OBJDEC_y', 'LOGMSTARS', 'SINI', 'VOPT', 'SESSION', 'EXP', 'RMS', 'LOGHILIM200KMS', 'PEAK', 'SNR', 'FHI', 'EFHI', 'LOGMHI', 'VHI', 'EV', 'WM50', 'WP50', 'WP20', 'W2P50', 'WF50', 'DW', 'PR', 'PL', 'AR', 'BR', 'AL', 'BL', 'NEGDET', 'BLSTRUCT', 'conflag', 'conf_prob', 'name', 'plate', 'ifudsgn', 'plateifu', 'objra_y', 'objdec', 'log_SFR_Ha', 'FoV', 'Re_kpc', 'e_log_Mass', 'e_log_SFR_Ha', 'log_Mass', 'log_SFR_ssp', 'log_NII_Ha_cen', 'e_log_NII_Ha_cen', 'log_OIII_Hb_cen', 'e_log_OIII_Hb_cen', 'log_SII_Ha_cen', 'e_log_SII_Ha_cen', 'log_OII_Hb_cen', 'e_log_OII_Hb_cen', 'EW_Ha_cen', 'e_EW_Ha_cen', 'ZH_LW_Re_fit', 'e_ZH_LW_Re_fit', 'alpha_ZH_LW_Re_fit', 'e_alpha_ZH_LW_Re_fit', 'ZH_MW_Re_fit', 'e_ZH_MW_Re_fit', 'alpha_ZH_MW_Re_fit', 'e_alpha_ZH_MW_Re_fit', 'Age_LW_Re_fit', 'e_Age_LW_Re_fit', 'alpha_Age_LW_Re_fit', 'e_alpha_Age_LW_Re_fit', 'Age_MW_Re_fit', 'e_Age_MW_Re_fit', 'alpha_Age_MW_Re_fit', 'e_alpha_Age_MW_Re_fit', 'Re_arc', 'DL', 'DA', 'PA', 'ellip', 'log_Mass_gas', 'vel_sigma_Re', 'e_vel_sigma_Re', 'log_SFR_SF', 'log_SFR_D_C', 'OH_O3N2_cen', 'e_OH_O3N2_cen', 'OH_N2_cen', 'e_OH_N2_cen', 'OH_ONS_cen', 'e_OH_ONS_cen', 'OH_R23_cen', 'e_OH_R23_cen', 'OH_pyqz_cen', 'e_OH_pyqz_cen', 'OH_t2_cen', 'e_OH_t2_cen', 'OH_M08_cen', 'e_OH_M08_cen', 'OH_T04_cen', 'e_OH_T04_cen', 'OH_dop_cen', 'e_OH_dop_cen', 'OH_O3N2_EPM09_cen', 'e_OH_O3N2_EPM09_cen', 'log_OI_Ha_cen', 'e_log_OI_Ha_cen', 'Ha_Hb_cen', 'e_Ha_Hb_cen', 'log_NII_Ha_Re', 'e_log_NII_Ha_Re', 'log_OIII_Hb_Re', 'e_log_OIII_Hb_Re', 'log_SII_Ha_Re', 'e_log_SII_Ha_Re', 'log_OII_Hb_Re', 'e_log_OII_Hb_Re', 'log_OI_Ha_Re', 'e_log_OI_Ha_Re', 'EW_Ha_Re', 'e_EW_Ha_Re', 'Ha_Hb_Re', 'e_Ha_Hb_Re', 'log_NII_Ha_ALL', 'e_log_NII_Ha_ALL', 'log_OIII_Hb_ALL', 'e_log_OIII_Hb_ALL', 'log_SII_Ha_ALL', 'e_log_SII_Ha_ALL', 'log_OII_Hb_ALL', 'e_log_OII_Hb_ALL', 'log_OI_Ha_ALL', 'e_log_OI_Ha_ALL', 'EW_Ha_ALL', 'e_EW_Ha_ALL', 'Ha_Hb_ALL', 'Sigma_Mass_cen', 'e_Sigma_Mass_cen', 'Sigma_Mass_Re', 'e_Sigma_Mass_Re', 'Sigma_Mass_ALL', 'e_Sigma_Mass_ALL', 'T30', 'ZH_T30', 'ZH_Re_T30', 'a_ZH_T30', 'T40', 'ZH_T40', 'ZH_Re_T40', 'a_ZH_T40', 'T50',

'ZH_T50', 'ZH_Re_T50', 'a_ZH_T50', 'T60', 'ZH_T60', 'ZH_Re_T60', 'a_ZH_T60', 'T70',
'ZH_T70', 'ZH_Re_T70', 'a_ZH_T70', 'T80', 'ZH_T80', 'ZH_Re_T80', 'a_ZH_T80', 'T90',
'ZH_T90', 'ZH_Re_T90', 'a_ZH_T90', 'T95', 'ZH_T95', 'ZH_Re_T95', 'a_ZH_T95', 'T99',
'ZH_T99', 'ZH_Re_T99', 'a_ZH_T99', 'log_Mass_gas_Av_gas_OH',
'log_Mass_gas_Av_ssp_OH', 'vel_ssp_2', 'e_vel_ssp_2', 'vel_Ha_2', 'e_vel_Ha_2',
'vel_ssp_1', 'e_vel_ssp_1', 'vel_Ha_1', 'e_vel_Ha_1', 'log_SFR_ssp_100Myr',
'log_SFR_ssp_10Myr', 'vel_disp_Ha_cen', 'vel_disp_ssp_cen', 'vel_disp_Ha_1Re',
'vel_disp_ssp_1Re', 'log_Mass_in_Re', 'ML_int', 'ML_avg', 'F_Ha_cen', 'e_F_Ha_cen',
'R50_kpc_V', 'e_R50_kpc_V', 'R50_kpc_Mass', 'e_R50_kpc_Mass',
'log_Mass_corr_in_R50_V', 'e_log_Mass_corr_in_R50_V',
'log_Mass_gas_Av_gas_log_log', 'Av_gas_Re', 'e_Av_gas_Re', 'Av_ssp_Re',
'e_Av_ssp_Re', 'Lambda_Re', 'e_Lambda_Re', 'nsa_redshift', 'nsa_mstar', 'nsa_inclination',
'flux_[OII]3726.03_Re_fit', 'e_flux_[OII]3726.03_Re_fit', 'flux_[OII]3726.03_alpha_fit',
'e_flux_[OII]3726.03_alpha_fit', 'flux_[OII]3728.82_Re_fit', 'e_flux_[OII]3728.82_Re_fit',
'flux_[OII]3728.82_alpha_fit', 'e_flux_[OII]3728.82_alpha_fit', 'flux_HI3734.37_Re_fit',
'e_flux_HI3734.37_Re_fit', 'flux_HI3734.37_alpha_fit', 'e_flux_HI3734.37_alpha_fit',
'flux_HI3797.9_Re_fit', 'e_flux_HI3797.9_Re_fit', 'flux_HI3797.9_alpha_fit',
'e_flux_HI3797.9_alpha_fit', 'flux_HeI3888.65_Re_fit', 'e_flux_HeI3888.65_Re_fit',
'flux_HeI3888.65_alpha_fit', 'e_flux_HeI3888.65_alpha_fit', 'flux_HI3889.05_Re_fit',
'e_flux_HI3889.05_Re_fit', 'flux_HI3889.05_alpha_fit', 'e_flux_HI3889.05_alpha_fit',
'flux_HeI3964.73_Re_fit', 'e_flux_HeI3964.73_Re_fit', 'flux_HeI3964.73_alpha_fit',
'e_flux_HeI3964.73_alpha_fit', 'flux_[NeIII]3967.46_Re_fit', 'e_flux_[NeIII]3967.46_Re_fit',
'flux_[NeIII]3967.46_alpha_fit', 'e_flux_[NeIII]3967.46_alpha_fit', 'flux_CaII3968.47_Re_fit',
'e_flux_CaII3968.47_Re_fit', 'flux_CaII3968.47_alpha_fit', 'e_flux_CaII3968.47_alpha_fit',
'flux_Hepsilon3970.07_Re_fit', 'e_flux_Hepsilon3970.07_Re_fit',
'flux_Hepsilon3970.07_alpha_fit', 'e_flux_Hepsilon3970.07_alpha_fit',
'flux_Hdelta4101.77_Re_fit', 'e_flux_Hdelta4101.77_Re_fit', 'flux_Hdelta4101.77_alpha_fit',
'e_flux_Hdelta4101.77_alpha_fit', 'flux_Hgamma4340.49_Re_fit',
'e_flux_Hgamma4340.49_Re_fit', 'flux_Hgamma4340.49_alpha_fit',
'e_flux_Hgamma4340.49_alpha_fit', 'flux_Hbeta4861.36_Re_fit',
'e_flux_Hbeta4861.36_Re_fit', 'flux_Hbeta4861.36_alpha_fit',
'e_flux_Hbeta4861.36_alpha_fit', 'flux_[OIII]4958.91_Re_fit', 'e_flux_[OIII]4958.91_Re_fit',
'flux_[OIII]4958.91_alpha_fit', 'e_flux_[OIII]4958.91_alpha_fit', 'flux_[OIII]5006.84_Re_fit',
'e_flux_[OIII]5006.84_Re_fit', 'flux_[OIII]5006.84_alpha_fit', 'e_flux_[OIII]5006.84_alpha_fit',
'flux_HeI5015.68_Re_fit', 'e_flux_HeI5015.68_Re_fit', 'flux_HeI5015.68_alpha_fit',
'e_flux_HeI5015.68_alpha_fit', 'flux_[NI]5197.9_Re_fit', 'e_flux_[NI]5197.9_Re_fit',
'flux_[NI]5197.9_alpha_fit', 'e_flux_[NI]5197.9_alpha_fit', 'flux_[NI]5200.26_Re_fit',
'e_flux_[NI]5200.26_Re_fit', 'flux_[NI]5200.26_alpha_fit', 'e_flux_[NI]5200.26_alpha_fit',
'flux_HeI5876.0_Re_fit', 'e_flux_HeI5876.0_Re_fit', 'flux_HeI5876.0_alpha_fit',
'e_flux_HeI5876.0_alpha_fit', 'flux_NaI5889.95_Re_fit', 'e_flux_NaI5889.95_Re_fit',
'flux_NaI5889.95_alpha_fit', 'e_flux_NaI5889.95_alpha_fit', 'flux_NaI5895.92_Re_fit',
'e_flux_NaI5895.92_Re_fit', 'flux_NaI5895.92_alpha_fit', 'e_flux_NaI5895.92_alpha_fit',

'flux_[OI]6300.3_Re_fit', 'e_flux_[OI]6300.3_Re_fit', 'flux_[OI]6300.3_alpha_fit',
'e_flux_[OI]6300.3_alpha_fit', 'flux_[NII]6548.05_Re_fit', 'e_flux_[NII]6548.05_Re_fit',
'flux_[NII]6548.05_alpha_fit', 'e_flux_[NII]6548.05_alpha_fit', 'flux_Halpha6562.85_Re_fit',
'e_flux_Halpha6562.85_Re_fit', 'flux_Halpha6562.85_alpha_fit',
'e_flux_Halpha6562.85_alpha_fit', 'flux_[NII]6583.45_Re_fit', 'e_flux_[NII]6583.45_Re_fit',
'flux_[NII]6583.45_alpha_fit', 'e_flux_[NII]6583.45_alpha_fit', 'flux_[SII]6716.44_Re_fit',
'e_flux_[SII]6716.44_Re_fit', 'flux_[SII]6716.44_alpha_fit', 'e_flux_[SII]6716.44_alpha_fit',
'flux_[SII]6730.82_Re_fit', 'e_flux_[SII]6730.82_Re_fit', 'flux_[SII]6730.82_alpha_fit',
'e_flux_[SII]6730.82_alpha_fit', 'flux_[ArIII]7135.8_Re_fit', 'e_flux_[ArIII]7135.8_Re_fit',
'flux_[ArIII]7135.8_alpha_fit', 'e_flux_[ArIII]7135.8_alpha_fit', 'flux_HI9014.91_Re_fit',
'e_flux_HI9014.91_Re_fit', 'flux_HI9014.91_alpha_fit', 'e_flux_HI9014.91_alpha_fit',
'flux_[SIII]9069.0_Re_fit', 'e_flux_[SIII]9069.0_Re_fit', 'flux_[SIII]9069.0_alpha_fit',
'e_flux_[SIII]9069.0_alpha_fit', 'flux_[FeII]9470.93_Re_fit', 'e_flux_[FeII]9470.93_Re_fit',
'flux_[FeII]9470.93_alpha_fit', 'e_flux_[FeII]9470.93_alpha_fit', 'flux_[SIII]9531.1_Re_fit',
'e_flux_[SIII]9531.1_Re_fit', 'flux_[SIII]9531.1_alpha_fit', 'e_flux_[SIII]9531.1_alpha_fit',
'OH_Mar13_N2_Re_fit', 'e_OH_Mar13_N2_Re_fit', 'OH_Mar13_N2_alpha_fit',
'e_OH_Mar13_N2_alpha_fit', 'OH_Mar13_O3N2_Re_fit', 'e_OH_Mar13_O3N2_Re_fit',
'OH_Mar13_O3N2_alpha_fit', 'e_OH_Mar13_O3N2_alpha_fit', 'OH_T04_Re_fit',
'e_OH_T04_Re_fit', 'OH_T04_alpha_fit', 'e_OH_T04_alpha_fit', 'OH_Pet04_N2_lin_Re_fit',
'e_OH_Pet04_N2_lin_Re_fit', 'OH_Pet04_N2_lin_alpha_fit',
'e_OH_Pet04_N2_lin_alpha_fit', 'OH_Pet04_N2_poly_Re_fit',
'e_OH_Pet04_N2_poly_Re_fit', 'OH_Pet04_N2_poly_alpha_fit',
'e_OH_Pet04_N2_poly_alpha_fit', 'OH_Pet04_O3N2_Re_fit', 'e_OH_Pet04_O3N2_Re_fit',
'OH_Pet04_O3N2_alpha_fit', 'e_OH_Pet04_O3N2_alpha_fit', 'OH_Kew02_N2O2_Re_fit',
'e_OH_Kew02_N2O2_Re_fit', 'OH_Kew02_N2O2_alpha_fit',
'e_OH_Kew02_N2O2_alpha_fit', 'OH_Pil10_ONS_Re_fit', 'e_OH_Pil10_ONS_Re_fit',
'OH_Pil10_ONS_alpha_fit', 'e_OH_Pil10_ONS_alpha_fit', 'OH_Pil10_ON_Re_fit',
'e_OH_Pil10_ON_Re_fit', 'OH_Pil10_ON_alpha_fit', 'e_OH_Pil10_ON_alpha_fit',
'OH_Pil11_NS_Re_fit', 'e_OH_Pil11_NS_Re_fit', 'OH_Pil11_NS_alpha_fit',
'e_OH_Pil11_NS_alpha_fit', 'OH_Cur20_RS32_Re_fit', 'e_OH_Cur20_RS32_Re_fit',
'OH_Cur20_RS32_alpha_fit', 'e_OH_Cur20_RS32_alpha_fit', 'OH_Cur20_R3_Re_fit',
'e_OH_Cur20_R3_Re_fit', 'OH_Cur20_R3_alpha_fit', 'e_OH_Cur20_R3_alpha_fit',
'OH_Cur20_O3O2_Re_fit', 'e_OH_Cur20_O3O2_Re_fit', 'OH_Cur20_O3O2_alpha_fit',
'e_OH_Cur20_O3O2_alpha_fit', 'OH_Cur20_S2_Re_fit', 'e_OH_Cur20_S2_Re_fit',
'OH_Cur20_S2_alpha_fit', 'e_OH_Cur20_S2_alpha_fit', 'OH_Cur20_R2_Re_fit',
'e_OH_Cur20_R2_Re_fit', 'OH_Cur20_R2_alpha_fit', 'e_OH_Cur20_R2_alpha_fit',
'OH_Cur20_N2_Re_fit', 'e_OH_Cur20_N2_Re_fit', 'OH_Cur20_N2_alpha_fit',
'e_OH_Cur20_N2_alpha_fit', 'OH_Cur20_R23_Re_fit', 'e_OH_Cur20_R23_Re_fit',
'OH_Cur20_R23_alpha_fit', 'e_OH_Cur20_R23_alpha_fit', 'OH_Cur20_O3N2_Re_fit',
'e_OH_Cur20_O3N2_Re_fit', 'OH_Cur20_O3N2_alpha_fit', 'e_OH_Cur20_O3N2_alpha_fit',
'OH_Cur20_O3S2_Re_fit', 'e_OH_Cur20_O3S2_Re_fit', 'OH_Cur20_O3S2_alpha_fit',
'e_OH_Cur20_O3S2_alpha_fit', 'OH_KK04_Re_fit', 'e_OH_KK04_Re_fit',

'OH_KK04_alpha_fit', 'e_OH_KK04_alpha_fit', 'OH_Pil16_R_Re_fit',
'e_OH_Pil16_R_Re_fit', 'OH_Pil16_R_alpha_fit', 'e_OH_Pil16_R_alpha_fit',
'OH_Pil16_S_Re_fit', 'e_OH_Pil16_S_Re_fit', 'OH_Pil16_S_alpha_fit',
'e_OH_Pil16_S_alpha_fit', 'OH_Ho_Re_fit', 'e_OH_Ho_Re_fit', 'OH_Ho_alpha_fit',
'e_OH_Ho_alpha_fit', 'U_Dors_O32_Re_fit', 'e_U_Dors_O32_Re_fit',
'U_Dors_O32_alpha_fit', 'e_U_Dors_O32_alpha_fit', 'U_Dors_S_Re_fit',
'e_U_Dors_S_Re_fit', 'U_Dors_S_alpha_fit', 'e_U_Dors_S_alpha_fit',
'U_Mor16_O23_fs_Re_fit', 'e_U_Mor16_O23_fs_Re_fit', 'U_Mor16_O23_fs_alpha_fit',
'e_U_Mor16_O23_fs_alpha_fit', 'U_Mor16_O23_ts_Re_fit', 'e_U_Mor16_O23_ts_Re_fit',
'U_Mor16_O23_ts_alpha_fit', 'e_U_Mor16_O23_ts_alpha_fit', 'NH_Pil16_R_Re_fit',
'e_NH_Pil16_R_Re_fit', 'NH_Pil16_R_alpha_fit', 'e_NH_Pil16_R_alpha_fit',
'NO_Pil16_R_Re_fit', 'e_NO_Pil16_R_Re_fit', 'NO_Pil16_R_alpha_fit',
'e_NO_Pil16_R_alpha_fit', 'NO_Pil16_Ho_R_Re_fit', 'e_NO_Pil16_Ho_R_Re_fit',
'NO_Pil16_Ho_R_alpha_fit', 'e_NO_Pil16_Ho_R_alpha_fit', 'NO_Pil16_N2_R2_Re_fit',
'e_NO_Pil16_N2_R2_Re_fit', 'NO_Pil16_N2_R2_alpha_fit', 'e_NO_Pil16_N2_R2_alpha_fit',
'Ne_Oster_S_Re_fit', 'e_Ne_Oster_S_Re_fit', 'Ne_Oster_S_alpha_fit',
'e_Ne_Oster_S_alpha_fit', 'Hd_Re_fit', 'e_Hd_Re_fit', 'Hd_alpha_fit', 'e_Hd_alpha_fit',
'Hb_Re_fit', 'e_Hb_Re_fit', 'Hb_alpha_fit', 'e_Hb_alpha_fit', 'Mgb_Re_fit', 'e_Mgb_Re_fit',
'Mgb_alpha_fit', 'e_Mgb_alpha_fit', 'Fe5270_Re_fit', 'e_Fe5270_Re_fit', 'Fe5270_alpha_fit',
'e_Fe5270_alpha_fit', 'Fe5335_Re_fit', 'e_Fe5335_Re_fit', 'Fe5335_alpha_fit',
'e_Fe5335_alpha_fit', 'D4000_Re_fit1', 'e_D4000_Re_fit', 'D4000_alpha_fit',
'e_D4000_alpha_fit', 'Hdmod_Re_fit', 'e_Hdmod_Re_fit', 'Hdmod_alpha_fit',
'e_Hdmod_alpha_fit', 'Hg_Re_fit', 'e_Hg_Re_fit', 'Hg_alpha_fit', 'e_Hg_alpha_fit',
'u_band_mag', 'u_band_mag_error', 'u_band_abs_mag', 'u_band_abs_mag_error',
'g_band_mag', 'g_band_mag_error', 'g_band_abs_mag', 'g_band_abs_mag_error',
'r_band_mag', 'r_band_mag_error', 'r_band_abs_mag', 'r_band_abs_mag_error',
'i_band_mag', 'i_band_mag_error', 'i_band_abs_mag', 'i_band_abs_mag_error',
'B_band_mag', 'B_band_mag_error', 'B_band_abs_mag', 'B_band_abs_mag_error',
'V_band_mag', 'V_band_mag_error', 'V_band_abs_mag', 'V_band_abs_mag_error',
'RJ_band_mag', 'RJ_band_mag_error', 'RJ_band_abs_mag', 'RJ_band_abs_mag_error',
'R50', 'error_R50', 'R90', 'error_R90', 'C', 'e_C', 'B-V', 'error_B-V', 'B-R', 'error_B-R',
'log_Mass_phot', 'e_log_Mass_phot', 'V-band_SB_at_Re', 'error_V-band_SB_at_Re',
'V-band_SB_at_R_50', 'error_V-band_SB_at_R_50', 'nsa_sersic_n_morph', 'u-g', 'g-r', 'r-i',
'i-z', 'P(CD)', 'P(E)', 'P(S0)', 'P(Sa)', 'P(Sab)', 'P(Sb)', 'P(Sbc)', 'P(Sc)', 'P(Scd)', 'P(Sd)',
'P(Sdm)', 'P(Sm)', 'P(Irr)', 'best_type_n', 'best_type', 'nsa_nsaid', 'Vmax_w', 'Num_w',
'QCFLAG'] Columns in sdss_spec (V/154): ['objID', 'RA_ICRS', 'DE_ICRS', 'mode', 'class',
'clean', 'e_RA_ICRS', 'e_DE_ICRS', 'umag', 'gmag', 'rmag', 'imag', 'zmag', 'e_umag',
'e_gmag', 'e_rmag', 'e_imag', 'e_zmag', 'zsp', 'e_zsp', 'f_zsp', 'pmRA', 'pmDE', 'zph', 'e_zph',
'<zph>', 'Q', 'SDSS16', 'Sp-ID', 'MJD', 'zs', 'zPrad', 'zPA', 'zoRA', 'zoDE', 'zMJD', 'zFlags',
'zdVrad', 'zdVell', 'zc', 'Vdisp', 'us', 'uPrad', 'uPA', 'uoRA', 'uoDE', 'uMJD', 'uFlags', 'udVrad',
'udVell', 'uc', 'subCl', 'Status', 'spType', 'SpObjID', 'spInst', 'spCl', 'sigRA', 'sigDE', 'rs', 'rPrad',
'rPA', 'roRA', 'roDE', 'rMJD', 'rFlags', 'rdVrad', 'rdVell', 'rc', 'parentID', 'N', 'M', 'iPrad', 'iPA',

'ioRA', 'ioDE', 'iMJD', 'iFlags', 'idVrad', 'idVell', 'ic', 'gs', 'gPrad', 'gPA', 'goRA', 'goDE', 'gMJD', 'gFlags', 'gdVrad', 'gdVell', 'gc', 'flags', 'fieldID', 'e_zPrad', 'e_Vdisp', 'e_uPrad', 'e_rPrad', 'e_pmRA', 'e_pmDE', 'e_iPrad', 'e_gPrad', 'chi2', 'zPmag', 'zpmag', 'uPmag', 'upmag', 'spS/N', 'SDSS-ID', 'rPmag', 'rpmag', 'r(F)', 'r(E)', 'is', 'iPmag', 'ipmag', 'i(N)', 'gPmag', 'gpmag', 'g(O)', 'g(J)', 'e_zPmag', 'e_zpmag', 'e_uPmag', 'e_upmag', 'e_rPmag', 'e_rpmag', 'e_iPmag', 'e_ipmag', 'e_gPmag', 'e_gpmag'] Columns in sdss_photo (V/147): ['RA_ICRS', 'DE_ICRS', 'mode', 'q_mode', 'class', 'SDSS12', 'm_SDSS12', 'objID', 'Sp-ID', 'ObsDate', 'Q', 'umag', 'e_umag', 'gmag', 'e_gmag', 'rmag', 'e_rmag', 'imag', 'e_imag', 'zmag', 'e_zmag', 'zsp', 'rpmag', 'pmRA', 'pmDE', 'zph', 'e_zph', '<zph>'] Columns in twomass (II/246): ['RAJ2000', 'DEJ2000', '2MASS', 'Jmag', 'e_Jmag', 'Hmag', 'e_Hmag', 'Kmag', 'e_Kmag', 'Qflg', 'Rflg', 'Bflg', 'Cflg', 'Xflg', 'Aflg', 'XMMOM', 'IDsum', 'ObsID', 'ID', 'UVW2r', 'UVM2r', 'UVM1r', 'Ur', 'Br', 'Vr', 'RAJ2000.1', 'DEJ2000.1', 'ePos', 'Nobs', 'sig(UVW2)', 'sig(UVM2)', 'sig(UVW1)', 'sig(U)', 'sig(B)', 'sig(V)', 'F(UVW2)', 'F(UVM2)', 'F(UVW1)', 'F(U)', 'F(B)', 'F(V)', 'UVW2mAB', 'UVM2mAB', 'UVW1mAB', 'UmAB', 'BmAB', 'VmAB', 'q_UVW2', 'q_UVM2', 'q_UVW1', 'q_U', 'q_B', 'q_V', 'xUVW2', 'xUVM2', 'xUVW1', 'xU', 'xB', 'xV'] Columns in gama (II/356): ['XMMOM', 'IDsum', 'ObsID', 'ID', 'UVW2r', 'UVM2r', 'UVM1r', 'Ur', 'Br', 'Vr', 'RAJ2000', 'DEJ2000', 'ePos', 'Nobs', 'sig(UVW2)', 'sig(UVM2)', 'sig(UVW1)', 'sig(U)', 'sig(B)', 'sig(V)', 'F(UVW2)', 'F(UVM2)', 'F(UVW1)', 'F(U)', 'F(B)', 'F(V)', 'UVW2mAB', 'UVM2mAB', 'UVW1mAB', 'UmAB', 'BmAB', 'VmAB', 'q_UVW2', 'q_UVM2', 'q_UVW1', 'q_U', 'q_B', 'q_V', 'xUVW2', 'xUVM2', 'xUVW1', 'xU', 'xB', 'xV'] Removed 561 rows from DataFrame due to NaN or inf in objra_y or objdec Removed 0 rows from DataFrame due to NaN or inf in RA_ICRS or DE_ICRS After V/154 cross-match: 0 galaxies Removed 0 rows from DataFrame due to NaN or inf in objra_y or objdec Removed 0 rows from DataFrame due to NaN or inf in RA_ICRS or DE_ICRS After V/147 cross-match: 0 galaxies Removed 0 rows from DataFrame due to NaN or inf in objra_y or objdec Removed 0 rows from DataFrame due to NaN or inf in RAJ2000 or DEJ2000 After II/246 cross-match: 0 galaxies Removed 0 rows from DataFrame due to NaN or inf in objra_y or objdec Removed 0 rows from DataFrame due to NaN or inf in RAJ2000 or DEJ2000 After II/356 cross-match: 0 galaxies NaN count in flux_Ha: 0 NaN count in flux_Hb: 0 NaN count in flux_OIII_5007: 0 NaN count in flux_NII_6584: 0 NaN count in e_flux_Ha: 0 Traceback (most recent call last): File "/home/pmqr7/STAR.py", line 184, in <module> gz_df[['morph_sum_norm', 'Num_w_norm', 'nsa_z_norm']] = scaler_rank.fit_transform(^^^^^^^^^^^^^^^^^^^^^^^^^^^^ File "/home/pmqr7/myenv/lib/python3.12/site-packages/sklearn/utils/_set_output.py", line 319, in wrapped data_to_wrap = f(self, X, *args, **kwargs) ^^^^^^^^^^^^^^^^^^^^^^^^^^ File "/home/pmqr7/myenv/lib/python3.12/site-packages/sklearn/base.py", line 918, in fit_transform return self.fit(X, **fit_params).transform(X) ^^^^^^^^^^^^^^^^^^^^^^^^ File "/home/pmqr7/myenv/lib/python3.12/site-packages/sklearn/preprocessing/_data.py", line 447, in fit return self.partial_fit(X, y) ^^^^^^^^^^^^^^^^^^^^ File "/home/pmqr7/myenv/lib/python3.12/site-packages/sklearn/base.py", line 1389, in wrapper return fit_method(estimator, *args, **kwargs) ^^ File "/home/pmqr7/myenv/lib/python3.12/site-packages/sklearn/preprocessing/_data.py", line 487, in partial_fit X = validate_data(^^^^^^^^^^^^^^^ File

"/home/pmqr7/myenv/lib/python3.12/site-packages/sklearn/utils/validation.py", line 2944, in validate_data out = check_array(X, input_name="X", **check_params) ^^^

- Removed 561 rows from DataFrame due to NaN or inf in objra_y or objdec
- This suggests that `merged_data.csv` initially had some rows (at least 561), but after cleaning, the remaining rows did not match any entries in `sdss_spec`.
- No rows were removed from `sdss_spec` due to invalid coordinates:
- Removed 0 rows from DataFrame due to NaN or inf in RA_ICRS or DE_ICRS

Step 2: Why Are There No Matches?

The cross-matching process is failing to find any pairs within the specified 1 arcsecond radius. This could happen for several reasons:

1. **Coordinate Mismatch**:
   - The coordinates in `merged_data.csv` (`objra_y`, `objdec`) may not align with the coordinates in `sdss_spec` (`RA_ICRS`, `DE_ICRS`).
   - For example, there could be a systematic offset in the RA/Dec values between the two catalogs, or the coordinates might be in different systems (e.g., one in degrees and the other in a different unit, though the script converts both to radians correctly).
2. **Small Matching Radius**:
   - The maximum matching distance is set to 1 arcsecond (`max_dist_arcsec=1.0`). This is a very tight constraint, and if the positional uncertainties or offsets between the catalogs are larger than 1 arcsecond, no matches will be found.
   - In astronomical cross-matching, a radius of 1–5 arcseconds is often used, depending on the astrometric precision of the catalogs.
3. **Catalog Coverage**:
   - The objects in `merged_data.csv` might not overlap with the sky regions covered by `sdss_spec`. For example, if `merged_data.csv` contains galaxies from a different survey region (e.g., MaNGA) that doesn't overlap with the SDSS DR16 spectroscopic catalog, there will be no matches.
4. **Data Issues**:
   - There could be an issue with the data in `merged_data.csv` or `sdss_spec`. For instance, if the RA/Dec columns in `merged_data.csv` are not populated correctly after the initial cleaning (e.g., all remaining values are outside the range of `sdss_spec`), no matches will be found.

Step 3: Investigate the Data

Let's examine the data to understand why no matches are being found.

Check the Coordinate Ranges

We can add some debugging to print the range of RA and Dec values in both `merged_data.csv` and `sdss_spec` after cleaning but before cross-matching. This will help us determine if there's a mismatch in the coordinate ranges.

Check the Matching Radius

The 1 arcsecond radius might be too small. We can try increasing it to 5 arcseconds to see if that yields matches.

Check Catalog Overlap

We can also check if the objects in `merged_data.csv` are expected to have counterparts in `sdss_spec`. Since `merged_data.csv` contains MaNGA-related columns (e.g., `mangaid`, `plateifu`), and MaNGA is part of SDSS-IV, there should be some overlap with the SDSS DR16 spectroscopic catalog (V/154). However, we need to confirm this.

Step 4: Update the Script to Debug and Fix the Issue

I'll modify the script to:

1. Print the RA/Dec ranges of both DataFrames after cleaning to check for coordinate mismatches.
2. Increase the matching radius to 5 arcseconds to allow for more matches.
3. Add a check to ensure `df` is not empty after the first cross-match, and if it is, provide a more informative error message and stop the script.
4. If necessary, we can skip cross-matching with `sdss_spec` and proceed with the other catalogs, or adjust the script to not rely on `sdss_spec` data (e.g., use `nsa_z` from `merged_data.csv` instead of `zsp`).

Here’s the updated script with these changes:

```
python
# STAR.py: Model for predicting log_SFR_Ha using the STAR framework
# STAR: Symbolic, Tuned (Optuna), Astronomy Regressors (Random Forest + Gradient Boosting)
# - S: Symbolic Regressor (via gplearn)
# - T: Tuned (hyperparameter optimization with Optuna)
# - A: Astronomy (predicting star formation rates in galaxies)
# - R: Regressors (Random Forest and Gradient Boosting)

import pandas as pd
import numpy as np
import shap
import matplotlib.pyplot as plt
```

```
import seaborn as sns
from sklearn.model_selection import train_test_split, cross_val_score, KFold
from sklearn.ensemble import RandomForestRegressor, GradientBoostingRegressor
from sklearn.metrics import r2_score
from sklearn.preprocessing import StandardScaler, MinMaxScaler
from gplearn.genetic import SymbolicRegressor
import optuna
import warnings
from numpy.polynomial import Polynomial
from scipy.spatial import cKDTree

warnings.filterwarnings("ignore")

# Load the original dataset
df = pd.read_csv("merged_data.csv")

# Load VizieR data (update paths as needed)
sdss_spec = pd.read_csv("sdss_dr16_spec.csv")  # V/154: Spectroscopic data
sdss_photo = pd.read_csv("sdss_dr12_photo.csv")  # V/147: Photometric data
twomass = pd.read_csv("2mass_xsc.csv")  # II/246: 2MASS Extended Source Catalogue
gama = pd.read_csv("gama_dr3.csv")  # II/356: GAMA DR3

# Print columns for debugging
print("Columns in merged_data.csv:", df.columns.tolist())
print("Columns in sdss_spec (V/154):", sdss_spec.columns.tolist())
print("Columns in sdss_photo (V/147):", sdss_photo.columns.tolist())
print("Columns in twomass (II/246):", twomass.columns.tolist())
print("Columns in gama (II/356):", gama.columns.tolist())

# Function to clean DataFrame by removing rows with NaN or inf in RA/Dec columns
def clean_coordinates(df, ra_col, dec_col):
    initial_len = len(df)
    # Check for NaN or inf in RA and Dec columns
    mask = (
        df[ra_col].notna() & df[dec_col].notna() &  # Not NaN
        np.isfinite(df[ra_col]) & np.isfinite(df[dec_col])  # Not inf
    )
    df_cleaned = df[mask].copy()
    print(f"Removed {initial_len - len(df_cleaned)} rows from DataFrame due to NaN or inf in {ra_col} or
{dec_col}")
    # Print RA/Dec ranges for debugging
    if len(df_cleaned) > 0:
        print(f"RA range in {ra_col}: {df_cleaned[ra_col].min():.4f} to {df_cleaned[ra_col].max():.4f}")
        print(f"Dec range in {dec_col}: {df_cleaned[dec_col].min():.4f} to {df_cleaned[dec_col].max():.4f}")
    else:
        print(f"No valid RA/Dec data after cleaning in {ra_col}/{dec_col}")
    return df_cleaned

# Cross-match function
```

```
def deg_to_rad(df, ra_col, dec_col):
    try:
        df['ra_rad'] = np.radians(df[ra_col])
        df['dec_rad'] = np.radians(df[dec_col])
        return df
    except KeyError as e:
        print(f"KeyError in deg_to_rad: {e}")
        print(f"Available columns in DataFrame: {df.columns.tolist()}")
        raise

def cross_match(df1, df2, ra_col1, dec_col1, ra_col2, dec_col2, max_dist_arcsec=5.0):
    try:
        # Clean DataFrames to remove rows with NaN or inf in RA/Dec
        df1_cleaned = clean_coordinates(df1, ra_col1, dec_col1)
        df2_cleaned = clean_coordinates(df2, ra_col2, dec_col2)

        # Check if either DataFrame is empty after cleaning
        if len(df1_cleaned) == 0 or len(df2_cleaned) == 0:
            print("One of the DataFrames is empty after cleaning. Cannot perform cross-match.")
            return df1_cleaned, pd.DataFrame()

        # Convert degrees to radians
        df1_cleaned = deg_to_rad(df1_cleaned, ra_col1, dec_col1)
        df2_cleaned = deg_to_rad(df2_cleaned, ra_col2, dec_col2)

        # Create coordinate arrays
        coords1 = np.array([df1_cleaned['ra_rad'], df1_cleaned['dec_rad']]).T
        coords2 = np.array([df2_cleaned['ra_rad'], df2_cleaned['dec_rad']]).T

        # Perform cross-match using cKDTree
        tree = cKDTree(coords2)
        max_dist = np.radians(max_dist_arcsec / 3600.0)
        dist, idx = tree.query(coords1, k=1, distance_upper_bound=max_dist)

        # Filter matches within the maximum distance
        matched = dist < max_dist
        df1_matched = df1_cleaned[matched].copy()
        df2_matched = df2_cleaned.iloc[idx[matched]].copy()

        # Reset indices
        df1_matched = df1_matched.reset_index(drop=True)
        df2_matched = df2_matched.reset_index(drop=True)

        return df1_matched, df2_matched
    except KeyError as e:
        print(f"KeyError in cross_match: {e}")
        print(f"df1 columns: {df1.columns.tolist()}")
        print(f"df2 columns: {df2.columns.tolist()}")
        raise
```

```
    except Exception as e:
        print(f"Error in cross_match: {e}")
        raise

# Cross-match with VizieR catalogues
# V/154: SDSS DR16 spectroscopic data (uses RA_ICRS, DE_ICRS)
df, sdss_spec_matched = cross_match(df, sdss_spec, 'objra_y', 'objdec', 'RA_ICRS', 'DE_ICRS',
max_dist_arcsec=5.0)
print(f"After V/154 cross-match: {len(df)} galaxies")
if len(df) == 0:
    raise ValueError("No matches found with SDSS DR16 spectroscopic data (V/154). Check coordinate
ranges or catalog overlap.")

# Use zsp from sdss_spec if available, otherwise fall back to nsa_z
if len(df) > 0:
    df = pd.concat([df, sdss_spec_matched[['zsp']]], axis=1)  # Use zsp instead of specz
else:
    print("Using nsa_z from merged_data.csv as redshift since no matches were found with sdss_spec.")
    df['zsp'] = df['nsa_z']

# V/147: SDSS DR12 photometric data (uses RA_ICRS, DE_ICRS)
df, sdss_photo_matched = cross_match(df, sdss_photo, 'objra_y', 'objdec', 'RA_ICRS', 'DE_ICRS',
max_dist_arcsec=5.0)
print(f"After V/147 cross-match: {len(df)} galaxies")
if len(df) > 0:
    df = pd.concat([df, sdss_photo_matched[['umag', 'gmag', 'rmag', 'imag', 'zmag', 'e_umag', 'e_gmag',
'e_rmag', 'e_imag', 'e_zmag']]], axis=1)

# II/246: 2MASS Extended Source Catalogue (uses RAJ2000, DEJ2000)
df, twomass_matched = cross_match(df, twomass, 'objra_y', 'objdec', 'RAJ2000', 'DEJ2000',
max_dist_arcsec=5.0)
print(f"After II/246 cross-match: {len(df)} galaxies")
if len(df) > 0:
    df = pd.concat([df, twomass_matched[['Jmag', 'Hmag', 'Kmag', 'e_Jmag', 'e_Hmag', 'e_Kmag']]], axis=1)

# II/356: GAMA DR3 (uses RAJ2000, DEJ2000)
df, gama_matched = cross_match(df, gama, 'objra_y', 'objdec', 'RAJ2000', 'DEJ2000',
max_dist_arcsec=5.0)
print(f"After II/356 cross-match: {len(df)} galaxies")
if len(df) > 0:
    df = pd.concat([df, gama_matched[['UmAB', 'BmAB', 'VmAB']]], axis=1)

# Recompute log_SFR_Ha and metallicity using fluxes from merged_data.csv
df['flux_Ha'] = df['F_Ha_cen']  # Use F_Ha_cen as flux_Ha
df['e_flux_Ha'] = df['e_F_Ha_cen']
df['flux_Hb'] = df['flux_Hbeta4861.36_Re_fit']
df['flux_OIII_5007'] = df['flux_[OIII]5006.84_Re_fit']
df['flux_NII_6584'] = df['flux_[NII]6583.45_Re_fit']
```

```
# Check for NaN in flux columns and handle them
flux_cols = ['flux_Ha', 'flux_Hb', 'flux_OIII_5007', 'flux_NII_6584', 'e_flux_Ha']
for col in flux_cols:
    nan_count = df[col].isna().sum()
    print(f"NaN count in {col}: {nan_count}")
    # Replace NaN with median for flux columns
    if nan_count > 0:
        df[col] = df[col].fillna(df[col].median())

df['Ha_Hb_observed'] = df['flux_Ha'] / df['flux_Hb']
Ha_Hb_intrinsic = 2.86
k_Ha = 2.468
k_Hb = 3.634
df['A_Ha'] = 2.5 * np.log10(df['Ha_Hb_observed'] / Ha_Hb_intrinsic) * (k_Ha / (k_Hb - k_Ha))
df['flux_Ha_corr'] = df['flux_Ha'] * 10**(0.4 * df['A_Ha'])
H0 = 70  # km/s/Mpc
c = 3e5  # km/s
df['DL'] = (c * df['zsp'] / H0) * 3.0856e24  # cm, using zsp
df['L_Ha'] = df['flux_Ha_corr'] * 4 * np.pi * df['DL']**2  # erg/s
df['log_SFR_Ha_raw'] = np.log10(df['L_Ha'] * 7.9e-42)  # M_sun/yr
df['log_O3N2_raw'] = np.log10((df['flux_OIII_5007'] / df['flux_Hb']) / (df['flux_NII_6584'] / df['flux_Ha']))
df['OH_O3N2_raw'] = 8.533 - 0.214 * df['log_O3N2_raw']

# Add raw fluxes as features
df['log_flux_Ha'] = np.log10(df['flux_Ha'] + 1e-10)
df['log_flux_Hb'] = np.log10(df['flux_Hb'] + 1e-10)
df['log_flux_OIII_5007'] = np.log10(df['flux_OIII_5007'] + 1e-10)
df['log_flux_NII_6584'] = np.log10(df['flux_NII_6584'] + 1e-10)
df['log_e_flux_Ha'] = np.log10(df['e_flux_Ha'] + 1e-10)

# Add photometric colors (if available)
if 'umag' in df.columns:
    df['color_ug'] = df['umag'] - df['gmag']
    df['color_gr'] = df['gmag'] - df['rmag']
    df['color_ri'] = df['rmag'] - df['imag']
    df['color_iz'] = df['imag'] - df['zmag']
    df['e_color_ug'] = np.sqrt(df['e_umag']**2 + df['e_gmag']**2)
    df['e_color_gr'] = np.sqrt(df['e_gmag']**2 + df['e_rmag']**2)

# Add 2MASS colors (if available)
if 'Jmag' in df.columns:
    df['color_JK'] = df['Jmag'] - df['Kmag']
    df['e_color_JK'] = np.sqrt(df['e_Jmag']**2 + df['e_Kmag']**2)

# Drop rows with missing target or excessive NaNs
df = df.dropna(subset=["log_SFR_Ha"])
df = df.dropna(axis=0, thresh=int(0.8 * df.shape[1]))
df.fillna(df.median(numeric_only=True), inplace=True)
```

```
# Step 1: Cosmo-Rank Construction
gz_df = df.copy()
morph_cols = ['P(CD)', 'P(E)', 'P(S0)', 'P(Sa)', 'P(Sab)', 'P(Sb)', 'P(Sbc)',
              'P(Sc)', 'P(Scd)', 'P(Sd)', 'P(Sdm)', 'P(Sm)', 'P(Irr)']
gz_df['morph_sum'] = gz_df[morph_cols].sum(axis=1)
gz_df['Num_w'] = gz_df['conf_prob']
scaler_rank = MinMaxScaler()
gz_df[['morph_sum_norm', 'Num_w_norm', 'nsa_z_norm']] = scaler_rank.fit_transform(
    gz_df[['morph_sum', 'Num_w', 'nsa_z']]
)
gz_df['cosmo_rank'] = (0.5 * gz_df['Num_w_norm'] + 0.3 * gz_df['morph_sum_norm'] + 0.2 *
gz_df['nsa_z_norm'])
df['cosmo_rank'] = gz_df['cosmo_rank']

# Step 2: L_cosmo(s) Construction
alpha = -1.5
M_star = df['log_Mass_gas'].median()
df['a_n'] = (10 ** df['log_Mass_gas'])**(1 + alpha) * np.exp(-10 ** df['log_Mass_gas'] / (10 ** M_star))
bins = pd.qcut(df['Re_kpc'], q=20, labels=False, duplicates='drop') + 1
df['z_bin'] = bins
n_bins = df['z_bin'].nunique()
print(f"Number of unique bins after qcut: {n_bins}")
print("Bin distribution for Re_kpc (after qcut):")
print(df['z_bin'].value_counts().sort_index())

s_vals = [0.5, 1.0, 1.5, 2.0]
s_range = np.linspace(0.5, 2, 50)
for s in s_vals:
    col_name = f'L_cosmo_s_{s:.1f}'
    df[col_name] = df.groupby('z_bin')['a_n'].transform('mean') / (df['z_bin'] ** s)

print("Columns after creating L_cosmo_s*:")
print(df.columns.tolist())

print("Checking for NaN/infinite values in cosmo_rank and L_cosmo_s_1.0:")
print("cosmo_rank NaN count:", df['cosmo_rank'].isna().sum())
print("L_cosmo_s_1.0 NaN count:", df['L_cosmo_s_1.0'].isna().sum())
print("cosmo_rank infinite count:", np.isinf(df['cosmo_rank']).sum())
print("L_cosmo_s_1.0 infinite count:", np.isinf(df['L_cosmo_s_1.0']).sum())

df['cosmo_rank_L'] = df['cosmo_rank'] * df['L_cosmo_s_1.0']
df['cosmo_rank_L'] = df['cosmo_rank_L'].replace([np.inf, -np.inf], np.nan).fillna(0)
print("cosmo_rank_L created, NaN count:", df['cosmo_rank_L'].isna().sum())

# Plot L_cosmo(s) vs s
grouped_means = df.groupby('z_bin')['a_n'].mean().reindex(range(1, n_bins + 1), fill_value=0)
L_vals = [grouped_means / (np.arange(1, n_bins + 1) ** s) for s in s_range]
L_vals = np.array([sum(l) for l in L_vals])
plt.plot(s_range, L_vals)
```

```
plt.xlabel("s")
plt.ylabel("L_cosmo(s)")
plt.title("L_cosmo(s) Behavior")
plt.savefig("L_cosmo_curve.png")
plt.close()

# Scatter plot of L_cosmo_s_1.0 vs log_SFR_Ha
plt.scatter(df['L_cosmo_s_1.0'], df['log_SFR_Ha'], alpha=0.5)
plt.xlabel("L_cosmo_s_1.0")
plt.ylabel("log_SFR_Ha")
plt.title("L_cosmo_s_1.0 vs log_SFR_Ha")
plt.savefig("L_cosmo_s1_vs_SFR.png")
plt.close()

# Step 3: Define features and target
target = "log_SFR_Ha"
base_features = [
    "log_Mass_gas", "log_Mass", "Av_gas_Re", "Av_ssp_Re",
    "OH_O3N2_cen", "OH_T04_cen", "OH_dop_cen", "Age_LW_Re_fit",
    "ZH_LW_Re_fit", "Re_kpc", "vel_disp_Ha_cen", "Lambda_Re",
    "Sigma_Mass_Re", "EW_Ha_cen", "Ha_Hb_cen"
]
df["mass_metallicity"] = df["log_Mass"] * df["OH_O3N2_cen"]
df["dust_metallicity"] = df["Av_gas_Re"] * df["OH_T04_cen"]
df["disp_mass_ratio"] = df["vel_disp_Ha_cen"] / (df["Sigma_Mass_Re"] + 1e-5)
df["age_metallicity"] = df["Age_LW_Re_fit"] * df["ZH_LW_Re_fit"]
df["sqrt_Re_kpc"] = np.sqrt(df["Re_kpc"] + 1e-5)
df["log_mass"] = np.log(df["log_Mass_gas"] + 1e-5)
df["BSD_likelihood"] = (
    df["log_Mass_gas"] * df["OH_O3N2_cen"] /
    (df["Av_gas_Re"] + 1e-5 + df["Age_LW_Re_fit"])
)
df["cosmo_rank_mass"] = df["cosmo_rank"] * df["log_Mass"]
df["L_cosmo_s1_mass"] = df["L_cosmo_s_1.0"] * df["log_Mass"]
df["cosmo_rank_EW"] = df["cosmo_rank"] * df["EW_Ha_cen"]
df["L_cosmo_s1_EW"] = df["L_cosmo_s_1.0"] * df["EW_Ha_cen"]
df["cosmo_rank_scaled"] = df["cosmo_rank"] * 20
df["L_cosmo_s1_scaled"] = df["L_cosmo_s_1.0"] * 20
df["cosmo_rank_L_mass"] = df["cosmo_rank_L"] * df["log_Mass"]
df["L_cosmo_s1_metallicity"] = df["L_cosmo_s_1.0"] * df["OH_O3N2_cen"]
features = base_features + [
    "mass_metallicity", "dust_metallicity", "disp_mass_ratio",
    "age_metallicity", "sqrt_Re_kpc", "log_mass", "BSD_likelihood",
    "cosmo_rank", "L_cosmo_s_0.5", "L_cosmo_s_1.0", "L_cosmo_s_1.5",
    "L_cosmo_s_2.0", "cosmo_rank_L", "cosmo_rank_mass", "L_cosmo_s1_mass",
    "cosmo_rank_EW", "L_cosmo_s1_EW", "cosmo_rank_scaled", "L_cosmo_s1_scaled",
    "cosmo_rank_L_mass", "L_cosmo_s1_metallicity",
    "log_flux_Ha", "log_flux_Hb", "log_flux_OIII_5007", "log_flux_NII_6584",
    "log_e_flux_Ha", "log_SFR_Ha_raw", "OH_O3N2_raw"
```

```
]

# Add color features if available
if 'color_ug' in df.columns:
    features.extend(["color_ug", "color_gr", "color_ri", "color_iz", "e_color_ug", "e_color_gr"])
if 'color_JK' in df.columns:
    features.extend(["color_JK", "e_color_JK"])

X = df[features]
y = df[target]

# Train/test split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Normalize features
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# Optuna optimization for Random Forest
def objective_rf(trial):
    params = {
        'n_estimators': trial.suggest_int('n_estimators', 100, 500),
        'max_depth': trial.suggest_int('max_depth', 5, 20),
        'min_samples_split': trial.suggest_int('min_samples_split', 2, 10),
        'random_state': 42
    }
    model = RandomForestRegressor(**params)
    cv = KFold(n_splits=5, shuffle=True, random_state=42)
    scores = cross_val_score(model, X_train_scaled, y_train, cv=cv, scoring='r2')
    return scores.mean()

study_rf = optuna.create_study(direction='maximize')
study_rf.optimize(objective_rf, n_trials=20)
best_params_rf = study_rf.best_params
print("Best Random Forest Parameters:", best_params_rf)
print("Best Random Forest CV R²:", study_rf.best_value)

# Train Random Forest with best parameters
rf = RandomForestRegressor(**best_params_rf, random_state=42)
rf.fit(X_train_scaled, y_train)
y_pred_rf = rf.predict(X_test_scaled)
r2_rf = r2_score(y_test, y_pred_rf)

# Cross-validation for final RF model
cv_scores_rf = cross_val_score(rf, X_train_scaled, y_train, cv=5, scoring='r2')
print(f"Random Forest 5-Fold CV R²: Mean = {cv_scores_rf.mean():.4f}, Std = {cv_scores_rf.std():.4f}")

# Optuna optimization for Gradient Boosting
```

```
def objective_gb(trial):
    params = {
        'n_estimators': trial.suggest_int('n_estimators', 100, 500),
        'learning_rate': trial.suggest_float('learning_rate', 0.01, 0.2, log=True),
        'max_depth': trial.suggest_int('max_depth', 3, 10),
        'random_state': 42
    }
    model = GradientBoostingRegressor(**params)
    cv = KFold(n_splits=5, shuffle=True, random_state=42)
    scores = cross_val_score(model, X_train_scaled, y_train, cv=cv, scoring='r2')
    return scores.mean()

study_gb = optuna.create_study(direction='maximize')
study_gb.optimize(objective_gb, n_trials=20)
best_params_gb = study_gb.best_params
print("Best Gradient Boosting Parameters:", best_params_gb)
print("Best Gradient Boosting CV R²:", study_gb.best_value)

# Train Gradient Boosting with best parameters
gb = GradientBoostingRegressor(**best_params_gb, random_state=42)
gb.fit(X_train_scaled, y_train)
y_pred_gb = gb.predict(X_test_scaled)
r2_gb = r2_score(y_test, y_pred_gb)

# Cross-validation for final GB model
cv_scores_gb = cross_val_score(gb, X_train_scaled, y_train, cv=5, scoring='r2')
print(f"Gradient Boosting 5-Fold CV R²: Mean = {cv_scores_gb.mean():.4f}, Std =
{cv_scores_gb.std():.4f}")

print(f"R^2 Score (Random Forest, Test Set): {r2_rf:.4f}")
print(f"R^2 Score (Gradient Boosting, Test Set): {r2_gb:.4f}")

# SHAP for feature selection
explainer_rf = shap.Explainer(rf, X_train_scaled)
shap_values_rf = explainer_rf(X_test_scaled)
explainer_gb = shap.Explainer(gb, X_train_scaled)
shap_values_gb = explainer_gb(X_test_scaled)
shap.summary_plot(shap_values_rf, X_test_scaled, plot_type="bar", show=False)
plt.title("SHAP Summary - Random Forest")
plt.tight_layout()
plt.savefig("shap_rf_summary.png")
plt.clf()
shap.summary_plot(shap_values_gb, X_test_scaled, plot_type="bar", show=False)
plt.title("SHAP Summary - Gradient Boosting")
plt.tight_layout()
plt.savefig("shap_gb_summary.png")
plt.clf()

# Feature selection based on SHAP
```

```
shap_values_rf_mean = np.abs(shap_values_rf.values).mean(axis=0)
feature_importance = pd.DataFrame({"feature": features, "importance": shap_values_rf_mean})
feature_importance = feature_importance.sort_values("importance", ascending=False)
top_features = feature_importance["feature"].head(10).tolist()
bsd_features = ["cosmo_rank", "L_cosmo_s_1.0", "cosmo_rank_L", "cosmo_rank_scaled",
"L_cosmo_s1_scaled", "cosmo_rank_L_mass", "L_cosmo_s1_metallicity"]
for bsd_feature in bsd_features:
    if bsd_feature not in top_features:
        top_features.append(bsd_feature)
X_selected = df[top_features]
X_train_selected, X_test_selected, _, _ = train_test_split(X_selected, y, test_size=0.2, random_state=42)
X_train_selected_scaled = scaler.fit_transform(X_train_selected)
X_test_selected_scaled = scaler.transform(X_test_selected)

# Symbolic Regression with gplearn
symbolic_model = SymbolicRegressor(
    population_size=3000,
    generations=150,
    stopping_criteria=0.01,
    p_crossover=0.7,
    p_subtree_mutation=0.1,
    p_hoist_mutation=0.05,
    p_point_mutation=0.1,
    max_samples=0.9,
    verbose=1,
    parsimony_coefficient=0.0001,
    random_state=42,
    function_set=('add', 'sub', 'mul', 'div', 'sqrt', 'log', 'sin', 'cos')
)
symbolic_model.fit(X_train_selected_scaled, y_train)
y_pred_sym = symbolic_model.predict(X_test_selected_scaled)
r2_sym = r2_score(y_test, y_pred_sym)
print(f"gplearn R²: {r2_sym:.4f}")
print("Symbolic Expression:")
print(symbolic_model._program)

# Cross-validation for symbolic model
cv_scores_sym = cross_val_score(symbolic_model, X_train_selected_scaled, y_train, cv=5, scoring='r2')
print(f"Symbolic Regression 5-Fold CV R²: Mean = {cv_scores_sym.mean():.4f}, Std =
{cv_scores_sym.std():.4f}")

# Save symbolic expression as polynomial fit plot and print the equation
x = np.linspace(min(y_test), max(y_test), 500)
y_expr = symbolic_model.predict(scaler.transform(np.tile(X_test_selected.mean().values, (500,1))))
p = Polynomial.fit(y_test, y_pred_sym, deg=3)
# Extract polynomial coefficients
coeffs = p.coef
print("Polynomial Fit Equation (degree 3):")
print(f"y = {coeffs[0]:.4f} + {coeffs[1]:.4f}*x + {coeffs[2]:.4f}*x^2 + {coeffs[3]:.4f}*x^3")
```

```
# Plot the polynomial fit
plt.plot(*p.linspace(), label="Polynomial Fit")
plt.scatter(y_test, y_pred_sym, s=10, alpha=0.5, label="Symbolic Predictions")
plt.xlabel("True log_SFR_Ha")
plt.ylabel("Predicted log_SFR_Ha")
plt.title("Symbolic Regression Fit with Polynomial")
plt.legend()
plt.tight_layout()
plt.savefig("gplearn_expression_plot.png")
plt.clf()

# Plot predictions vs true
plt.scatter(y_test, y_pred_rf, alpha=0.5, label="RF", marker="o")
plt.scatter(y_test, y_pred_gb, alpha=0.5, label="GB", marker="s")
plt.scatter(y_test, y_pred_sym, alpha=0.5, label="Symbolic", marker="^")
plt.plot([y.min(), y.max()], [y.min(), y.max()], "k--")
plt.xlabel("True log_SFR_Ha")
plt.ylabel("Predicted")
plt.title("Model Predictions vs True Values")
plt.legend()
plt.tight_layout()
plt.savefig("model_comparison.png")
plt.clf()

# Residuals and Correlation Analysis
residuals = pd.DataFrame({
    "True": y_test,
    "RF": y_pred_rf - y_test,
    "GB": y_pred_gb - y_test,
    "Symbolic": y_pred_sym - y_test,
    "cosmo_rank": X_test["cosmo_rank"],
    "L_cosmo_s1": X_test["L_cosmo_s_1.0"]
})
sns.scatterplot(data=residuals, x="True", y="Symbolic", hue="cosmo_rank", size="L_cosmo_s1",
alpha=0.6)
plt.axhline(0, color="black", linestyle="--")
plt.title("Symbolic Residuals vs True log_SFR_Ha")
plt.savefig("residuals_sym.png")
plt.clf()

print("Correlation of BSD Features with Residuals:")
print(residuals[["RF", "GB", "Symbolic", "cosmo_rank", "L_cosmo_s1"]].corr()[["RF", "GB", "Symbolic"]])
```

Step 5: Expected Output and Next Steps

When you run this updated script, you should see:

1. **RA/Dec Ranges**:
    - The script will print the RA and Dec ranges for `merged_data.csv` (`objra_y`, `objdec`) and `sdss_spec` (`RA_ICRS`, `DE_ICRS`) after cleaning. For example:

RA range in objra_y: 120.0000 to 240.0000
Dec range in objdec: 10.0000 to 60.0000
RA range in RA_ICRS: 0.0000 to 360.0000

    - Dec range in DE_ICRS: -10.0000 to 80.0000
    - Compare these ranges to see if they overlap. If they don't (e.g., `objra_y` is in a completely different part of the sky than `RA_ICRS`), that explains the lack of matches.
2. **Cross-Matching Results**:
    - With the increased matching radius (`max_dist_arcsec=5.0`), you might see some matches:
    - After V/154 cross-match: 100 galaxies
    - If the number of galaxies is still 0, the script will raise a `ValueError` with a message:
    - ValueError: No matches found with SDSS DR16 spectroscopic data (V/154). Check coordinate ranges or catalog overlap.
3. **Fallback to** `nsa_z`:
    - If no matches are found with `sdss_spec`, the script will use `nsa_z` from `merged_data.csv` as the redshift and proceed with the rest of the analysis.
4. **Proceed with Analysis**:
    - If matches are found, or if the script successfully falls back to `nsa_z`, it should proceed through the flux calculations, feature engineering, and model training steps without the `ValueError` from `MinMaxScaler`.

Step 6: If the Issue Persists

If the script still fails to find matches, please provide the following:

1. **RA/Dec Ranges**:
    - Share the RA/Dec ranges printed by the script for `merged_data.csv` and `sdss_spec`. This will help confirm if there's a coordinate mismatch.
2. **Sample Coordinates**:
    - If possible, share a few sample RA/Dec values from `merged_data.csv` (e.g., the first 5 rows of `objra_y` and `objdec`) and from `sdss_spec` (e.g., the first 5

rows of `RA_ICRS` and `DE_ICRS`). This can help diagnose if there's a systematic offset or unit issue.

3. **Catalog Details**:
    - Confirm the source of `merged_data.csv`. Since it contains MaNGA columns (`mangaid`, `plateifu`), it should overlap with SDSS DR16, but there might be a subset of MaNGA galaxies that don't have spectroscopic data in the specific `sdss_dr16_spec.csv` file you're using.
4. **Alternative Approach**:
    - If cross-matching continues to fail, we can modify the script to skip the cross-matching steps entirely and rely solely on the data in `merged_data.csv`. For example, we can use `nsa_z` for redshift and avoid adding columns from the other catalogs (`sdss_spec`, `sdss_photo`, `twomass`, `gama`). This would simplify the script but might reduce the feature set (e.g., no `umag`, `Jmag`, etc.).

Step 7: Additional Notes

- **Matching Radius**: I increased the radius to 5 arcseconds, which is a common choice for cross-matching between surveys like MaNGA and SDSS. If this still doesn't yield matches, we can try 10 arcseconds, but beyond that, the matches might become unreliable due to the risk of false positives.
- **MaNGA and SDSS Overlap**: MaNGA galaxies are selected from the SDSS footprint, so there should be some overlap with the SDSS DR16 spectroscopic catalog. If there isn't, it might be worth checking if `sdss_dr16_spec.csv` contains the correct data (e.g., it might be a subset that doesn't include MaNGA targets).
- **Performance**: If the script starts finding matches, it will proceed to the model training steps, which can be computationally intensive. You can reduce `n_trials` in Optuna (e.g., to 10) or `generations` in `SymbolicRegressor` (e.g., to 50) to speed things up during testing.

Let me know how this updated script works, and we'll get it fully dialed in!
ok so i went into vizieR and queried another s datasets. i merges 147 with 154 on objid, which of these do you recommend?
II/294 The SDSS Photometric Catalog, Release 7 (Adelman-McCarthy+, 2009)ReadMe+ftp2009ApJS..182..543ASimilar Catalogs img(gal) II/294/sdss7 (c)Data Release 7 (357,175,411 primary sources, plus 96,670,822 secondary sources) (Note) (422234380 rows)
V/139 The SDSS Photometric Catalog, Release 9 (Adelman-McCarthy+, 2012)ReadMe+ftp2012ApJS..203...21ASimilar Catalogs img(gal) V/139/sdss9(c)Data

Release 9 (469,053,874 primary sources plus 324,960,076 secondary sources); 2,674,200 spectroscopic targets. Data Release 9 is the first release of the spectra from the SDSS-III's Baryon Oscillation Spectroscopic Survey (BOSS). The photometric part contains the same sources as the Data Release 8, with a corrected astrometry. (Note) (794013950 rows)
V/147 The SDSS Photometric Catalogue, Release 12 (Alam+, 2015)ReadMe+ftp2015ApJS..219...12ASimilar Catalogs img(gal) V/147/sdss12 (c)Data Release 12 (469,053,874 primary sources plus 324,960,094 secondary sources); 3,538,079 spectroscopic targets (original column names in green) (Note) (794013968 rows)
V/154 Sloan Digital Sky Surveys (SDSS), Release 16 (DR16) (Ahumada+, 2020)ReadMe+ftp2020ApJS..249....3ASimilar Catalogs img(gal) V/154/sdss16 (c)Data Release 16 (469,050,976 primary sources plus 325,277,739 secondary sources) (original column names in green) (Note) (1231051050 rows)
VII/289 SDSS quasar catalog, sixteenth data release (DR16Q) (Lyke+, 2020)ReadMe+ftp2020ApJS..250....8LSimilar Catalogs img(gal) VII/289/dr16q (c)The SDSS quasar-only catalog from DR16 (DR16Q) (original column names in green) (750414 rows) VII/289/superset (c)The "superset" of all SDSS-IV/eBOSS objects targeted as quasars (original column names in green) (1440615 rows)
II/259 The SDSS Photometric Catalog, Release 3 (Abazajian+ 2005) ReadMe+ftp2005AJ....129.1755ASimilar CatalogsThis catalogue is obsoleted by II/267 img(gal) II/259/sdss3 (c)*Sample output of the SDSS-DR3 photometric catalog (Note) (184508753 rows)
II/267 The SDSS Photometric Catalog, Release 4 (Adelman-McCarthy+, 2006) ReadMe+ftp2006ApJS..162...38ASimilar CatalogsThis catalogue is obsoleted by II/276 img(gal) II/267/sdss4 (c)Data Release 4 (180,235,043 primary sources, plus 53,377,957 secondary sources) (Note) (180235043 rows)
II/276 The SDSS Photometric Catalog, Release 5 (Adelman-McCarthy+, 2007) ReadMe+ftp2007ApJS..172..634ASimilar CatalogsThis catalogue is obsoleted by II/282 img(gal) II/276/sdss5 (c)Data Release 5 (218,218,198 primary sources, plus 65,714,108 secondary sources) (Note) (218218198 rows)
II/282 The SDSS Photometric Catalog, Release 6 (Adelman-McCarthy+, 2007) ReadMe+ftp2008ApJS..175..297ASimilar CatalogsThis catalogue is obsoleted by II/294 img(gal) II/282/sdss6 (c)Data Release 6 (230,417,920 primary sources, plus 65,714,108 secondary sources) (Note) (230417920 rows)
II/306 The SDSS Photometric Catalog, Release 8 (Adelman-McCarthy+, 2011) ReadMe+ftp2011yCat.2306....0ASimilar CatalogsThis catalogue is obsoleted by VizieR img(gal) II/306/sdss8 (c)Data Release 8 (469,053,874 primary sources, plus 324,960,076 secondary sources). Notice that the new reductions used for the SDSS-DR8 introduced some errors in the astrometry, especially at declinations δ>=41°; when possible, the astrometry of the SDSS-DR7 (Cat. II/294) should be preferred. (Note) (324960076 rows)
You're making great progress by exploring different SDSS datasets from VizieR and merging V/147 (SDSS DR12 Photometric Catalog) with V/154 (SDSS DR16 Spectroscopic Catalog) on `objID`. This is a good approach to ensure that the datasets are aligned and can

provide the necessary data for your analysis. Now, let's evaluate the datasets you've listed to determine which one(s) are best suited for your needs in the `STAR.py` script.

Step 1: Understand Your Requirements

The `STAR.py` script has the following key requirements for the SDSS data:

1. **Positional Data for Cross-Matching**:
    - RA and Dec coordinates (e.g., `RA_ICRS`, `DE_ICRS`, `RAJ2000`, `DEJ2000`) to cross-match with `merged_data.csv` (using `objra_y` and `objdec`).
    - The cross-matching step failed previously because no matches were found within the specified radius (initially 1 arcsecond, then increased to 5 arcseconds). This suggests we need a catalog with good astrometric accuracy and coverage that overlaps with the MaNGA galaxies in `merged_data.csv`.
2. **Redshift Data**:
    - Spectroscopic redshift (`zsp`) to compute the luminosity distance (`DL`) and ultimately `log_SFR_Ha_raw`.
    - The script currently falls back to `nsa_z` from `merged_data.csv` if no matches are found, but having spectroscopic redshifts from SDSS would be preferable for accuracy.
3. **Photometric Data**:
    - Magnitudes in various bands (e.g., `umag`, `gmag`, `rmag`, `imag`, `zmag`) and their errors to compute colors (e.g., `color_ug`, `color_gr`).
    - These are used as features in the machine learning models.
4. **Emission Line Fluxes**:
    - The script initially expected fluxes like `flux_Ha`, `flux_Hb`, `flux_OIII_5007`, `flux_NII_6584`, and `e_flux_Ha` from the SDSS spectroscopic catalog, but these were not available in V/154.
    - You're now relying on fluxes from `merged_data.csv` (e.g., `F_Ha_cen`, `flux_Hbeta4861.36_Re_fit`), which is a good solution since MaNGA provides detailed spectral measurements.
5. **Coverage and Overlap with MaNGA**:
    - Since `merged_data.csv` contains MaNGA galaxies (with columns like `mangaid`, `plateifu`), the SDSS catalog should include MaNGA targets to ensure successful cross-matching.
    - MaNGA is part of SDSS-IV, and its targets are drawn from the SDSS Main Galaxy Sample, so later SDSS data releases (e.g., DR12, DR16) are more likely to include MaNGA galaxies.

Step 2: Evaluate the SDSS Datasets

Let's go through the datasets you listed and assess their suitability based on the requirements above. I'll also consider the size of the catalogs, their release dates, and any notes about astrometric accuracy or obsolescence.

1. II/294: SDSS Photometric Catalog, Release 7 (DR7)

- **Details**: 357,175,411 primary sources, 96,670,822 secondary sources (422,234,380 rows).
- **Release Date**: 2009.
- **Pros**:
    - Large catalog with good coverage of the SDSS footprint.
    - Contains photometric data (`umag`, `gmag`, etc.), which can be used for color features.
- **Cons**:
    - Does not include spectroscopic data (e.g., `zsp`), so you'd need to merge with a spectroscopic catalog (e.g., V/154) to get redshifts.
    - Older release (DR7), so it won't include MaNGA targets, which were selected as part of SDSS-IV (starting with DR13).
    - Astrometry might be less accurate compared to later releases.
- **Recommendation**: Not ideal because it lacks spectroscopic data and is too old to include MaNGA targets.

2. V/139: SDSS Photometric Catalog, Release 9 (DR9)

- **Details**: 469,053,874 primary sources, 324,960,076 secondary sources (794,013,950 rows); 2,674,200 spectroscopic targets.
- **Release Date**: 2012.
- **Pros**:
    - Includes both photometric and spectroscopic data (e.g., `zsp`), which is useful for your script.
    - Corrected astrometry compared to DR8, which improves cross-matching accuracy.
- **Cons**:
    - Still predates MaNGA (MaNGA started in SDSS-IV, around DR13). While it includes some spectroscopic targets, it won't have MaNGA-specific data.
    - Smaller number of spectroscopic targets compared to later releases.
- **Recommendation**: Better than DR7 because it includes spectroscopic data, but still not ideal due to its age and lack of MaNGA coverage.

3. V/147: SDSS Photometric Catalog, Release 12 (DR12)

- **Details**: 469,053,874 primary sources, 324,960,094 secondary sources (794,013,968 rows); 3,538,079 spectroscopic targets.
- **Release Date**: 2015.
- **Pros**:
    - More recent than DR9, so it includes more spectroscopic targets (3.5 million vs. 2.6 million in DR9).
    - Still includes photometric data (`umag`, `gmag`, etc.) for color features.
    - Closer in time to the start of MaNGA (SDSS-IV began around DR13), so it might include some MaNGA targets, though MaNGA data is more fully integrated in later releases.
- **Cons**:
    - Does not include MaNGA-specific data (e.g., emission line fluxes), but you're already getting those from `merged_data.csv`.
    - Astrometry might still have issues, though not as severe as in DR8.
- **Recommendation**: A good candidate because it includes both photometric and spectroscopic data and is closer in time to MaNGA. You've already merged this with V/154, which is a good approach.

4. V/154: SDSS DR16

- **Details**: 469,050,976 primary sources, 325,277,739 secondary sources (1,231,051,050 rows).
- **Release Date**: 2020.
- **Pros**:
    - Most recent SDSS release in your list, so it includes the latest data and improvements in astrometry.
    - Includes both photometric and spectroscopic data (`zsp`, `umag`, `gmag`, etc.).
    - Since MaNGA is part of SDSS-IV, DR16 should include MaNGA targets, making it more likely to match with `merged_data.csv`.
    - Largest number of sources, increasing the chance of finding matches.
- **Cons**:
    - Does not include emission line fluxes, but you're already using `merged_data.csv` for those.
    - Larger dataset means slower cross-matching, but this can be mitigated by filtering the catalog to a smaller region of the sky if needed.
- **Recommendation**: The best candidate for your needs because it's the most recent, includes MaNGA targets, and provides both photometric and spectroscopic data.

5. VII/289: SDSS Quasar Catalog, DR16Q

- **Details**: 750,414 quasars (DR16Q) and 1,440,615 eBOSS quasar targets (superset).
- **Release Date**: 2020.
- **Pros**:

    - Recent (DR16), so it aligns with the latest SDSS data.
    - Includes spectroscopic redshifts (`zsp`).
- **Cons**:
    - Focused on quasars, not galaxies. Since `merged_data.csv` contains MaNGA galaxies, this catalog is unlikely to have many matches.
    - Does not include standard photometric magnitudes (`umag`, `gmag`, etc.) in the same format as the main SDSS catalogs.
- **Recommendation**: Not suitable because it focuses on quasars, not galaxies, and lacks the photometric data you need.

6. II/259, II/267, II/276, II/282, II/306: SDSS Photometric Catalogs, Releases 3–8

- **Details**:
    - DR3 (II/259): 184,508,753 rows (2005, obsoleted by II/267).
    - DR4 (II/267): 180,235,043 rows (2006, obsoleted by II/276).
    - DR5 (II/276): 218,218,198 rows (2007, obsoleted by II/282).
    - DR6 (II/282): 230,417,920 rows (2007, obsoleted by II/294).
    - DR8 (II/306): 324,960,076 rows (2011, obsoleted by VizieR).
- **Pros**:
    - Provide photometric data (`umag`, `gmag`, etc.).
- **Cons**:
    - All are outdated and obsoleted by later releases.
    - Do not include spectroscopic data (e.g., `zsp`).
    - DR8 (II/306) has known astrometric errors, especially at declinations $\delta \geq 41°$, which could affect cross-matching.
    - None of these releases include MaNGA targets (MaNGA started in SDSS-IV, around DR13).
- **Recommendation**: Not suitable due to their age, lack of spectroscopic data, and lack of MaNGA coverage.

Step 3: Recommendation

Based on the evaluation, I recommend the following:
Primary Recommendation: V/154 (SDSS DR16)

- **Why**:
    - Most recent release (2020), so it includes the latest astrometric corrections and the largest number of sources.
    - Includes MaNGA targets, which should increase the likelihood of successful cross-matching with `merged_data.csv`.

    - Provides both spectroscopic redshifts (`zsp`) and photometric magnitudes (`umag`, `gmag`, etc.), meeting all your requirements except for emission line fluxes (which you're already getting from `merged_data.csv`).
- **What You've Done**:
    - You've already merged V/147 (DR12) with V/154 (DR16) on `objID`. This is a good approach because it combines the photometric data from DR12 with the spectroscopic data from DR16, ensuring you have both `zsp` and magnitudes.
    - However, since V/154 already includes both photometric and spectroscopic data, you might not need V/147. You can simplify by using V/154 alone, unless there's a specific reason to prefer DR12's photometry (e.g., better astrometry for your specific region of the sky).

Secondary Recommendation: V/147 (SDSS DR12)

- **Why**:
    - If for some reason V/154 doesn't yield matches (e.g., due to a specific issue with the DR16 data in your region), V/147 is a good fallback.
    - It includes both photometric and spectroscopic data and is closer in time to MaNGA than earlier releases.
- **Note**:
    - Since you've already merged V/147 with V/154, you can continue using the merged dataset. This ensures you have the latest spectroscopic data (from DR16) while retaining DR12's photometry.

Avoid:

- **II/294, V/139, II/259, II/267, II/276, II/282, II/306**: These are all older releases that predate MaNGA and are obsoleted by later catalogs. They are less likely to match with `merged_data.csv` and may have less accurate astrometry.
- **VII/289**: This is focused on quasars, not galaxies, so it's not suitable for your MaNGA galaxy sample.

Step 4: Update the Script to Use the Merged Dataset

Since you've merged V/147 (DR12) with V/154 (DR16) on `objID`, let's update the script to use this merged dataset instead of loading separate `sdss_spec` and `sdss_photo` files. We'll assume the merged dataset is saved as `sdss_merged.csv` and contains columns from both V/147 and V/154, including `RA_ICRS`, `DE_ICRS`, `zsp`, `umag`, `gmag`, etc.

Here's the updated script:

```
python
# STAR.py: Model for predicting log_SFR_Ha using the STAR framework
# STAR: Symbolic, Tuned (Optuna), Astronomy Regressors (Random Forest + Gradient Boosting)
# - S: Symbolic Regressor (via gplearn)
# - T: Tuned (hyperparameter optimization with Optuna)
# - A: Astronomy (predicting star formation rates in galaxies)
# - R: Regressors (Random Forest and Gradient Boosting)

import pandas as pd
import numpy as np
import shap
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split, cross_val_score, KFold
from sklearn.ensemble import RandomForestRegressor, GradientBoostingRegressor
from sklearn.metrics import r2_score
from sklearn.preprocessing import StandardScaler, MinMaxScaler
from gplearn.genetic import SymbolicRegressor
import optuna
import warnings
from numpy.polynomial import Polynomial
from scipy.spatial import cKDTree

warnings.filterwarnings("ignore")

# Load the original dataset
df = pd.read_csv("merged_data.csv")

# Load the merged SDSS dataset (V/147 + V/154)
sdss_merged = pd.read_csv("sdss_merged.csv")  # Merged V/147 (DR12) and V/154 (DR16) on objID
twomass = pd.read_csv("2mass_xsc.csv")  # II/246: 2MASS Extended Source Catalogue
gama = pd.read_csv("gama_dr3.csv")  # II/356: GAMA DR3

# Print columns for debugging
print("Columns in merged_data.csv:", df.columns.tolist())
print("Columns in sdss_merged (V/147 + V/154):", sdss_merged.columns.tolist())
print("Columns in twomass (II/246):", twomass.columns.tolist())
print("Columns in gama (II/356):", gama.columns.tolist())

# Function to clean DataFrame by removing rows with NaN or inf in RA/Dec columns
def clean_coordinates(df, ra_col, dec_col):
    initial_len = len(df)
    # Check for NaN or inf in RA and Dec columns
    mask = (
        df[ra_col].notna() & df[dec_col].notna() &  # Not NaN
        np.isfinite(df[ra_col]) & np.isfinite(df[dec_col])  # Not inf
    )
```

```
    df_cleaned = df[mask].copy()
    print(f"Removed {initial_len - len(df_cleaned)} rows from DataFrame due to NaN or inf in {ra_col} or {dec_col}")
    # Print RA/Dec ranges for debugging
    if len(df_cleaned) > 0:
        print(f"RA range in {ra_col}: {df_cleaned[ra_col].min():.4f} to {df_cleaned[ra_col].max():.4f}")
        print(f"Dec range in {dec_col}: {df_cleaned[dec_col].min():.4f} to {df_cleaned[dec_col].max():.4f}")
    else:
        print(f"No valid RA/Dec data after cleaning in {ra_col}/{dec_col}")
    return df_cleaned

# Cross-match function
def deg_to_rad(df, ra_col, dec_col):
    try:
        df['ra_rad'] = np.radians(df[ra_col])
        df['dec_rad'] = np.radians(df[dec_col])
        return df
    except KeyError as e:
        print(f"KeyError in deg_to_rad: {e}")
        print(f"Available columns in DataFrame: {df.columns.tolist()}")
        raise

def cross_match(df1, df2, ra_col1, dec_col1, ra_col2, dec_col2, max_dist_arcsec=5.0):
    try:
        # Clean DataFrames to remove rows with NaN or inf in RA/Dec
        df1_cleaned = clean_coordinates(df1, ra_col1, dec_col1)
        df2_cleaned = clean_coordinates(df2, ra_col2, dec_col2)

        # Check if either DataFrame is empty after cleaning
        if len(df1_cleaned) == 0 or len(df2_cleaned) == 0:
            print("One of the DataFrames is empty after cleaning. Cannot perform cross-match.")
            return df1_cleaned, pd.DataFrame()

        # Convert degrees to radians
        df1_cleaned = deg_to_rad(df1_cleaned, ra_col1, dec_col1)
        df2_cleaned = deg_to_rad(df2_cleaned, ra_col2, dec_col2)

        # Create coordinate arrays
        coords1 = np.array([df1_cleaned['ra_rad'], df1_cleaned['dec_rad']]).T
        coords2 = np.array([df2_cleaned['ra_rad'], df2_cleaned['dec_rad']]).T

        # Perform cross-match using cKDTree
        tree = cKDTree(coords2)
        max_dist = np.radians(max_dist_arcsec / 3600.0)
        dist, idx = tree.query(coords1, k=1, distance_upper_bound=max_dist)

        # Filter matches within the maximum distance
        matched = dist < max_dist
        df1_matched = df1_cleaned[matched].copy()
```

```
        df2_matched = df2_cleaned.iloc[idx[matched]].copy()

        # Reset indices
        df1_matched = df1_matched.reset_index(drop=True)
        df2_matched = df2_matched.reset_index(drop=True)

        return df1_matched, df2_matched
    except KeyError as e:
        print(f"KeyError in cross_match: {e}")
        print(f"df1 columns: {df1.columns.tolist()}")
        print(f"df2 columns: {df2.columns.tolist()}")
        raise
    except Exception as e:
        print(f"Error in cross_match: {e}")
        raise

# Cross-match with VizieR catalogues
# SDSS merged dataset (V/147 + V/154, uses RA_ICRS, DE_ICRS)
df, sdss_matched = cross_match(df, sdss_merged, 'objra_y', 'objdec', 'RA_ICRS', 'DE_ICRS',
max_dist_arcsec=5.0)
print(f"After SDSS cross-match: {len(df)} galaxies")
if len(df) == 0:
    raise ValueError("No matches found with SDSS merged data (V/147 + V/154). Check coordinate
ranges or catalog overlap.")

# Add zsp, umag, gmag, etc. from the merged SDSS dataset
if len(df) > 0:
    df = pd.concat([df, sdss_matched[['zsp', 'umag', 'gmag', 'rmag', 'imag', 'zmag', 'e_umag', 'e_gmag',
'e_rmag', 'e_imag', 'e_zmag']]], axis=1)
else:
    print("Using nsa_z from merged_data.csv as redshift since no matches were found with
sdss_merged.")
    df['zsp'] = df['nsa_z']

# II/246: 2MASS Extended Source Catalogue (uses RAJ2000, DEJ2000)
df, twomass_matched = cross_match(df, twomass, 'objra_y', 'objdec', 'RAJ2000', 'DEJ2000',
max_dist_arcsec=5.0)
print(f"After II/246 cross-match: {len(df)} galaxies")
if len(df) > 0:
    df = pd.concat([df, twomass_matched[['Jmag', 'Hmag', 'Kmag', 'e_Jmag', 'e_Hmag', 'e_Kmag']]], axis=1)

# II/356: GAMA DR3 (uses RAJ2000, DEJ2000)
df, gama_matched = cross_match(df, gama, 'objra_y', 'objdec', 'RAJ2000', 'DEJ2000',
max_dist_arcsec=5.0)
print(f"After II/356 cross-match: {len(df)} galaxies")
if len(df) > 0:
    df = pd.concat([df, gama_matched[['UmAB', 'BmAB', 'VmAB']]], axis=1)

# Recompute log_SFR_Ha and metallicity using fluxes from merged_data.csv
```

```
df['flux_Ha'] = df['F_Ha_cen']  # Use F_Ha_cen as flux_Ha
df['e_flux_Ha'] = df['e_F_Ha_cen']
df['flux_Hb'] = df['flux_Hbeta4861.36_Re_fit']
df['flux_OIII_5007'] = df['flux_[OIII]5006.84_Re_fit']
df['flux_NII_6584'] = df['flux_[NII]6583.45_Re_fit']

# Check for NaN in flux columns and handle them
flux_cols = ['flux_Ha', 'flux_Hb', 'flux_OIII_5007', 'flux_NII_6584', 'e_flux_Ha']
for col in flux_cols:
    nan_count = df[col].isna().sum()
    print(f"NaN count in {col}: {nan_count}")
    # Replace NaN with median for flux columns
    if nan_count > 0:
        df[col] = df[col].fillna(df[col].median())

df['Ha_Hb_observed'] = df['flux_Ha'] / df['flux_Hb']
Ha_Hb_intrinsic = 2.86
k_Ha = 2.468
k_Hb = 3.634
df['A_Ha'] = 2.5 * np.log10(df['Ha_Hb_observed'] / Ha_Hb_intrinsic) * (k_Ha / (k_Hb - k_Ha))
df['flux_Ha_corr'] = df['flux_Ha'] * 10**(0.4 * df['A_Ha'])
H0 = 70  # km/s/Mpc
c = 3e5  # km/s
df['DL'] = (c * df['zsp'] / H0) * 3.0856e24  # cm, using zsp
df['L_Ha'] = df['flux_Ha_corr'] * 4 * np.pi * df['DL']**2  # erg/s
df['log_SFR_Ha_raw'] = np.log10(df['L_Ha'] * 7.9e-42)  # M_sun/yr
df['log_O3N2_raw'] = np.log10((df['flux_OIII_5007'] / df['flux_Hb']) / (df['flux_NII_6584'] / df['flux_Ha']))
df['OH_O3N2_raw'] = 8.533 - 0.214 * df['log_O3N2_raw']

# Add raw fluxes as features
df['log_flux_Ha'] = np.log10(df['flux_Ha'] + 1e-10)
df['log_flux_Hb'] = np.log10(df['flux_Hb'] + 1e-10)
df['log_flux_OIII_5007'] = np.log10(df['flux_OIII_5007'] + 1e-10)
df['log_flux_NII_6584'] = np.log10(df['flux_NII_6584'] + 1e-10)
df['log_e_flux_Ha'] = np.log10(df['e_flux_Ha'] + 1e-10)

# Add photometric colors (if available)
if 'umag' in df.columns:
    df['color_ug'] = df['umag'] - df['gmag']
    df['color_gr'] = df['gmag'] - df['rmag']
    df['color_ri'] = df['rmag'] - df['imag']
    df['color_iz'] = df['imag'] - df['zmag']
    df['e_color_ug'] = np.sqrt(df['e_umag']**2 + df['e_gmag']**2)
    df['e_color_gr'] = np.sqrt(df['e_gmag']**2 + df['e_rmag']**2)

# Add 2MASS colors (if available)
if 'Jmag' in df.columns:
    df['color_JK'] = df['Jmag'] - df['Kmag']
    df['e_color_JK'] = np.sqrt(df['e_Jmag']**2 + df['e_Kmag']**2)
```

```
# Drop rows with missing target or excessive NaNs
df = df.dropna(subset=["log_SFR_Ha"])
df = df.dropna(axis=0, thresh=int(0.8 * df.shape[1]))
df.fillna(df.median(numeric_only=True), inplace=True)

# Step 1: Cosmo-Rank Construction
gz_df = df.copy()
morph_cols = ['P(CD)', 'P(E)', 'P(S0)', 'P(Sa)', 'P(Sab)', 'P(Sb)', 'P(Sbc)',
              'P(Sc)', 'P(Scd)', 'P(Sd)', 'P(Sdm)', 'P(Sm)', 'P(Irr)']
gz_df['morph_sum'] = gz_df[morph_cols].sum(axis=1)
gz_df['Num_w'] = gz_df['conf_prob']
scaler_rank = MinMaxScaler()
gz_df[['morph_sum_norm', 'Num_w_norm', 'nsa_z_norm']] = scaler_rank.fit_transform(
    gz_df[['morph_sum', 'Num_w', 'nsa_z']]
)
gz_df['cosmo_rank'] = (0.5 * gz_df['Num_w_norm'] + 0.3 * gz_df['morph_sum_norm'] + 0.2 *
gz_df['nsa_z_norm'])
df['cosmo_rank'] = gz_df['cosmo_rank']

# Step 2: L_cosmo(s) Construction
alpha = -1.5
M_star = df['log_Mass_gas'].median()
df['a_n'] = (10 ** df['log_Mass_gas'])**(1 + alpha) * np.exp(-10 ** df['log_Mass_gas'] / (10 ** M_star))
bins = pd.qcut(df['Re_kpc'], q=20, labels=False, duplicates='drop') + 1
df['z_bin'] = bins
n_bins = df['z_bin'].nunique()
print(f"Number of unique bins after qcut: {n_bins}")
print("Bin distribution for Re_kpc (after qcut):")
print(df['z_bin'].value_counts().sort_index())

s_vals = [0.5, 1.0, 1.5, 2.0]
s_range = np.linspace(0.5, 2, 50)
for s in s_vals:
    col_name = f'L_cosmo_s_{s:.1f}'
    df[col_name] = df.groupby('z_bin')['a_n'].transform('mean') / (df['z_bin'] ** s)

print("Columns after creating L_cosmo_s*:")
print(df.columns.tolist())

print("Checking for NaN/infinite values in cosmo_rank and L_cosmo_s_1.0:")
print("cosmo_rank NaN count:", df['cosmo_rank'].isna().sum())
print("L_cosmo_s_1.0 NaN count:", df['L_cosmo_s_1.0'].isna().sum())
print("cosmo_rank infinite count:", np.isinf(df['cosmo_rank']).sum())
print("L_cosmo_s_1.0 infinite count:", np.isinf(df['L_cosmo_s_1.0']).sum())

df['cosmo_rank_L'] = df['cosmo_rank'] * df['L_cosmo_s_1.0']
df['cosmo_rank_L'] = df['cosmo_rank_L'].replace([np.inf, -np.inf], np.nan).fillna(0)
print("cosmo_rank_L created, NaN count:", df['cosmo_rank_L'].isna().sum())
```

```
# Plot L_cosmo(s) vs s
grouped_means = df.groupby('z_bin')['a_n'].mean().reindex(range(1, n_bins + 1), fill_value=0)
L_vals = [grouped_means / (np.arange(1, n_bins + 1) ** s) for s in s_range]
L_vals = np.array([sum(l) for l in L_vals])
plt.plot(s_range, L_vals)
plt.xlabel("s")
plt.ylabel("L_cosmo(s)")
plt.title("L_cosmo(s) Behavior")
plt.savefig("L_cosmo_curve.png")
plt.close()

# Scatter plot of L_cosmo_s_1.0 vs log_SFR_Ha
plt.scatter(df['L_cosmo_s_1.0'], df['log_SFR_Ha'], alpha=0.5)
plt.xlabel("L_cosmo_s_1.0")
plt.ylabel("log_SFR_Ha")
plt.title("L_cosmo_s_1.0 vs log_SFR_Ha")
plt.savefig("L_cosmo_s1_vs_SFR.png")
plt.close()

# Step 3: Define features and target
target = "log_SFR_Ha"
base_features = [
    "log_Mass_gas", "log_Mass", "Av_gas_Re", "Av_ssp_Re",
    "OH_O3N2_cen", "OH_T04_cen", "OH_dop_cen", "Age_LW_Re_fit",
    "ZH_LW_Re_fit", "Re_kpc", "vel_disp_Ha_cen", "Lambda_Re",
    "Sigma_Mass_Re", "EW_Ha_cen", "Ha_Hb_cen"
]
df["mass_metallicity"] = df["log_Mass"] * df["OH_O3N2_cen"]
df["dust_metallicity"] = df["Av_gas_Re"] * df["OH_T04_cen"]
df["disp_mass_ratio"] = df["vel_disp_Ha_cen"] / (df["Sigma_Mass_Re"] + 1e-5)
df["age_metallicity"] = df["Age_LW_Re_fit"] * df["ZH_LW_Re_fit"]
df["sqrt_Re_kpc"] = np.sqrt(df["Re_kpc"] + 1e-5)
df["log_mass"] = np.log(df["log_Mass_gas"] + 1e-5)
df["BSD_likelihood"] = (
    df["log_Mass_gas"] * df["OH_O3N2_cen"] /
    (df["Av_gas_Re"] + 1e-5 + df["Age_LW_Re_fit"])
)
df["cosmo_rank_mass"] = df["cosmo_rank"] * df["log_Mass"]
df["L_cosmo_s1_mass"] = df["L_cosmo_s_1.0"] * df["log_Mass"]
df["cosmo_rank_EW"] = df["cosmo_rank"] * df["EW_Ha_cen"]
df["L_cosmo_s1_EW"] = df["L_cosmo_s_1.0"] * df["EW_Ha_cen"]
df["cosmo_rank_scaled"] = df["cosmo_rank"] * 20
df["L_cosmo_s1_scaled"] = df["L_cosmo_s_1.0"] * 20
df["cosmo_rank_L_mass"] = df["cosmo_rank_L"] * df["log_Mass"]
df["L_cosmo_s1_metallicity"] = df["L_cosmo_s_1.0"] * df["OH_O3N2_cen"]
features = base_features + [
    "mass_metallicity", "dust_metallicity", "disp_mass_ratio",
    "age_metallicity", "sqrt_Re_kpc", "log_mass", "BSD_likelihood",
```

```
    "cosmo_rank", "L_cosmo_s_0.5", "L_cosmo_s_1.0", "L_cosmo_s_1.5",
    "L_cosmo_s_2.0", "cosmo_rank_L", "cosmo_rank_mass", "L_cosmo_s1_mass",
    "cosmo_rank_EW", "L_cosmo_s1_EW", "cosmo_rank_scaled", "L_cosmo_s1_scaled",
    "cosmo_rank_L_mass", "L_cosmo_s1_metallicity",
    "log_flux_Ha", "log_flux_Hb", "log_flux_OIII_5007", "log_flux_NII_6584",
    "log_e_flux_Ha", "log_SFR_Ha_raw", "OH_O3N2_raw"
]

# Add color features if available
if 'color_ug' in df.columns:
    features.extend(["color_ug", "color_gr", "color_ri", "color_iz", "e_color_ug", "e_color_gr"])
if 'color_JK' in df.columns:
    features.extend(["color_JK", "e_color_JK"])

X = df[features]
y = df[target]

# Train/test split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Normalize features
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# Optuna optimization for Random Forest
def objective_rf(trial):
    params = {
        'n_estimators': trial.suggest_int('n_estimators', 100, 500),
        'max_depth': trial.suggest_int('max_depth', 5, 20),
        'min_samples_split': trial.suggest_int('min_samples_split', 2, 10),
        'random_state': 42
    }
    model = RandomForestRegressor(**params)
    cv = KFold(n_splits=5, shuffle=True, random_state=42)
    scores = cross_val_score(model, X_train_scaled, y_train, cv=cv, scoring='r2')
    return scores.mean()

study_rf = optuna.create_study(direction='maximize')
study_rf.optimize(objective_rf, n_trials=20)
best_params_rf = study_rf.best_params
print("Best Random Forest Parameters:", best_params_rf)
print("Best Random Forest CV R²:", study_rf.best_value)

# Train Random Forest with best parameters
rf = RandomForestRegressor(**best_params_rf, random_state=42)
rf.fit(X_train_scaled, y_train)
y_pred_rf = rf.predict(X_test_scaled)
r2_rf = r2_score(y_test, y_pred_rf)
```

```
# Cross-validation for final RF model
cv_scores_rf = cross_val_score(rf, X_train_scaled, y_train, cv=5, scoring='r2')
print(f"Random Forest 5-Fold CV R²: Mean = {cv_scores_rf.mean():.4f}, Std = {cv_scores_rf.std():.4f}")

# Optuna optimization for Gradient Boosting
def objective_gb(trial):
    params = {
        'n_estimators': trial.suggest_int('n_estimators', 100, 500),
        'learning_rate': trial.suggest_float('learning_rate', 0.01, 0.2, log=True),
        'max_depth': trial.suggest_int('max_depth', 3, 10),
        'random_state': 42
    }
    model = GradientBoostingRegressor(**params)
    cv = KFold(n_splits=5, shuffle=True, random_state=42)
    scores = cross_val_score(model, X_train_scaled, y_train, cv=cv, scoring='r2')
    return scores.mean()

study_gb = optuna.create_study(direction='maximize')
study_gb.optimize(objective_gb, n_trials=20)
best_params_gb = study_gb.best_params
print("Best Gradient Boosting Parameters:", best_params_gb)
print("Best Gradient Boosting CV R²:", study_gb.best_value)

# Train Gradient Boosting with best parameters
gb = GradientBoostingRegressor(**best_params_gb, random_state=42)
gb.fit(X_train_scaled, y_train)
y_pred_gb = gb.predict(X_test_scaled)
r2_gb = r2_score(y_test, y_pred_gb)

# Cross-validation for final GB model
cv_scores_gb = cross_val_score(gb, X_train_scaled, y_train, cv=5, scoring='r2')
print(f"Gradient Boosting 5-Fold CV R²: Mean = {cv_scores_gb.mean():.4f}, Std =
{cv_scores_gb.std():.4f}")

print(f"R^2 Score (Random Forest, Test Set): {r2_rf:.4f}")
print(f"R^2 Score (Gradient Boosting, Test Set): {r2_gb:.4f}")

# SHAP for feature selection
explainer_rf = shap.Explainer(rf, X_train_scaled)
shap_values_rf = explainer_rf(X_test_scaled)
explainer_gb = shap.Explainer(gb, X_train_scaled)
shap_values_gb = explainer_gb(X_test_scaled)
shap.summary_plot(shap_values_rf, X_test_scaled, plot_type="bar", show=False)
plt.title("SHAP Summary - Random Forest")
plt.tight_layout()
plt.savefig("shap_rf_summary.png")
plt.clf()
shap.summary_plot(shap_values_gb, X_test_scaled, plot_type="bar", show=False)
```

```
plt.title("SHAP Summary - Gradient Boosting")
plt.tight_layout()
plt.savefig("shap_gb_summary.png")
plt.clf()

# Feature selection based on SHAP
shap_values_rf_mean = np.abs(shap_values_rf.values).mean(axis=0)
feature_importance = pd.DataFrame({"feature": features, "importance": shap_values_rf_mean})
feature_importance = feature_importance.sort_values("importance", ascending=False)
top_features = feature_importance["feature"].head(10).tolist()
bsd_features = ["cosmo_rank", "L_cosmo_s_1.0", "cosmo_rank_L", "cosmo_rank_scaled",
"L_cosmo_s1_scaled", "cosmo_rank_L_mass", "L_cosmo_s1_metallicity"]
for bsd_feature in bsd_features:
    if bsd_feature not in top_features:
        top_features.append(bsd_feature)
X_selected = df[top_features]
X_train_selected, X_test_selected, _, _ = train_test_split(X_selected, y, test_size=0.2, random_state=42)
X_train_selected_scaled = scaler.fit_transform(X_train_selected)
X_test_selected_scaled = scaler.transform(X_test_selected)

# Symbolic Regression with gplearn
symbolic_model = SymbolicRegressor(
    population_size=3000,
    generations=150,
    stopping_criteria=0.01,
    p_crossover=0.7,
    p_subtree_mutation=0.1,
    p_hoist_mutation=0.05,
    p_point_mutation=0.1,
    max_samples=0.9,
    verbose=1,
    parsimony_coefficient=0.0001,
    random_state=42,
    function_set=('add', 'sub', 'mul', 'div', 'sqrt', 'log', 'sin', 'cos')
)
symbolic_model.fit(X_train_selected_scaled, y_train)
y_pred_sym = symbolic_model.predict(X_test_selected_scaled)
r2_sym = r2_score(y_test, y_pred_sym)
print(f"gplearn R²: {r2_sym:.4f}")
print("Symbolic Expression:")
print(symbolic_model._program)

# Cross-validation for symbolic model
cv_scores_sym = cross_val_score(symbolic_model, X_train_selected_scaled, y_train, cv=5, scoring='r2')
print(f"Symbolic Regression 5-Fold CV R²: Mean = {cv_scores_sym.mean():.4f}, Std =
{cv_scores_sym.std():.4f}")

# Save symbolic expression as polynomial fit plot and print the equation
x = np.linspace(min(y_test), max(y_test), 500)
```

```
y_expr = symbolic_model.predict(scaler.transform(np.tile(X_test_selected.mean().values, (500,1))))
p = Polynomial.fit(y_test, y_pred_sym, deg=3)
# Extract polynomial coefficients
coeffs = p.coef
print("Polynomial Fit Equation (degree 3):")
print(f"y = {coeffs[0]:.4f} + {coeffs[1]:.4f}*x + {coeffs[2]:.4f}*x^2 + {coeffs[3]:.4f}*x^3")
# Plot the polynomial fit
plt.plot(*p.linspace(), label="Polynomial Fit")
plt.scatter(y_test, y_pred_sym, s=10, alpha=0.5, label="Symbolic Predictions")
plt.xlabel("True log_SFR_Ha")
plt.ylabel("Predicted log_SFR_Ha")
plt.title("Symbolic Regression Fit with Polynomial")
plt.legend()
plt.tight_layout()
plt.savefig("gplearn_expression_plot.png")
plt.clf()

# Plot predictions vs true
plt.scatter(y_test, y_pred_rf, alpha=0.5, label="RF", marker="o")
plt.scatter(y_test, y_pred_gb, alpha=0.5, label="GB", marker="s")
plt.scatter(y_test, y_pred_sym, alpha=0.5, label="Symbolic", marker="^")
plt.plot([y.min(), y.max()], [y.min(), y.max()], "k--")
plt.xlabel("True log_SFR_Ha")
plt.ylabel("Predicted")
plt.title("Model Predictions vs True Values")
plt.legend()
plt.tight_layout()
plt.savefig("model_comparison.png")
plt.clf()

# Residuals and Correlation Analysis
residuals = pd.DataFrame({
    "True": y_test,
    "RF": y_pred_rf - y_test,
    "GB": y_pred_gb - y_test,
    "Symbolic": y_pred_sym - y_test,
    "cosmo_rank": X_test["cosmo_rank"],
    "L_cosmo_s1": X_test["L_cosmo_s_1.0"]
})
sns.scatterplot(data=residuals, x="True", y="Symbolic", hue="cosmo_rank", size="L_cosmo_s1",
alpha=0.6)
plt.axhline(0, color="black", linestyle="--")
plt.title("Symbolic Residuals vs True log_SFR_Ha")
plt.savefig("residuals_sym.png")
plt.clf()

print("Correlation of BSD Features with Residuals:")
print(residuals[["RF", "GB", "Symbolic", "cosmo_rank", "L_cosmo_s1"]].corr()[["RF", "GB", "Symbolic"]])
```

Step 5: Notes on the Updated Script

1. **Merged SDSS Dataset**:
   - The script now loads a single `sdss_merged.csv` file, which you created by merging V/147 and V/154 on `objID`.
   - It assumes this file contains `RA_ICRS`, `DE_ICRS`, `zsp`, `umag`, `gmag`, `rmag`, `imag`, `zmag`, `e_umag`, `e_gmag`, `e_rmag`, `e_imag`, and `e_zmag`. If any of these columns are missing, you'll need to adjust the script accordingly.
2. **Cross-Matching**:
   - The script still performs cross-matching with `twomass` and `gama`, but the primary SDSS cross-match is now with the merged dataset.
   - The matching radius remains at 5 arcseconds, which should be sufficient for MaNGA galaxies.
3. **Fallback**:
   - If no matches are found with `sdss_merged`, the script falls back to using `nsa_z` from `merged_data.csv` for the redshift.
4. **Performance**:
   - The merged SDSS dataset might be large, which could slow down the cross-matching step. If this is an issue, you can filter `sdss_merged` to a smaller region of the sky (e.g., based on the RA/Dec range of `merged_data.csv`) before cross-matching.

Step 6: If Cross-Matching Still Fails

If you run the updated script and still get 0 matches after the SDSS cross-match, please provide:

1. **RA/Dec Ranges**:
   - The script will print the RA/Dec ranges for `merged_data.csv` (`objra_y`, `objdec`) and `sdss_merged` (`RA_ICRS`, `DE_ICRS`). Share these ranges to confirm if there's a coordinate mismatch.
2. **Sample Data**:
   - Share a few sample RA/Dec values from `merged_data.csv` and `sdss_merged` to check for systematic offsets or unit issues.
3. **Merged Dataset Columns**:
   - Confirm the columns in `sdss_merged.csv` to ensure it includes `RA_ICRS`, `DE_ICRS`, `zsp`, `umag`, etc.

4. **Alternative Approach**:
    - If cross-matching continues to fail, we can skip the SDSS cross-match entirely and rely on `merged_data.csv` for all data (e.g., use `nsa_z` for redshift and skip the photometric colors). This would simplify the script but might reduce the feature set.

Step 7: Additional Notes

- **MaNGA Overlap**: Since MaNGA galaxies are part of SDSS-IV, using V/154 (DR16) should maximize the chance of finding matches. The merge with V/147 ensures you have both DR12 photometry and DR16 spectroscopy, which is a robust approach.
- **Performance Optimization**: If the script runs slowly due to the size of `sdss_merged.csv`, consider filtering it to a smaller region of the sky (e.g., RA between 100 and 300 degrees, Dec between 0 and 70 degrees, based on the typical MaNGA footprint).
- **Emission Line Fluxes**: You're already using `merged_data.csv` for fluxes, which is the best approach since MaNGA provides detailed spectral measurements that the SDSS catalogs lack.

Let me know how this updated script works, and we'll get it fully operational!
i deleted the other dataset csv files as they did not contain many rows of information. these are the new datasets
2Mass.csv = RAJ2000 DEJ2000 2MASS Jmag e_Jmag Hmag e_Hmag Kmag e_Kmag Qflg Rflg Bflg Cflg Xflg Aflg -- 1,000,000 rows
V154sdss16.csv = objID RA_ICRS DE_ICRS mode class clean e_RA_ICRS e_DE_ICRS umag gmag rmag imag zmag e_umag e_gmag e_rmag e_imag e_zmag zsp e_zsp f_zsp pmRA pmDE zph e_zph <zph> Q SDSS16 Sp-ID MJD -- 1,000,000 rows
V147sdss12.csv = RA_ICRS DE_ICRS mode q_mode class SDSS12 m_SDSS12 objID Sp-ID ObsDate Q umag e_umag gmag e_gmag rmag e_rmag imag e_imag zmag e_zmag zsp rpmag pmRA pmDE zph e_zph <zph> -- 1,000,000 roes
II356xmmom41s.csv = XMMOM IDsum ObsID ID UVW2r UVM2rUVM1rUr Br Vr RAJ2000 DEJ2000 ePos Nobs sig(UVW2) sig(UVM2) sig(UVW1) sig(U) sig(B) sig(V) F(UVW2) F(UVM2) F(UVW1) F(U) F(B) F(V) UVW2mAB UVM2mAB UVW1mAB UmAB BmAB VmAB q_UVW2

q_UVM2 q_UVW1 q_U q_B q_V xUVW2 xUVM2 xUVW1 xU xB xV -- 1,000,000 rows

Thank you for providing the updated dataset information! It looks like you've streamlined the datasets to four CSV files, each containing 1,000,000 rows, and you've deleted the other datasets that didn't have enough rows. Let's update the script to work with these new datasets and address the ongoing issue of the cross-matching step resulting in 0 matches.

Step 1: Understand the New Datasets

Let's map the new datasets to the ones used in the script and confirm their contents:

1. **2Mass.csv** (replaces `2mass_xsc.csv`):
   - Columns: `RAJ2000`, `DEJ2000`, `2MASS`, `Jmag`, `e_Jmag`, `Hmag`, `e_Hmag`, `Kmag`, `e_Kmag`, `Qflg`, `Rflg`, `Bflg`, `Cflg`, `Xflg`, `Aflg`
   - Rows: 1,000,000
   - This matches the expected 2MASS Extended Source Catalogue (II/246) data, with RA/Dec coordinates (`RAJ2000`, `DEJ2000`) and magnitudes (`Jmag`, `Hmag`, `Kmag`) with their errors.
2. **V154sdss16.csv** (replaces `sdss_dr16_spec.csv`):
   - Columns: `objID`, `RA_ICRS`, `DE_ICRS`, `mode`, `class`, `clean`, `e_RA_ICRS`, `e_DE_ICRS`, `umag`, `gmag`, `rmag`, `imag`, `zmag`, `e_umag`, `e_gmag`, `e_rmag`, `e_imag`, `e_zmag`, `zsp`, `e_zsp`, `f_zsp`, `pmRA`, `pmDE`, `zph`, `e_zph`, `<zph>`, `Q`, `SDSS16`, `Sp-ID`, `MJD`, ...
   - Rows: 1,000,000
   - This matches the V/154 (SDSS DR16) dataset, with RA/Dec (`RA_ICRS`, `DE_ICRS`), spectroscopic redshift (`zsp`), and magnitudes (`umag`, `gmag`, `rmag`, `imag`, `zmag`) with their errors.
3. **V147sdss12.csv** (replaces `sdss_dr12_photo.csv`):
   - Columns: `RA_ICRS`, `DE_ICRS`, `mode`, `q_mode`, `class`, `SDSS12`, `m_SDSS12`, `objID`, `Sp-ID`, `ObsDate`, `Q`, `umag`, `e_umag`, `gmag`, `e_gmag`, `rmag`, `e_rmag`, `imag`, `e_imag`, `zmag`, `e_zmag`, `zsp`, `rpmag`, `pmRA`, `pmDE`, `zph`, `e_zph`, `<zph>`, ...
   - Rows: 1,000,000
   - This matches the V/147 (SDSS DR12) dataset, with RA/Dec (`RA_ICRS`, `DE_ICRS`), spectroscopic redshift (`zsp`), and magnitudes (`umag`, `gmag`, `rmag`, `imag`, `zmag`) with their errors.
4. **II356xmmom41s.csv** (replaces `gama_dr3.csv`):
   - Columns: `XMMOM`, `IDsum`, `ObsID`, `ID`, `UVW2r`, `UVM2r`, `UVM1r`, `Ur`, `Br`, `Vr`, `RAJ2000`, `DEJ2000`, `ePos`, `Nobs`, `sig(UVW2)`, `sig(UVM2)`, `sig(UVW1)`, `sig(U)`, `sig(B)`, `sig(V)`, `F(UVW2)`, `F(UVM2)`, `F(UVW1)`, `F(U)`, `F(B)`, `F(V)`, `UVW2mAB`, `UVM2mAB`, `UVW1mAB`, `UmAB`, `BmAB`, `VmAB`, `q_UVW2`, `q_UVM2`, `q_UVW1`, `q_U`, `q_B`, `q_V`, `xUVW2`, `xUVM2`, `xUVW1`, `xU`, `xB`, `xV`

- Rows: 1,000,000
- This matches the II/356 (GAMA DR3) dataset, with RA/Dec (`RAJ2000`, `DEJ2000`) and magnitudes (`UmAB`, `BmAB`, `VmAB`).

Step 2: Address the Cross-Matching Issue

The primary issue in your previous run was that the cross-matching step resulted in 0 matches, leading to an empty DataFrame (`df`) and causing the script to fail during the `MinMaxScaler` step. In my last response, I provided an updated script that:

1. Uses a merged SDSS dataset (`sdss_merged.csv`) that you created by merging V/147 and V/154 on `objID`.
2. Includes a fallback mechanism to use the original DataFrame and `nsa_z` for redshift if no matches are found.
3. Adds debugging to print RA/Dec ranges to diagnose why no matches are being found.

However, you mentioned that you deleted the other datasets, including `sdss_merged.csv`, because they didn't contain many rows. Since `sdss_merged.csv` is no longer available, we'll need to recreate the merged dataset by merging `V154sdss16.csv` (V/154) and `V147sdss12.csv` (V/147) on `objID`. Then, we'll use this merged dataset for cross-matching, as it ensures that the photometric and spectroscopic data are aligned.

Step 3: Merge V/147 and V/154

Let's merge `V154sdss16.csv` and `V147sdss12.csv` on `objID` to create a new `sdss_merged.csv`. We'll include the key columns needed for the script: `RA_ICRS`, `DE_ICRS`, `zsp`, `umag`, `gmag`, `rmag`, `imag`, `zmag`, and their errors.
Add this code at the beginning of the script to perform the merge:

```
python
# Load the SDSS datasets
sdss_dr16 = pd.read_csv("V154sdss16.csv")  # V/154: SDSS DR16
sdss_dr12 = pd.read_csv("V147sdss12.csv")  # V/147: SDSS DR12

# Merge V/154 and V/147 on objID
sdss_merged = pd.merge(
    sdss_dr16[['objID', 'RA_ICRS', 'DE_ICRS', 'zsp', 'umag', 'gmag', 'rmag', 'imag', 'zmag', 'e_umag',
'e_gmag', 'e_rmag', 'e_imag', 'e_zmag']],
```

```
    sdss_dr12[['objID', 'RA_ICRS', 'DE_ICRS', 'zsp', 'umag', 'gmag', 'rmag', 'imag', 'zmag', 'e_umag',
'e_gmag', 'e_rmag', 'e_imag', 'e_zmag']],
    on='objID',
    how='outer',
    suffixes=('_dr16', '_dr12')
)

# Combine columns, preferring DR16 values where available, otherwise use DR12
sdss_merged['RA_ICRS'] =
sdss_merged['RA_ICRS_dr16'].combine_first(sdss_merged['RA_ICRS_dr12'])
sdss_merged['DE_ICRS'] =
sdss_merged['DE_ICRS_dr16'].combine_first(sdss_merged['DE_ICRS_dr12'])
sdss_merged['zsp'] = sdss_merged['zsp_dr16'].combine_first(sdss_merged['zsp_dr12'])
sdss_merged['umag'] = sdss_merged['umag_dr16'].combine_first(sdss_merged['umag_dr12'])
sdss_merged['gmag'] = sdss_merged['gmag_dr16'].combine_first(sdss_merged['gmag_dr12'])
sdss_merged['rmag'] = sdss_merged['rmag_dr16'].combine_first(sdss_merged['rmag_dr12'])
sdss_merged['imag'] = sdss_merged['imag_dr16'].combine_first(sdss_merged['imag_dr12'])
sdss_merged['zmag'] = sdss_merged['zmag_dr16'].combine_first(sdss_merged['zmag_dr12'])
sdss_merged['e_umag'] = sdss_merged['e_umag_dr16'].combine_first(sdss_merged['e_umag_dr12'])
sdss_merged['e_gmag'] = sdss_merged['e_gmag_dr16'].combine_first(sdss_merged['e_gmag_dr12'])
sdss_merged['e_rmag'] = sdss_merged['e_rmag_dr16'].combine_first(sdss_merged['e_rmag_dr12'])
sdss_merged['e_imag'] = sdss_merged['e_imag_dr16'].combine_first(sdss_merged['e_imag_dr12'])
sdss_merged['e_zmag'] = sdss_merged['e_zmag_dr16'].combine_first(sdss_merged['e_zmag_dr12'])

# Drop the intermediate columns
sdss_merged = sdss_merged[['objID', 'RA_ICRS', 'DE_ICRS', 'zsp', 'umag', 'gmag', 'rmag', 'imag', 'zmag',
'e_umag', 'e_gmag', 'e_rmag', 'e_imag', 'e_zmag']]

# Save the merged dataset for future use
sdss_merged.to_csv("sdss_merged.csv", index=False)
print(f"Merged SDSS dataset created with {len(sdss_merged)} rows.")
```

This code merges the two SDSS datasets, prioritizes DR16 values (since it's more recent), and saves the result as `sdss_merged.csv`. The merged dataset will have up to 1,000,000 rows, depending on how many `objID` values overlap between the two datasets.

Step 4: Update the Script with New Datasets

Now, let's update the script to use the new dataset filenames and the merged SDSS dataset. We'll also increase the cross-matching radius to 10 arcseconds (from 5 arcseconds) to see if that helps find matches, and we'll add more debugging to understand why the cross-matching is failing.
Here’s the updated script:

```
python
# STAR.py: Model for predicting log_SFR_Ha using the STAR framework
# STAR: Symbolic, Tuned (Optuna), Astronomy Regressors (Random Forest + Gradient Boosting)
```

```
# - S: Symbolic Regressor (via gplearn)
# - T: Tuned (hyperparameter optimization with Optuna)
# - A: Astronomy (predicting star formation rates in galaxies)
# - R: Regressors (Random Forest and Gradient Boosting)

import pandas as pd
import numpy as np
import shap
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split, cross_val_score, KFold
from sklearn.ensemble import RandomForestRegressor, GradientBoostingRegressor
from sklearn.metrics import r2_score
from sklearn.preprocessing import StandardScaler, MinMaxScaler
from gplearn.genetic import SymbolicRegressor
import optuna
import warnings
from numpy.polynomial import Polynomial
from scipy.spatial import cKDTree

warnings.filterwarnings("ignore")

# Load the original dataset
df = pd.read_csv("merged_data.csv")

# Load the SDSS datasets and merge them
sdss_dr16 = pd.read_csv("V154sdss16.csv")  # V/154: SDSS DR16
sdss_dr12 = pd.read_csv("V147sdss12.csv")  # V/147: SDSS DR12

# Merge V/154 and V/147 on objID
sdss_merged = pd.merge(
    sdss_dr16[['objID', 'RA_ICRS', 'DE_ICRS', 'zsp', 'umag', 'gmag', 'rmag', 'imag', 'zmag', 'e_umag',
'e_gmag', 'e_rmag', 'e_imag', 'e_zmag']],
    sdss_dr12[['objID', 'RA_ICRS', 'DE_ICRS', 'zsp', 'umag', 'gmag', 'rmag', 'imag', 'zmag', 'e_umag',
'e_gmag', 'e_rmag', 'e_imag', 'e_zmag']],
    on='objID',
    how='outer',
    suffixes=('_dr16', '_dr12')
)

# Combine columns, preferring DR16 values where available, otherwise use DR12
sdss_merged['RA_ICRS'] =
sdss_merged['RA_ICRS_dr16'].combine_first(sdss_merged['RA_ICRS_dr12'])
sdss_merged['DE_ICRS'] =
sdss_merged['DE_ICRS_dr16'].combine_first(sdss_merged['DE_ICRS_dr12'])
sdss_merged['zsp'] = sdss_merged['zsp_dr16'].combine_first(sdss_merged['zsp_dr12'])
sdss_merged['umag'] = sdss_merged['umag_dr16'].combine_first(sdss_merged['umag_dr12'])
sdss_merged['gmag'] = sdss_merged['gmag_dr16'].combine_first(sdss_merged['gmag_dr12'])
sdss_merged['rmag'] = sdss_merged['rmag_dr16'].combine_first(sdss_merged['rmag_dr12'])
```

```
sdss_merged['imag'] = sdss_merged['imag_dr16'].combine_first(sdss_merged['imag_dr12'])
sdss_merged['zmag'] = sdss_merged['zmag_dr16'].combine_first(sdss_merged['zmag_dr12'])
sdss_merged['e_umag'] = sdss_merged['e_umag_dr16'].combine_first(sdss_merged['e_umag_dr12'])
sdss_merged['e_gmag'] = sdss_merged['e_gmag_dr16'].combine_first(sdss_merged['e_gmag_dr12'])
sdss_merged['e_rmag'] = sdss_merged['e_rmag_dr16'].combine_first(sdss_merged['e_rmag_dr12'])
sdss_merged['e_imag'] = sdss_merged['e_imag_dr16'].combine_first(sdss_merged['e_imag_dr12'])
sdss_merged['e_zmag'] = sdss_merged['e_zmag_dr16'].combine_first(sdss_merged['e_zmag_dr12'])

# Drop the intermediate columns
sdss_merged = sdss_merged[['objID', 'RA_ICRS', 'DE_ICRS', 'zsp', 'umag', 'gmag', 'rmag', 'imag', 'zmag',
'e_umag', 'e_gmag', 'e_rmag', 'e_imag', 'e_zmag']]

# Save the merged dataset for future use
sdss_merged.to_csv("sdss_merged.csv", index=False)
print(f"Merged SDSS dataset created with {len(sdss_merged)} rows.")

# Load the other datasets
twomass = pd.read_csv("2Mass.csv")  # II/246: 2MASS Extended Source Catalogue
gama = pd.read_csv("II356xmmom41s.csv")  # II/356: GAMA DR3

# Print columns for debugging
print("Columns in merged_data.csv:", df.columns.tolist())
print("Columns in sdss_merged (V/147 + V/154):", sdss_merged.columns.tolist())
print("Columns in twomass (II/246):", twomass.columns.tolist())
print("Columns in gama (II/356):", gama.columns.tolist())

# Function to clean DataFrame by removing rows with NaN or inf in RA/Dec columns
def clean_coordinates(df, ra_col, dec_col):
    initial_len = len(df)
    # Check for NaN or inf in RA and Dec columns
    mask = (
        df[ra_col].notna() & df[dec_col].notna() &  # Not NaN
        np.isfinite(df[ra_col]) & np.isfinite(df[dec_col])  # Not inf
    )
    df_cleaned = df[mask].copy()
    print(f"Removed {initial_len - len(df_cleaned)} rows from DataFrame due to NaN or inf in {ra_col} or
{dec_col}")
    # Print RA/Dec ranges for debugging
    if len(df_cleaned) > 0:
        print(f"RA range in {ra_col}: {df_cleaned[ra_col].min():.4f} to {df_cleaned[ra_col].max():.4f}")
        print(f"Dec range in {dec_col}: {df_cleaned[dec_col].min():.4f} to {df_cleaned[dec_col].max():.4f}")
    else:
        print(f"No valid RA/Dec data after cleaning in {ra_col}/{dec_col}")
    return df_cleaned

# Cross-match function
def deg_to_rad(df, ra_col, dec_col):
    try:
        df['ra_rad'] = np.radians(df[ra_col])
```

```
        df['dec_rad'] = np.radians(df[dec_col])
        return df
    except KeyError as e:
        print(f"KeyError in deg_to_rad: {e}")
        print(f"Available columns in DataFrame: {df.columns.tolist()}")
        raise

def cross_match(df1, df2, ra_col1, dec_col1, ra_col2, dec_col2, max_dist_arcsec=10.0):
    try:
        # Clean DataFrames to remove rows with NaN or inf in RA/Dec
        df1_cleaned = clean_coordinates(df1, ra_col1, dec_col1)
        df2_cleaned = clean_coordinates(df2, ra_col2, dec_col2)

        # Check if either DataFrame is empty after cleaning
        if len(df1_cleaned) == 0 or len(df2_cleaned) == 0:
            print("One of the DataFrames is empty after cleaning. Cannot perform cross-match.")
            return df1_cleaned, pd.DataFrame()

        # Convert degrees to radians
        df1_cleaned = deg_to_rad(df1_cleaned, ra_col1, dec_col1)
        df2_cleaned = deg_to_rad(df2_cleaned, ra_col2, dec_col2)

        # Create coordinate arrays
        coords1 = np.array([df1_cleaned['ra_rad'], df1_cleaned['dec_rad']]).T
        coords2 = np.array([df2_cleaned['ra_rad'], df2_cleaned['dec_rad']]).T

        # Perform cross-match using cKDTree
        tree = cKDTree(coords2)
        max_dist = np.radians(max_dist_arcsec / 3600.0)
        dist, idx = tree.query(coords1, k=1, distance_upper_bound=max_dist)

        # Filter matches within the maximum distance
        matched = dist < max_dist
        df1_matched = df1_cleaned[matched].copy()
        df2_matched = df2_cleaned.iloc[idx[matched]].copy()

        # Reset indices
        df1_matched = df1_matched.reset_index(drop=True)
        df2_matched = df2_matched.reset_index(drop=True)

        # Print sample matches for debugging
        if len(df1_matched) > 0:
            print("Sample matches (first 5):")
            for i in range(min(5, len(df1_matched))):
                print(f"Match {i+1}: {ra_col1}={df1_matched[ra_col1].iloc[i]:.4f},
{dec_col1}={df1_matched[dec_col1].iloc[i]:.4f} "
                      f"matched to {ra_col2}={df2_matched[ra_col2].iloc[i]:.4f},
{dec_col2}={df2_matched[dec_col2].iloc[i]:.4f} "
                      f"(distance={(dist[matched][i] * 3600 * 180 / np.pi):.2f} arcsec)")
```

```
        else:
            print("No matches found within the specified radius.")

        return df1_matched, df2_matched
    except KeyError as e:
        print(f"KeyError in cross_match: {e}")
        print(f"df1 columns: {df1.columns.tolist()}")
        print(f"df2 columns: {df2.columns.tolist()}")
        raise
    except Exception as e:
        print(f"Error in cross_match: {e}")
        raise

# Store the original DataFrame before cross-matching
df_original = df.copy()

# Cross-match with VizieR catalogues
# SDSS merged dataset (V/147 + V/154, uses RA_ICRS, DE_ICRS)
df, sdss_matched = cross_match(df, sdss_merged, 'objra_y', 'objdec', 'RA_ICRS', 'DE_ICRS',
max_dist_arcsec=10.0)
print(f"After SDSS cross-match: {len(df)} galaxies")

# Check if cross-match succeeded; if not, fall back to original DataFrame
if len(df) == 0:
    print("No matches found with SDSS merged data (V/147 + V/154). Falling back to original data and
using nsa_z for redshift.")
    df = df_original.copy()
    df['zsp'] = df['nsa_z']
else:
    # Add zsp, umag, gmag, etc. from the merged SDSS dataset
    df = pd.concat([df, sdss_matched[['zsp', 'umag', 'gmag', 'rmag', 'imag', 'zmag', 'e_umag', 'e_gmag',
'e_rmag', 'e_imag', 'e_zmag']]], axis=1)

# II/246: 2MASS Extended Source Catalogue (uses RAJ2000, DEJ2000)
df, twomass_matched = cross_match(df, twomass, 'objra_y', 'objdec', 'RAJ2000', 'DEJ2000',
max_dist_arcsec=10.0)
print(f"After II/246 cross-match: {len(df)} galaxies")
if len(df) > 0:
    df = pd.concat([df, twomass_matched[['Jmag', 'Hmag', 'Kmag', 'e_Jmag', 'e_Hmag', 'e_Kmag']]], axis=1)

# II/356: GAMA DR3 (uses RAJ2000, DEJ2000)
df, gama_matched = cross_match(df, gama, 'objra_y', 'objdec', 'RAJ2000', 'DEJ2000',
max_dist_arcsec=10.0)
print(f"After II/356 cross-match: {len(df)} galaxies")
if len(df) > 0:
    df = pd.concat([df, gama_matched[['UmAB', 'BmAB', 'VmAB']]], axis=1)

# Recompute log_SFR_Ha and metallicity using fluxes from merged_data.csv
df['flux_Ha'] = df['F_Ha_cen']  # Use F_Ha_cen as flux_Ha
```

```
df['e_flux_Ha'] = df['e_F_Ha_cen']
df['flux_Hb'] = df['flux_Hbeta4861.36_Re_fit']
df['flux_OIII_5007'] = df['flux_[OIII]5006.84_Re_fit']
df['flux_NII_6584'] = df['flux_[NII]6583.45_Re_fit']

# Check for NaN in flux columns and handle them
flux_cols = ['flux_Ha', 'flux_Hb', 'flux_OIII_5007', 'flux_NII_6584', 'e_flux_Ha']
for col in flux_cols:
    nan_count = df[col].isna().sum()
    print(f"NaN count in {col}: {nan_count}")
    # Replace NaN with median for flux columns
    if nan_count > 0:
        df[col] = df[col].fillna(df[col].median())

df['Ha_Hb_observed'] = df['flux_Ha'] / df['flux_Hb']
Ha_Hb_intrinsic = 2.86
k_Ha = 2.468
k_Hb = 3.634
df['A_Ha'] = 2.5 * np.log10(df['Ha_Hb_observed'] / Ha_Hb_intrinsic) * (k_Ha / (k_Hb - k_Ha))
df['flux_Ha_corr'] = df['flux_Ha'] * 10**(0.4 * df['A_Ha'])
H0 = 70  # km/s/Mpc
c = 3e5  # km/s
df['DL'] = (c * df['zsp'] / H0) * 3.0856e24  # cm, using zsp
df['L_Ha'] = df['flux_Ha_corr'] * 4 * np.pi * df['DL']**2  # erg/s
df['log_SFR_Ha_raw'] = np.log10(df['L_Ha'] * 7.9e-42)  # M_sun/yr
df['log_O3N2_raw'] = np.log10((df['flux_OIII_5007'] / df['flux_Hb']) / (df['flux_NII_6584'] / df['flux_Ha']))
df['OH_O3N2_raw'] = 8.533 - 0.214 * df['log_O3N2_raw']

# Add raw fluxes as features
df['log_flux_Ha'] = np.log10(df['flux_Ha'] + 1e-10)
df['log_flux_Hb'] = np.log10(df['flux_Hb'] + 1e-10)
df['log_flux_OIII_5007'] = np.log10(df['flux_OIII_5007'] + 1e-10)
df['log_flux_NII_6584'] = np.log10(df['flux_NII_6584'] + 1e-10)
df['log_e_flux_Ha'] = np.log10(df['e_flux_Ha'] + 1e-10)

# Add photometric colors (if available)
if 'umag' in df.columns:
    df['color_ug'] = df['umag'] - df['gmag']
    df['color_gr'] = df['gmag'] - df['rmag']
    df['color_ri'] = df['rmag'] - df['imag']
    df['color_iz'] = df['imag'] - df['zmag']
    df['e_color_ug'] = np.sqrt(df['e_umag']**2 + df['e_gmag']**2)
    df['e_color_gr'] = np.sqrt(df['e_gmag']**2 + df['e_rmag']**2)

# Add 2MASS colors (if available)
if 'Jmag' in df.columns:
    df['color_JK'] = df['Jmag'] - df['Kmag']
    df['e_color_JK'] = np.sqrt(df['e_Jmag']**2 + df['e_Kmag']**2)
```

```
# Drop rows with missing target or excessive NaNs
df = df.dropna(subset=["log_SFR_Ha"])
df = df.dropna(axis=0, thresh=int(0.8 * df.shape[1]))
df.fillna(df.median(numeric_only=True), inplace=True)

# Step 1: Cosmo-Rank Construction
gz_df = df.copy()
morph_cols = ['P(CD)', 'P(E)', 'P(S0)', 'P(Sa)', 'P(Sab)', 'P(Sb)', 'P(Sbc)',
              'P(Sc)', 'P(Scd)', 'P(Sd)', 'P(Sdm)', 'P(Sm)', 'P(Irr)']
gz_df['morph_sum'] = gz_df[morph_cols].sum(axis=1)
gz_df['Num_w'] = gz_df['conf_prob']
scaler_rank = MinMaxScaler()
gz_df[['morph_sum_norm', 'Num_w_norm', 'nsa_z_norm']] = scaler_rank.fit_transform(
    gz_df[['morph_sum', 'Num_w', 'nsa_z']]
)
gz_df['cosmo_rank'] = (0.5 * gz_df['Num_w_norm'] + 0.3 * gz_df['morph_sum_norm'] + 0.2 *
gz_df['nsa_z_norm'])
df['cosmo_rank'] = gz_df['cosmo_rank']

# Step 2: L_cosmo(s) Construction
alpha = -1.5
M_star = df['log_Mass_gas'].median()
df['a_n'] = (10 ** df['log_Mass_gas'])**(1 + alpha) * np.exp(-10 ** df['log_Mass_gas'] / (10 ** M_star))
bins = pd.qcut(df['Re_kpc'], q=20, labels=False, duplicates='drop') + 1
df['z_bin'] = bins
n_bins = df['z_bin'].nunique()
print(f"Number of unique bins after qcut: {n_bins}")
print("Bin distribution for Re_kpc (after qcut):")
print(df['z_bin'].value_counts().sort_index())

s_vals = [0.5, 1.0, 1.5, 2.0]
s_range = np.linspace(0.5, 2, 50)
for s in s_vals:
    col_name = f'L_cosmo_s_{s:.1f}'
    df[col_name] = df.groupby('z_bin')['a_n'].transform('mean') / (df['z_bin'] ** s)

print("Columns after creating L_cosmo_s*:")
print(df.columns.tolist())

print("Checking for NaN/infinite values in cosmo_rank and L_cosmo_s_1.0:")
print("cosmo_rank NaN count:", df['cosmo_rank'].isna().sum())
print("L_cosmo_s_1.0 NaN count:", df['L_cosmo_s_1.0'].isna().sum())
print("cosmo_rank infinite count:", np.isinf(df['cosmo_rank']).sum())
print("L_cosmo_s_1.0 infinite count:", np.isinf(df['L_cosmo_s_1.0']).sum())

df['cosmo_rank_L'] = df['cosmo_rank'] * df['L_cosmo_s_1.0']
df['cosmo_rank_L'] = df['cosmo_rank_L'].replace([np.inf, -np.inf], np.nan).fillna(0)
print("cosmo_rank_L created, NaN count:", df['cosmo_rank_L'].isna().sum())
```

```
# Plot L_cosmo(s) vs s
grouped_means = df.groupby('z_bin')['a_n'].mean().reindex(range(1, n_bins + 1), fill_value=0)
L_vals = [grouped_means / (np.arange(1, n_bins + 1) ** s) for s in s_range]
L_vals = np.array([sum(l) for l in L_vals])
plt.plot(s_range, L_vals)
plt.xlabel("s")
plt.ylabel("L_cosmo(s)")
plt.title("L_cosmo(s) Behavior")
plt.savefig("L_cosmo_curve.png")
plt.close()

# Scatter plot of L_cosmo_s_1.0 vs log_SFR_Ha
plt.scatter(df['L_cosmo_s_1.0'], df['log_SFR_Ha'], alpha=0.5)
plt.xlabel("L_cosmo_s_1.0")
plt.ylabel("log_SFR_Ha")
plt.title("L_cosmo_s_1.0 vs log_SFR_Ha")
plt.savefig("L_cosmo_s1_vs_SFR.png")
plt.close()

# Step 3: Define features and target
target = "log_SFR_Ha"
base_features = [
    "log_Mass_gas", "log_Mass", "Av_gas_Re", "Av_ssp_Re",
    "OH_O3N2_cen", "OH_T04_cen", "OH_dop_cen", "Age_LW_Re_fit",
    "ZH_LW_Re_fit", "Re_kpc", "vel_disp_Ha_cen", "Lambda_Re",
    "Sigma_Mass_Re", "EW_Ha_cen", "Ha_Hb_cen"
]
df["mass_metallicity"] = df["log_Mass"] * df["OH_O3N2_cen"]
df["dust_metallicity"] = df["Av_gas_Re"] * df["OH_T04_cen"]
df["disp_mass_ratio"] = df["vel_disp_Ha_cen"] / (df["Sigma_Mass_Re"] + 1e-5)
df["age_metallicity"] = df["Age_LW_Re_fit"] * df["ZH_LW_Re_fit"]
df["sqrt_Re_kpc"] = np.sqrt(df['Re_kpc'] + 1e-5)
df["log_mass"] = np.log(df["log_Mass_gas"] + 1e-5)
df["BSD_likelihood"] = (
    df["log_Mass_gas"] * df["OH_O3N2_cen"] /
    (df["Av_gas_Re"] + 1e-5 + df["Age_LW_Re_fit"])
)
df["cosmo_rank_mass"] = df["cosmo_rank"] * df["log_Mass"]
df["L_cosmo_s1_mass"] = df["L_cosmo_s_1.0"] * df["log_Mass"]
df["cosmo_rank_EW"] = df["cosmo_rank"] * df["EW_Ha_cen"]
df["L_cosmo_s1_EW"] = df["L_cosmo_s_1.0"] * df["EW_Ha_cen"]
df["cosmo_rank_scaled"] = df["cosmo_rank"] * 20
df["L_cosmo_s1_scaled"] = df["L_cosmo_s_1.0"] * 20
df["cosmo_rank_L_mass"] = df["cosmo_rank_L"] * df["log_Mass"]
df["L_cosmo_s1_metallicity"] = df["L_cosmo_s_1.0"] * df["OH_O3N2_cen"]
features = base_features + [
    "mass_metallicity", "dust_metallicity", "disp_mass_ratio",
    "age_metallicity", "sqrt_Re_kpc", "log_mass", "BSD_likelihood",
    "cosmo_rank", "L_cosmo_s_0.5", "L_cosmo_s_1.5", "L_cosmo_s_2.0",
```

```
    "cosmo_rank_L", "cosmo_rank_mass", "L_cosmo_s1_mass",
    "cosmo_rank_EW", "L_cosmo_s1_EW", "cosmo_rank_scaled", "L_cosmo_s1_scaled",
    "cosmo_rank_L_mass", "L_cosmo_s1_metallicity",
    "log_flux_Ha", "log_flux_Hb", "log_flux_OIII_5007", "log_flux_NII_6584",
    "log_e_flux_Ha", "log_SFR_Ha_raw", "OH_O3N2_raw"
]

# Add color features if available
if 'color_ug' in df.columns:
    features.extend(["color_ug", "color_gr", "color_ri", "color_iz", "e_color_ug", "e_color_gr"])
if 'color_JK' in df.columns:
    features.extend(["color_JK", "e_color_JK"])

X = df[features]
y = df[target]

# Train/test split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Normalize features
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# Optuna optimization for Random Forest
def objective_rf(trial):
    params = {
        'n_estimators': trial.suggest_int('n_estimators', 100, 500),
        'max_depth': trial.suggest_int('max_depth', 5, 20),
        'min_samples_split': trial.suggest_int('min_samples_split', 2, 10),
        'random_state': 42
    }
    model = RandomForestRegressor(**params)
    cv = KFold(n_splits=5, shuffle=True, random_state=42)
    scores = cross_val_score(model, X_train_scaled, y_train, cv=cv, scoring='r2')
    return scores.mean()

study_rf = optuna.create_study(direction='maximize')
study_rf.optimize(objective_rf, n_trials=20)
best_params_rf = study_rf.best_params
print("Best Random Forest Parameters:", best_params_rf)
print("Best Random Forest CV R²:", study_rf.best_value)

# Train Random Forest with best parameters
rf = RandomForestRegressor(**best_params_rf, random_state=42)
rf.fit(X_train_scaled, y_train)
y_pred_rf = rf.predict(X_test_scaled)
r2_rf = r2_score(y_test, y_pred_rf)
```

```
# Cross-validation for final RF model
cv_scores_rf = cross_val_score(rf, X_train_scaled, y_train, cv=5, scoring='r2')
print(f"Random Forest 5-Fold CV R²: Mean = {cv_scores_rf.mean():.4f}, Std = {cv_scores_rf.std():.4f}")

# Optuna optimization for Gradient Boosting
def objective_gb(trial):
    params = {
        'n_estimators': trial.suggest_int('n_estimators', 100, 500),
        'learning_rate': trial.suggest_float('learning_rate', 0.01, 0.2, log=True),
        'max_depth': trial.suggest_int('max_depth', 3, 10),
        'random_state': 42
    }
    model = GradientBoostingRegressor(**params)
    cv = KFold(n_splits=5, shuffle=True, random_state=42)
    scores = cross_val_score(model, X_train_scaled, y_train, cv=cv, scoring='r2')
    return scores.mean()

study_gb = optuna.create_study(direction='maximize')
study_gb.optimize(objective_gb, n_trials=20)
best_params_gb = study_gb.best_params
print("Best Gradient Boosting Parameters:", best_params_gb)
print("Best Gradient Boosting CV R²:", study_gb.best_value)

# Train Gradient Boosting with best parameters
gb = GradientBoostingRegressor(**best_params_gb, random_state=42)
gb.fit(X_train_scaled, y_train)
y_pred_gb = gb.predict(X_test_scaled)
r2_gb = r2_score(y_test, y_pred_gb)

# Cross-validation for final GB model
cv_scores_gb = cross_val_score(gb, X_train_scaled, y_train, cv=5, scoring='r2')
print(f"Gradient Boosting 5-Fold CV R²: Mean = {cv_scores_gb.mean():.4f}, Std =
{cv_scores_gb.std():.4f}")

print(f"R^2 Score (Random Forest, Test Set): {r2_rf:.4f}")
print(f"R^2 Score (Gradient Boosting, Test Set): {r2_gb:.4f}")

# SHAP for feature selection
explainer_rf = shap.Explainer(rf, X_train_scaled)
shap_values_rf = explainer_rf(X_test_scaled)
explainer_gb = shap.Explainer(gb, X_train_scaled)
shap_values_gb = explainer_gb(X_test_scaled)
shap.summary_plot(shap_values_rf, X_test_scaled, plot_type="bar", show=False)
plt.title("SHAP Summary - Random Forest")
plt.tight_layout()
plt.savefig("shap_rf_summary.png")
plt.clf()
shap.summary_plot(shap_values_gb, X_test_scaled, plot_type="bar", show=False)
plt.title("SHAP Summary - Gradient Boosting")
```

```
plt.tight_layout()
plt.savefig("shap_gb_summary.png")
plt.clf()

# Feature selection based on SHAP
shap_values_rf_mean = np.abs(shap_values_rf.values).mean(axis=0)
feature_importance = pd.DataFrame({"feature": features, "importance": shap_values_rf_mean})
feature_importance = feature_importance.sort_values("importance", ascending=False)
top_features = feature_importance["feature"].head(10).tolist()
bsd_features = ["cosmo_rank", "L_cosmo_s_1.0", "cosmo_rank_L", "cosmo_rank_scaled",
"L_cosmo_s1_scaled", "cosmo_rank_L_mass", "L_cosmo_s1_metallicity"]
for bsd_feature in bsd_features:
    if bsd_feature not in top_features:
        top_features.append(bsd_feature)
X_selected = df[top_features]
X_train_selected, X_test_selected, _, _ = train_test_split(X_selected, y, test_size=0.2, random_state=42)
X_train_selected_scaled = scaler.fit_transform(X_train_selected)
X_test_selected_scaled = scaler.transform(X_test_selected)

# Symbolic Regression with gplearn
symbolic_model = SymbolicRegressor(
    population_size=3000,
    generations=150,
    stopping_criteria=0.01,
    p_crossover=0.7,
    p_subtree_mutation=0.1,
    p_hoist_mutation=0.05,
    p_point_mutation=0.1,
    max_samples=0.9,
    verbose=1,
    parsimony_coefficient=0.0001,
    random_state=42,
    function_set=('add', 'sub', 'mul', 'div', 'sqrt', 'log', 'sin', 'cos')
)
symbolic_model.fit(X_train_selected_scaled, y_train)
y_pred_sym = symbolic_model.predict(X_test_selected_scaled)
r2_sym = r2_score(y_test, y_pred_sym)
print(f"gplearn R²: {r2_sym:.4f}")
print("Symbolic Expression:")
print(symbolic_model._program)

# Cross-validation for symbolic model
cv_scores_sym = cross_val_score(symbolic_model, X_train_selected_scaled, y_train, cv=5, scoring='r2')
print(f"Symbolic Regression 5-Fold CV R²: Mean = {cv_scores_sym.mean():.4f}, Std =
{cv_scores_sym.std():.4f}")

# Save symbolic expression as polynomial fit plot and print the equation
x = np.linspace(min(y_test), max(y_test), 500)
y_expr = symbolic_model.predict(scaler.transform(np.tile(X_test_selected.mean().values, (500,1))))
```

```
p = Polynomial.fit(y_test, y_pred_sym, deg=3)
# Extract polynomial coefficients
coeffs = p.coef
print("Polynomial Fit Equation (degree 3):")
print(f"y = {coeffs[0]:.4f} + {coeffs[1]:.4f}*x + {coeffs[2]:.4f}*x^2 + {coeffs[3]:.4f}*x^3")
# Plot the polynomial fit
plt.plot(*p.linspace(), label="Polynomial Fit")
plt.scatter(y_test, y_pred_sym, s=10, alpha=0.5, label="Symbolic Predictions")
plt.xlabel("True log_SFR_Ha")
plt.ylabel("Predicted log_SFR_Ha")
plt.title("Symbolic Regression Fit with Polynomial")
plt.legend()
plt.tight_layout()
plt.savefig("gplearn_expression_plot.png")
plt.clf()

# Plot predictions vs true
plt.scatter(y_test, y_pred_rf, alpha=0.5, label="RF", marker="o")
plt.scatter(y_test, y_pred_gb, alpha=0.5, label="GB", marker="s")
plt.scatter(y_test, y_pred_sym, alpha=0.5, label="Symbolic", marker="^")
plt.plot([y.min(), y.max()], [y.min(), y.max()], "k--")
plt.xlabel("True log_SFR_Ha")
plt.ylabel("Predicted")
plt.title("Model Predictions vs True Values")
plt.legend()
plt.tight_layout()
plt.savefig("model_comparison.png")
plt.clf()

# Residuals and Correlation Analysis
residuals = pd.DataFrame({
    "True": y_test,
    "RF": y_pred_rf - y_test,
    "GB": y_pred_gb - y_test,
    "Symbolic": y_pred_sym - y_test,
    "cosmo_rank": X_test["cosmo_rank"],
    "L_cosmo_s1": X_test["L_cosmo_s_1.0"]
})
sns.scatterplot(data=residuals, x="True", y="Symbolic", hue="cosmo_rank", size="L_cosmo_s1",
alpha=0.6)
plt.axhline(0, color="black", linestyle="--")
plt.title("Symbolic Residuals vs True log_SFR_Ha")
plt.savefig("residuals_sym.png")
plt.clf()

print("Correlation of BSD Features with Residuals:")
print(residuals[["RF", "GB", "Symbolic", "cosmo_rank", "L_cosmo_s1"]].corr()[["RF", "GB", "Symbolic"]])
```

Step 5: Changes Made

1. **Updated Dataset Filenames**:
    - `2mass_xsc.csv` → `2Mass.csv`
    - `gama_dr3.csv` → `II356xmmom41s.csv`
    - Removed separate loading of `sdss_dr16_spec.csv` and `sdss_dr12_photo.csv`, replacing them with a merge of `V154sdss16.csv` and `V147sdss12.csv`.
2. **Merged SDSS Datasets**:
    - Added code to merge `V154sdss16.csv` (V/154) and `V147sdss12.csv` (V/147) on `objID`, creating `sdss_merged.csv`.
    - The merged dataset prioritizes DR16 values but fills in with DR12 values where DR16 is missing.
3. **Increased Cross-Matching Radius**:
    - Changed `max_dist_arcsec` from 5.0 to 10.0 to increase the likelihood of finding matches.
4. **Added Debugging for Cross-Matching**:
    - The `cross_match` function now prints the RA/Dec ranges of both DataFrames after cleaning.
    - It also prints sample matches (if any) with their coordinates and distances to help diagnose why matches are failing.
5. **Fallback Mechanism**:
    - If the SDSS cross-match fails, the script falls back to the original DataFrame (`df_original`) and uses `nsa_z` for redshift, ensuring the script can continue running.

Step 6: Diagnosing the Cross-Matching Issue

When you run this script, it will print the RA/Dec ranges for `merged_data.csv` (`objra_y`, `objdec`) and `sdss_merged` (`RA_ICRS`, `DE_ICRS`). This will help us determine why no matches are being found. Possible reasons include:

1. **Coordinate Mismatch**:
    - The RA/Dec values in `merged_data.csv` might not overlap with those in `sdss_merged`. For example, if `merged_data.csv` covers a different region of the sky than the SDSS datasets, there will be no matches.
    - The RA/Dec ranges printed by the script will confirm this.
2. **Unit or Format Issues**:

- The RA/Dec values might be in different units (e.g., degrees vs. radians) or formats. However, the script assumes all RA/Dec values are in degrees (which is standard for these catalogs), and the `deg_to_rad` function converts them to radians for the cross-match.

3. **Small Sample Size**:
   - After cleaning `merged_data.csv` for `NaN` or `inf` values in `objra_y` and `objdec`, there might be very few rows left to match. In your previous run, 561 rows were removed, leaving an unknown number of rows. If this number is small, the chance of finding matches decreases.
4. **Astrometric Errors**:
   - There might be systematic offsets in the RA/Dec values between `merged_data.csv` and the SDSS datasets, requiring a larger matching radius or an offset correction.

Step 7: What to Do Next

1. **Run the Updated Script**:
   - Save the script as `STAR.py` and run it:
   - `bash`
   - python STAR.py
   - The script will print the RA/Dec ranges for `merged_data.csv` and `sdss_merged`, as well as the number of matches found.
2. **Check the Output**:
   - Look at the RA/Dec ranges printed for `merged_data.csv` (`objra_y`, `objdec`) and `sdss_merged` (`RA_ICRS`, `DE_ICRS`). For example:

RA range in objra_y: 120.5 to 240.8
Dec range in objdec: 10.2 to 60.7
RA range in RA_ICRS: 0.0 to 359.9

   - Dec range in DE_ICRS: -1.0 to 85.0
   - If the ranges don't overlap (e.g., `objra_y` is between 300 and 350, but `RA_ICRS` is between 0 and 180), there will be no matches.
3. **If No Matches Are Found**:
   - The script will fall back to using `nsa_z` for redshift and continue running, so you should be able to get results even if cross-matching fails.

    - Share the RA/Dec ranges printed by the script, and we can investigate further. For example, we might need to:
        - Increase the matching radius further (e.g., to 30 arcseconds).
        - Apply an offset to the RA/Dec values if there's a systematic difference.
        - Filter the SDSS dataset to a specific region of the sky that overlaps with `merged_data.csv`.
4. **If Matches Are Found**:
    - The script will proceed with the cross-matched data, adding `zsp`, `umag`, `gmag`, etc., from the SDSS dataset, and continue with the machine learning steps.

Step 8: Additional Notes

- **Merged Dataset Size**: The merged SDSS dataset (`sdss_merged.csv`) might have fewer than 1,000,000 rows if there are few overlapping `objID` values between V/154 and V/147. The script will print the number of rows in the merged dataset, which will help us understand if this is an issue.
- **Performance**: Cross-matching with 1,000,000 rows can be slow. If this becomes a bottleneck, we can filter the SDSS dataset to a smaller region of the sky based on the RA/Dec range of `merged_data.csv`.
- **Fallback Mode**: Even if cross-matching fails, the script will now continue running using `nsa_z` for redshift, so you should be able to get results regardless.

Let me know how this updated script works, and we'll get it fully operational! If cross-matching still fails, please share the RA/Dec ranges printed by the script, and we can dig deeper into the issue.

ParserError Traceback (most recent call last) File ~/workspace/Temporary/pmqr771/scratch/STAR.py:301 298 df = pd.concat([df, sdss_matched], axis=1) 300 # II/246: 2MASS Extended Source Catalogue (uses RAJ2000, DEJ2000) --> 301 df, twomass_matched = cross_match_in_chunks(302 df, "2Mass_cleaned.csv" if '2Mass_cleaned.csv' in globals() else "2Mass.csv", 303 'objra_y', 'objdec', 'RAJ2000', 'DEJ2000', 304 ['Jmag', 'Hmag', 'Kmag', 'e_Jmag', 'e_Hmag', 'e_Kmag'], 305 max_dist_arcsec=10.0, chunksize=100000 306) 307 print(f"After II/246 cross-match: {len(df)} galaxies") 308 if len(df) > 0:
File ~/workspace/Temporary/pmqr771/scratch/STAR.py:220, in cross_match_in_chunks(df1, filepath, ra_col1, dec_col1, ra_col2, dec_col2, columns_to_keep, max_dist_arcsec, chunksize) 218 matched_df2_list = [] 219 chunk_reader = pd.read_csv(filepath, chunksize=chunksize) --> 220 for i, chunk in

enumerate(chunk_reader): 221 print(f"Processing chunk {i+1} with {len(chunk)} rows...") 222 # Filter chunk by RA/Dec range
File ~/mambaforge/envs/py39/lib/python3.9/site-packages/pandas/io/parsers/readers.py:1698, in TextFileReader.__next__(self) 1696 def __next__(self) -> DataFrame: 1697 try: -> 1698 return self.get_chunk() 1699 except StopIteration: 1700 self.close()
File ~/mambaforge/envs/py39/lib/python3.9/site-packages/pandas/io/parsers/readers.py:1810, in TextFileReader.get_chunk(self, size) 1808 raise StopIteration 1809 size = min(size, self.nrows - self._currow) -> 1810 return self.read(nrows=size)
File ~/mambaforge/envs/py39/lib/python3.9/site-packages/pandas/io/parsers/readers.py:1778, in TextFileReader.read(self, nrows) 1771 nrows = validate_integer("nrows", nrows) 1772 try: 1773 # error: "ParserBase" has no attribute "read" 1774 (1775 index, 1776 columns, 1777 col_dict, -> 1778) = self._engine.read(# type: ignore[attr-defined] 1779 nrows 1780) 1781 except Exception: 1782 self.close()
File ~/mambaforge/envs/py39/lib/python3.9/site-packages/pandas/io/parsers/c_parser_wrapper.py:230, in CParserWrapper.read(self, nrows) 228 try: 229 if self.low_memory: --> 230 chunks = self._reader.read_low_memory(nrows) 231 # destructive to chunks 232 data = _concatenate_chunks(chunks)
File ~/mambaforge/envs/py39/lib/python3.9/site-packages/pandas/_libs/parsers.pyx:820, in pandas._libs.parsers.TextReader.read_low_memory()
File ~/mambaforge/envs/py39/lib/python3.9/site-packages/pandas/_libs/parsers.pyx:866, in pandas._libs.parsers.TextReader._read_rows()
File ~/mambaforge/envs/py39/lib/python3.9/site-packages/pandas/_libs/parsers.pyx:852, in pandas._libs.parsers.TextReader._tokenize_rows()
File ~/mambaforge/envs/py39/lib/python3.9/site-packages/pandas/_libs/parsers.pyx:1973, in pandas._libs.parsers.raise_parser_error()
ParserError: Error tokenizing data. C error: Expected 15 fields in line 10050, saw 17
Thoughts
Expand for details
The error message indicates a `ParserError` when reading the `2Mass.csv` file in chunks. Specifically, the error states: "Error tokenizing data. C error: Expected 15 fields in line 10050, saw 17." This suggests that the CSV file has inconsistent row lengths, likely due to formatting issues or extra delimiters in some rows.
Let's break down the issue:

1. **File Parsing Error**:
    - The `pd.read_csv` function expects each row to have a consistent number of fields (columns). In this case, it expects 15 fields, but line 10050 has 17 fields, causing the parser to fail.
2. **Possible Causes**:

    - **Extra Delimiters**: Some rows might contain extra commas or other delimiters, leading to more fields than expected.
    - **Quoted Fields**: If fields contain commas and are not properly quoted, the parser might misinterpret them as separate fields.
    - **Data Corruption**: The file might be corrupted or have inconsistent formatting.
3. **Solution Approach**:
    - **Inspect the File**: Manually inspect line 10050 and nearby lines to identify the formatting issue.
    - **Handle Extra Fields**: Use options in `pd.read_csv` to handle rows with extra fields, such as `error_bad_lines=False` or `warn_bad_lines=True`.
    - **Clean the File**: If possible, clean the file to ensure consistent formatting.

Given that the error occurs in the `cross_match_in_chunks` function, which reads the file in chunks, we need to ensure that each chunk is parsed correctly.
Let's update the script to handle this error:

1. **Modify** `pd.read_csv`:
    - Use `error_bad_lines=False` to skip rows with too many fields.
    - Use `warn_bad_lines=True` to print warnings for bad lines.
2. **Add Error Handling**:
    - Catch `ParserError` exceptions and print informative messages.

Here's the updated code snippet for the `cross_match_in_chunks` function:

```
python
def cross_match_in_chunks(df1, filepath, ra_col1, dec_col1, ra_col2, dec_col2, columns_to_keep, max_dist_arcsec=10.0, chunksize=100000):
    try:
        # Filter df1 by RA/Dec range
        min_ra, max_ra = df1[ra_col1].min() - 0.1, df1[ra_col1].max() + 0.1
        min_dec, max_dec = df1[dec_col1].min() - 0.1, df1[dec_col1].max() + 0.1

        # Initialize lists to store matched DataFrames
        matched_df1_list = []
        matched_df2_list = []

        # Read the large CSV file in chunks
        chunk_reader = pd.read_csv(filepath, chunksize=chunksize, error_bad_lines=False, warn_bad_lines=True)
        for i, chunk in enumerate(chunk_reader):
            print(f"Processing chunk {i+1} with {len(chunk)} rows...")
            # Filter chunk by RA/Dec range
            chunk_filtered = chunk[
                (chunk[ra_col2] >= min_ra) & (chunk[ra_col2] <= max_ra) &
```

```
            (chunk[dec_col2] >= min_dec) & (chunk[dec_col2] <= max_dec)
        ]
        if len(chunk_filtered) == 0:
            continue
        # Perform cross-match on the filtered chunk
        _, matched_df2 = cross_match(df1, chunk_filtered, ra_col1, dec_col1, ra_col2, dec_col2,
max_dist_arcsec)
        if len(matched_df2) > 0:
            matched_df2_list.append(matched_df2[columns_to_keep])

    if len(matched_df2_list) == 0:
        print("No matches found in any chunk.")
        return df1, pd.DataFrame()

    # Concatenate all matched_df2
    matched_df2 = pd.concat(matched_df2_list, ignore_index=True)

    # Since df1 remains the same, we can use it directly
    matched_df1 = df1.copy()

    return matched_df1, matched_df2
  except Exception as e:
    print(f"Error in cross_match_in_chunks: {e}")
    raise
```

This update should help handle rows with extra fields by skipping them and printing warnings. If the issue persists, you might need to manually inspect and clean the `2Mass.csv` file.

Step 8: Inspect and Clean the CSV File

If the error continues, you'll need to inspect the `2Mass.csv` file to identify and fix the formatting issues. Here's how you can do it:

1. **Locate the Problematic Line**:
    - The error mentions line 10050. Open the CSV file in a text editor or use a command-line tool to view the line.
    - For example, using `sed`:
    - `bash`
    - sed -n '10050p' 2Mass.csv
2. **Check for Extra Delimiters**:
    - Look for extra commas or other delimiters that might be causing the row to have more fields than expected.
    - Ensure that any fields containing commas are properly quoted.
3. **Fix the Formatting**:

- If you find formatting issues, correct them manually or use a script to clean the file.
- For example, if there are extra commas, you can remove them or adjust the quoting.

4. **Re-run the Script**:
   - After cleaning the file, re-run the script to see if the error is resolved.

Step 9: Alternative Approach

If cleaning the file is not feasible, you can consider using a different parser or loading method that is more tolerant of formatting errors. For example, you can use `pandas.read_csv` with `engine='python'` or `on_bad_lines='skip'`, but these options might be slower or less efficient.
Here's an example:
`python`
chunk_reader = pd.read_csv(filepath, chunksize=chunksize, engine='python', on_bad_lines='skip')

This will skip rows with parsing errors, but it might lead to data loss if many rows are affected.

Step 10: Summary

The `ParserError` is caused by inconsistent row lengths in the `2Mass.csv` file, specifically line 10050 having 17 fields instead of the expected 15. To resolve this:

1. Update the script to handle bad lines by skipping them and printing warnings.
2. Inspect and clean the CSV file to fix formatting issues.
3. Consider alternative parsing methods if necessary.

By following these steps, you should be able to address the parsing error and continue with the cross-matching process. If you need further assistance with cleaning the file or interpreting the data, feel free to ask!

Understanding the ParserError in Your Script

The error message you're encountering, `ParserError: Error tokenizing data. C error: Expected 15 fields in line 10050, saw 17`, occurs when your script attempts to read the `2Mass.csv` file using `pandas.read_csv` within the `cross_match_in_chunks` function. This

error indicates that the CSV file has inconsistent row lengths: while the parser expects each row to have 15 columns (fields), line 10050 contains 17 fields, causing the parsing process to fail.
This issue is common when working with CSV files that have formatting inconsistencies. Below, I'll explain the problem, its likely causes, and provide actionable solutions to resolve it.

Why This Error Occurs

The `pandas.read_csv` function assumes that every row in the CSV file has the same number of fields, determined by the header or the first few rows. When it encounters a row with a different number of fields—like line 10050 with 17 fields instead of the expected 15—it raises a `ParserError` because it cannot properly tokenize (split) the row into the expected columns.
Possible Causes:

1. **Extra Delimiters**: Line 10050 might contain extra commas (or another delimiter) that split the row into more fields than intended.
2. **Improper Quoting**: If a field contains a comma (e.g., within a string) and isn't properly enclosed in quotes, `pandas` might interpret it as a field separator.
3. **Data Corruption or Formatting Issues**: The file could have been generated or edited in a way that introduced inconsistencies, such as manual edits or concatenation errors.

Since your script reads the file in chunks (`chunksize=100000`), the error arises during the processing of a chunk containing line 10050.

Solutions to Fix the Error

Here are two primary approaches to resolve this issue: modifying your script to handle the error gracefully and inspecting/cleaning the CSV file to fix the root cause.
Solution 1: Modify the Script to Handle Inconsistent Rows
You can update the `cross_match_in_chunks` function to skip problematic rows while providing feedback about skipped lines. This is a quick fix that allows your script to proceed without crashing, though it may result in some data loss (e.g., skipping line 10050).
Here's how to modify your code:
```
python
def cross_match_in_chunks(df1, filepath, ra_col1, dec_col1, ra_col2, dec_col2, columns_to_keep, max_dist_arcsec=10.0, chunksize=100000):
    try:
```

```
        # Filter df1 by RA/Dec range
        min_ra, max_ra = df1[ra_col1].min() - 0.1, df1[ra_col1].max() + 0.1
        min_dec, max_dec = df1[dec_col1].min() - 0.1, df1[dec_col1].max() + 0.1

        # Initialize lists to store matched DataFrames
        matched_df1_list = []
        matched_df2_list = []

        # Read the CSV file in chunks, skipping bad lines
        chunk_reader = pd.read_csv(
            filepath,
            chunksize=chunksize,
            error_bad_lines=False,  # Skip rows with too many/few fields
            warn_bad_lines=True     # Print warnings for skipped rows
        )
        for i, chunk in enumerate(chunk_reader):
            print(f"Processing chunk {i+1} with {len(chunk)} rows...")
            # Filter chunk by RA/Dec range
            chunk_filtered = chunk[
                (chunk[ra_col2] >= min_ra) & (chunk[ra_col2] <= max_ra) &
                (chunk[dec_col2] >= min_dec) & (chunk[dec_col2] <= max_dec)
            ]
            if len(chunk_filtered) == 0:
                continue
            # Perform cross-match on the filtered chunk
            _, matched_df2 = cross_match(df1, chunk_filtered, ra_col1, dec_col1, ra_col2, dec_col2,
max_dist_arcsec)
            if len(matched_df2) > 0:
                matched_df2_list.append(matched_df2[columns_to_keep])

        if len(matched_df2_list) == 0:
            print("No matches found in any chunk.")
            return df1, pd.DataFrame()

        # Concatenate all matched_df2
        matched_df2 = pd.concat(matched_df2_list, ignore_index=True)
        matched_df1 = df1.copy()

        return matched_df1, matched_df2
    except Exception as e:
        print(f"Error in cross_match_in_chunks: {e}")
        raise
```

Key Changes:

- Added `error_bad_lines=False`: This tells `pandas` to skip rows with an incorrect number of fields (e.g., line 10050).
- Added `warn_bad_lines=True`: This prints a warning for each skipped row, helping you identify how many rows are affected.

Note: If you're using a newer version of `pandas` (1.3.0+), replace `error_bad_lines=False` and `warn_bad_lines=True` with `on_bad_lines='warn'` for a similar effect:
`python`
chunk_reader = pd.read_csv(filepath, chunksize=chunksize, on_bad_lines='warn')

Pros: Quick to implement, allows the script to run without crashing. **Cons**: Skips problematic rows, potentially losing data (e.g., line 10050 won't be processed).

Solution 2: Inspect and Clean the CSV File

For a more robust solution, you should inspect the `2Mass.csv` file to identify and fix the formatting issue at line 10050. This ensures no data is lost and addresses the root cause.

Steps to Inspect and Clean:

1. **Locate Line 10050**:
    - Open `2Mass.csv` in a text editor (e.g., VS Code, Notepad++) or use a command-line tool to view the specific line.
    - On Unix-like systems (Linux/macOS), you can use:
    - `bash`
    - sed -n '10050p' 2Mass.csv
    - Alternatively, use Python to print the line:
    - `python`

```
with open('2Mass.csv', 'r') as f:
  lines = f.readlines()
```

    - print(lines[10049]) # Line numbers are 1-based, list indices are 0-based
2. **Analyze the Line**:
    - Count the number of fields (e.g., commas). If there are 16 commas (17 fields), compare this to the header row or other rows, which should have 14 commas (15 fields).
    - Look for:
        - Extra commas (e.g., `field1,field2,,field3`).
        - Unquoted fields with commas (e.g., `field with, comma` instead of `"field with, comma"`).
3. **Fix the Issue**:
    - **Manual Fix**: Edit the file in a text editor to remove extra delimiters or add proper quotes.
    - **Scripted Fix**: Write a Python script to clean the file. For example:
    - `python`

```
import csv
```

```
input_file = '2Mass.csv'
output_file = '2Mass_cleaned.csv'

with open(input_file, 'r') as infile, open(output_file, 'w', newline='') as outfile:
    reader = csv.reader(infile)
    writer = csv.writer(outfile)
    header = next(reader)  # Assuming first row is header
    expected_fields = len(header)
    writer.writerow(header)
    for i, row in enumerate(reader, start=2):  # Start at line 2
        if len(row) != expected_fields:
            print(f"Line {i} has {len(row)} fields, expected {expected_fields}. Adjusting...")
            # Truncate or pad the row as needed (example: truncate extra fields)
            row = row[:expected_fields]
```

- writer.writerow(row)
- This script ensures all rows match the header's field count by truncating extra fields. Adjust the logic (e.g., padding with empty strings) based on your data's needs.

4. **Update the Script**:
   - After cleaning, use `2Mass_cleaned.csv` in your script:
   - `python`

```
df, twomass_matched = cross_match_in_chunks(
    df, '2Mass_cleaned.csv', 'objra_y', 'objdec', 'RAJ2000', 'DEJ2000',
    ['Jmag', 'Hmag', 'Kmag', 'e_Jmag', 'e_Hmag', 'e_Kmag'],
    max_dist_arcsec=10.0, chunksize=100000
```

-)

Pros: Preserves all data, fixes the root cause. **Cons**: Requires manual effort or scripting, depending on the file size and number of issues.

Alternative Approach: Use a More Tolerant Parser

If cleaning the file isn't feasible and skipping rows isn't acceptable, you can use a more flexible parsing engine in `pandas`:

`python`

```
chunk_reader = pd.read_csv(filepath, chunksize=chunksize, engine='python', on_bad_lines='skip')
```

- `engine='python'`: Uses Python's built-in CSV parser instead of the faster C-based parser, offering more flexibility.
- `on_bad_lines='skip'`: Skips problematic rows (available in `pandas` 1.3.0+).

Trade-off: This is slower than the C engine, which may matter for large files like `2Mass.csv`.

Recommendation

1. **Short-Term Fix**: Start with **Solution 1** by adding `error_bad_lines=False` and `warn_bad_lines=True` to your `pd.read_csv` call. This lets your script run immediately while warning you about skipped rows.
2. **Long-Term Solution**: Follow **Solution 2** to inspect line 10050 and clean `2Mass.csv`. This ensures data integrity and prevents future errors.
3. **Verify Results**: After applying either fix, check the output (`twomass_matched`) to ensure the cross-matching results are as expected.

If you need help inspecting the file or interpreting line 10050, feel free to share more details (e.g., the header and line 10050), and I'll assist further!
(myenv) root@BigsbyGaming:/home/pmqr7# sed -n '10050p' 2Mass.csv
46.762759,2.78324,"03070306+0246596
",15.987,0.06,15.442,0.082,15.42,0.156,"AAC","222","111","000",0,0 (myenv)
root@BigsbyGaming:/home/pmqr7#
can you print the full corrected STAR.py script tellint the program to only look for the columns needed in 2mass and to ignore the rest?
Thank you for sharing the output of line 10050 from `2Mass.csv`. Let's analyze the issue and update the `STAR.py` script to handle the parsing error and only use the necessary columns from `2Mass.csv`.

Analysis of Line 10050 and the ParserError

The line you provided from `2Mass.csv` at line 10050 is:
46.762759,2.78324,"03070306+0246596
",15.987,0.06,15.442,0.082,15.42,0.156,"AAC","222","111","000",0,0

This line has 15 fields (separated by 14 commas), which matches the expected structure based on the header you provided earlier for `2Mass.csv`:
RAJ2000,DEJ2000,2MASS,Jmag,e_Jmag,Hmag,e_Hmag,Kmag,e_Kmag,Qflg,Rflg,Bflg,Cflg,Xflg,Aflg

- **Expected Fields**: 15 fields, as defined by the header.
- **Fields in Line 10050**: Also 15 fields, which seems correct at first glance.

However, the `ParserError` indicates that `pandas` expected 15 fields but found 17 fields on line 10050. This discrepancy suggests that the line might have been misinterpreted due to:

1. **Hidden Characters or Formatting Issues**: There might be hidden characters (e.g., extra commas, unescaped quotes, or newline characters) in the file that `sed` doesn't display but `pandas` detects during parsing.
2. **File Encoding or Line Endings**: The file might have inconsistent line endings (e.g., a mix of `\n` and `\r\n`), causing `pandas` to misinterpret rows.
3. **Error in a Different Line**: The error might not actually be on line 10050 but in a nearby line within the same chunk, as `pandas` processes the file in chunks (`chunksize=100000`).

Since the line you shared looks correct, the issue is likely due to another row in the chunk containing line 10050. However, to resolve this, we can:

- **Specify the Columns to Use**: Instruct `pandas` to only load the columns we need (`RAJ2000`, `DEJ2000`, `Jmag`, `e_Jmag`, `Hmag`, `e_Hmag`, `Kmag`, `e_Kmag`) and ignore the rest.
- **Handle Parsing Errors Gracefully**: Use `on_bad_lines='skip'` to skip problematic rows while warning about them.

Updating the Script

We'll update the `STAR.py` script to:

1. **Load Only Necessary Columns from** `2Mass.csv`: Use the `usecols` parameter in `pd.read_csv` to load only `RAJ2000`, `DEJ2000`, `Jmag`, `e_Jmag`, `Hmag`, `e_Hmag`, `Kmag`, `e_Kmag`, ignoring the other columns (`Qflg`, `Rflg`, `Bflg`, `Cflg`, `Xflg`, `Aflg`).
2. **Handle Parsing Errors**: Add `on_bad_lines='skip'` to skip rows with parsing issues.
3. **Keep the Rest of the Script Intact**: The rest of the script (SDSS merging, cross-matching, and machine learning steps) remains unchanged.

Here's the fully corrected `STAR.py` script:

```
python
# STAR.py: Model for predicting log_SFR_Ha using the STAR framework
# STAR: Symbolic, Tuned (Optuna), Astronomy Regressors (Random Forest + Gradient Boosting)
# - S: Symbolic Regressor (via gplearn)
# - T: Tuned (hyperparameter optimization with Optuna)
# - A: Astronomy (predicting star formation rates in galaxies)
# - R: Regressors (Random Forest and Gradient Boosting)

import pandas as pd
import numpy as np
```

```
import shap
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split, cross_val_score, KFold
from sklearn.ensemble import RandomForestRegressor, GradientBoostingRegressor
from sklearn.metrics import r2_score
from sklearn.preprocessing import StandardScaler, MinMaxScaler
from gplearn.genetic import SymbolicRegressor
import optuna
import warnings
from numpy.polynomial import Polynomial
from scipy.spatial import cKDTree

warnings.filterwarnings("ignore")

# Load the original dataset
df = pd.read_csv("merged_data.csv")

# Load the SDSS datasets and merge them
sdss_dr16 = pd.read_csv("V154sdss16.csv")  # V/154: SDSS DR16
sdss_dr12 = pd.read_csv("V147sdss12.csv")  # V/147: SDSS DR12

# Merge V/154 and V/147 on objID
sdss_merged = pd.merge(
    sdss_dr16[['objID', 'RA_ICRS', 'DE_ICRS', 'zsp', 'umag', 'gmag', 'rmag', 'imag', 'zmag', 'e_umag',
'e_gmag', 'e_rmag', 'e_imag', 'e_zmag']],
    sdss_dr12[['objID', 'RA_ICRS', 'DE_ICRS', 'zsp', 'umag', 'gmag', 'rmag', 'imag', 'zmag', 'e_umag',
'e_gmag', 'e_rmag', 'e_imag', 'e_zmag']],
    on='objID',
    how='outer',
    suffixes=('_dr16', '_dr12')
)

# Combine columns, preferring DR16 values where available, otherwise use DR12
sdss_merged['RA_ICRS'] =
sdss_merged['RA_ICRS_dr16'].combine_first(sdss_merged['RA_ICRS_dr12'])
sdss_merged['DE_ICRS'] =
sdss_merged['DE_ICRS_dr16'].combine_first(sdss_merged['DE_ICRS_dr12'])
sdss_merged['zsp'] = sdss_merged['zsp_dr16'].combine_first(sdss_merged['zsp_dr12'])
sdss_merged['umag'] = sdss_merged['umag_dr16'].combine_first(sdss_merged['umag_dr12'])
sdss_merged['gmag'] = sdss_merged['gmag_dr16'].combine_first(sdss_merged['gmag_dr12'])
sdss_merged['rmag'] = sdss_merged['rmag_dr16'].combine_first(sdss_merged['rmag_dr12'])
sdss_merged['imag'] = sdss_merged['imag_dr16'].combine_first(sdss_merged['imag_dr12'])
sdss_merged['zmag'] = sdss_merged['zmag_dr16'].combine_first(sdss_merged['zmag_dr12'])
sdss_merged['e_umag'] = sdss_merged['e_umag_dr16'].combine_first(sdss_merged['e_umag_dr12'])
sdss_merged['e_gmag'] = sdss_merged['e_gmag_dr16'].combine_first(sdss_merged['e_gmag_dr12'])
sdss_merged['e_rmag'] = sdss_merged['e_rmag_dr16'].combine_first(sdss_merged['e_rmag_dr12'])
sdss_merged['e_imag'] = sdss_merged['e_imag_dr16'].combine_first(sdss_merged['e_imag_dr12'])
sdss_merged['e_zmag'] = sdss_merged['e_zmag_dr16'].combine_first(sdss_merged['e_zmag_dr12'])
```

```
# Drop the intermediate columns
sdss_merged = sdss_merged[['objID', 'RA_ICRS', 'DE_ICRS', 'zsp', 'umag', 'gmag', 'rmag', 'imag', 'zmag',
'e_umag', 'e_gmag', 'e_rmag', 'e_imag', 'e_zmag']]

# Save the merged dataset for future use
sdss_merged.to_csv("sdss_merged.csv", index=False)
print(f"Merged SDSS dataset created with {len(sdss_merged)} rows.")

# Print columns for debugging
print("Columns in merged_data.csv:", df.columns.tolist())
print("Columns in sdss_merged (V/147 + V/154):", sdss_merged.columns.tolist())

# Function to clean DataFrame by removing rows with NaN or inf in RA/Dec columns
def clean_coordinates(df, ra_col, dec_col):
    initial_len = len(df)
    # Check for NaN or inf in RA and Dec columns
    mask = (
        df[ra_col].notna() & df[dec_col].notna() &  # Not NaN
        np.isfinite(df[ra_col]) & np.isfinite(df[dec_col])  # Not inf
    )
    df_cleaned = df[mask].copy()
    print(f"Removed {initial_len - len(df_cleaned)} rows from DataFrame due to NaN or inf in {ra_col} or
{dec_col}")
    # Print RA/Dec ranges for debugging
    if len(df_cleaned) > 0:
        print(f"RA range in {ra_col}: {df_cleaned[ra_col].min():.4f} to {df_cleaned[ra_col].max():.4f}")
        print(f"Dec range in {dec_col}: {df_cleaned[dec_col].min():.4f} to {df_cleaned[dec_col].max():.4f}")
    else:
        print(f"No valid RA/Dec data after cleaning in {ra_col}/{dec_col}")
    return df_cleaned

# Cross-match function
def deg_to_rad(df, ra_col, dec_col):
    try:
        df['ra_rad'] = np.radians(df[ra_col])
        df['dec_rad'] = np.radians(df[dec_col])
        return df
    except KeyError as e:
        print(f"KeyError in deg_to_rad: {e}")
        print(f"Available columns in DataFrame: {df.columns.tolist()}")
        raise

def cross_match(df1, df2, ra_col1, dec_col1, ra_col2, dec_col2, max_dist_arcsec=10.0):
    try:
        # Clean DataFrames to remove rows with NaN or inf in RA/Dec
        df1_cleaned = clean_coordinates(df1, ra_col1, dec_col1)
        df2_cleaned = clean_coordinates(df2, ra_col2, dec_col2)
```

```
        # Check if either DataFrame is empty after cleaning
        if len(df1_cleaned) == 0 or len(df2_cleaned) == 0:
            print("One of the DataFrames is empty after cleaning. Cannot perform cross-match.")
            return df1_cleaned, pd.DataFrame()

        # Convert degrees to radians
        df1_cleaned = deg_to_rad(df1_cleaned, ra_col1, dec_col1)
        df2_cleaned = deg_to_rad(df2_cleaned, ra_col2, dec_col2)

        # Create coordinate arrays
        coords1 = np.array([df1_cleaned['ra_rad'], df1_cleaned['dec_rad']]).T
        coords2 = np.array([df2_cleaned['ra_rad'], df2_cleaned['dec_rad']]).T

        # Perform cross-match using cKDTree
        tree = cKDTree(coords2)
        max_dist = np.radians(max_dist_arcsec / 3600.0)
        dist, idx = tree.query(coords1, k=1, distance_upper_bound=max_dist)

        # Filter matches within the maximum distance
        matched = dist < max_dist
        df1_matched = df1_cleaned[matched].copy()
        df2_matched = df2_cleaned.iloc[idx[matched]].copy()

        # Reset indices
        df1_matched = df1_matched.reset_index(drop=True)
        df2_matched = df2_matched.reset_index(drop=True)

        # Print sample matches for debugging
        if len(df1_matched) > 0:
            print("Sample matches (first 5):")
            for i in range(min(5, len(df1_matched))):
                print(f"Match {i+1}: {ra_col1}={df1_matched[ra_col1].iloc[i]:.4f},
{dec_col1}={df1_matched[dec_col1].iloc[i]:.4f} "
                      f"matched to {ra_col2}={df2_matched[ra_col2].iloc[i]:.4f},
{dec_col2}={df2_matched[dec_col2].iloc[i]:.4f} "
                      f"(distance={(dist[matched][i] * 3600 * 180 / np.pi):.2f} arcsec)")
        else:
            print("No matches found within the specified radius.")

        return df1_matched, df2_matched
    except KeyError as e:
        print(f"KeyError in cross_match: {e}")
        print(f"df1 columns: {df1.columns.tolist()}")
        print(f"df2 columns: {df2.columns.tolist()}")
        raise
    except Exception as e:
        print(f"Error in cross_match: {e}")
        raise
```

```
# Cross-match function for large datasets (in chunks)
def cross_match_in_chunks(df1, filepath, ra_col1, dec_col1, ra_col2, dec_col2, columns_to_keep,
max_dist_arcsec=10.0, chunksize=100000):
    try:
        # Filter df1 by RA/Dec range
        min_ra, max_ra = df1[ra_col1].min() - 0.1, df1[ra_col1].max() + 0.1
        min_dec, max_dec = df1[dec_col1].min() - 0.1, df1[dec_col1].max() + 0.1

        # Initialize lists to store matched DataFrames
        matched_df2_list = []

        # Define the columns to load (only those needed)
        usecols = [ra_col2, dec_col2] + columns_to_keep

        # Read the CSV file in chunks, loading only necessary columns
        chunk_reader = pd.read_csv(
            filepath,
            chunksize=chunksize,
            usecols=usecols,  # Load only specified columns
            on_bad_lines='warn'  # Skip bad lines and warn
        )
        for i, chunk in enumerate(chunk_reader):
            print(f"Processing chunk {i+1} with {len(chunk)} rows...")
            # Filter chunk by RA/Dec range
            chunk_filtered = chunk[
                (chunk[ra_col2] >= min_ra) & (chunk[ra_col2] <= max_ra) &
                (chunk[dec_col2] >= min_dec) & (chunk[dec_col2] <= max_dec)
            ]
            if len(chunk_filtered) == 0:
                continue
            # Perform cross-match on the filtered chunk
            _, matched_df2 = cross_match(df1, chunk_filtered, ra_col1, dec_col1, ra_col2, dec_col2,
max_dist_arcsec)
            if len(matched_df2) > 0:
                matched_df2_list.append(matched_df2[columns_to_keep])

        if len(matched_df2_list) == 0:
            print("No matches found in any chunk.")
            return df1, pd.DataFrame()

        # Concatenate all matched_df2
        matched_df2 = pd.concat(matched_df2_list, ignore_index=True)
        matched_df1 = df1.copy()

        return matched_df1, matched_df2
    except Exception as e:
        print(f"Error in cross_match_in_chunks: {e}")
        raise
```

```
# Store the original DataFrame before cross-matching
df_original = df.copy()

# Cross-match with VizieR catalogues
# SDSS merged dataset (V/147 + V/154, uses RA_ICRS, DE_ICRS)
df, sdss_matched = cross_match(df, sdss_merged, 'objra_y', 'objdec', 'RA_ICRS', 'DE_ICRS',
max_dist_arcsec=10.0)
print(f"After SDSS cross-match: {len(df)} galaxies")

# Check if cross-match succeeded; if not, fall back to original DataFrame
if len(df) == 0:
    print("No matches found with SDSS merged data (V/147 + V/154). Falling back to original data and
using nsa_z for redshift.")
    df = df_original.copy()
    df['zsp'] = df['nsa_z']
else:
    # Add zsp, umag, gmag, etc. from the merged SDSS dataset
    df = pd.concat([df, sdss_matched[['zsp', 'umag', 'gmag', 'rmag', 'imag', 'zmag', 'e_umag', 'e_gmag',
'e_rmag', 'e_imag', 'e_zmag']]], axis=1)

# II/246: 2MASS Extended Source Catalogue (uses RAJ2000, DEJ2000)
df, twomass_matched = cross_match_in_chunks(
    df, "2Mass.csv",
    'objra_y', 'objdec', 'RAJ2000', 'DEJ2000',
    ['Jmag', 'Hmag', 'Kmag', 'e_Jmag', 'e_Hmag', 'e_Kmag'],
    max_dist_arcsec=10.0, chunksize=100000
)
print(f"After II/246 cross-match: {len(df)} galaxies")
if len(df) > 0:
    df = pd.concat([df, twomass_matched[['Jmag', 'Hmag', 'Kmag', 'e_Jmag', 'e_Hmag', 'e_Kmag']]], axis=1)

# II/356: GAMA DR3 (uses RAJ2000, DEJ2000)
df, gama_matched = cross_match_in_chunks(
    df, "II356xmmom41s.csv",
    'objra_y', 'objdec', 'RAJ2000', 'DEJ2000',
    ['UmAB', 'BmAB', 'VmAB'],
    max_dist_arcsec=10.0, chunksize=100000
)
print(f"After II/356 cross-match: {len(df)} galaxies")
if len(df) > 0:
    df = pd.concat([df, gama_matched[['UmAB', 'BmAB', 'VmAB']]], axis=1)

# Print columns after cross-matching for debugging
print("Columns in df after cross-matching:", df.columns.tolist())

# Recompute log_SFR_Ha and metallicity using fluxes from merged_data.csv
df['flux_Ha'] = df['F_Ha_cen']  # Use F_Ha_cen as flux_Ha
df['e_flux_Ha'] = df['e_F_Ha_cen']
df['flux_Hb'] = df['flux_Hbeta4861.36_Re_fit']
```

```
df['flux_OIII_5007'] = df['flux_[OIII]5006.84_Re_fit']
df['flux_NII_6584'] = df['flux_[NII]6583.45_Re_fit']

# Check for NaN in flux columns and handle them
flux_cols = ['flux_Ha', 'flux_Hb', 'flux_OIII_5007', 'flux_NII_6584', 'e_flux_Ha']
for col in flux_cols:
    nan_count = df[col].isna().sum()
    print(f"NaN count in {col}: {nan_count}")
    # Replace NaN with median for flux columns
    if nan_count > 0:
        df[col] = df[col].fillna(df[col].median())

df['Ha_Hb_observed'] = df['flux_Ha'] / df['flux_Hb']
Ha_Hb_intrinsic = 2.86
k_Ha = 2.468
k_Hb = 3.634
df['A_Ha'] = 2.5 * np.log10(df['Ha_Hb_observed'] / Ha_Hb_intrinsic) * (k_Ha / (k_Hb - k_Ha))
df['flux_Ha_corr'] = df['flux_Ha'] * 10**(0.4 * df['A_Ha'])
H0 = 70  # km/s/Mpc
c = 3e5  # km/s
df['DL'] = (c * df['zsp'] / H0) * 3.0856e24  # cm, using zsp
df['L_Ha'] = df['flux_Ha_corr'] * 4 * np.pi * df['DL']**2  # erg/s
df['log_SFR_Ha_raw'] = np.log10(df['L_Ha'] * 7.9e-42)  # M_sun/yr
df['log_O3N2_raw'] = np.log10((df['flux_OIII_5007'] / df['flux_Hb']) / (df['flux_NII_6584'] / df['flux_Ha']))
df['OH_O3N2_raw'] = 8.533 - 0.214 * df['log_O3N2_raw']

# Add raw fluxes as features
df['log_flux_Ha'] = np.log10(df['flux_Ha'] + 1e-10)
df['log_flux_Hb'] = np.log10(df['flux_Hb'] + 1e-10)
df['log_flux_OIII_5007'] = np.log10(df['flux_OIII_5007'] + 1e-10)
df['log_flux_NII_6584'] = np.log10(df['flux_NII_6584'] + 1e-10)
df['log_e_flux_Ha'] = np.log10(df['e_flux_Ha'] + 1e-10)

# Add photometric colors (if available)
if 'umag' in df.columns:
    df['color_ug'] = df['umag'] - df['gmag']
    df['color_gr'] = df['gmag'] - df['rmag']
    df['color_ri'] = df['rmag'] - df['imag']
    df['color_iz'] = df['imag'] - df['zmag']
    df['e_color_ug'] = np.sqrt(df['e_umag']**2 + df['e_gmag']**2)
    df['e_color_gr'] = np.sqrt(df['e_gmag']**2 + df['e_rmag']**2)

# Add 2MASS colors (if available)
if 'Jmag' in df.columns:
    df['color_JK'] = df['Jmag'] - df['Kmag']
    df['e_color_JK'] = np.sqrt(df['e_Jmag']**2 + df['e_Kmag']**2)

# Drop rows with missing target or excessive NaNs
df = df.dropna(subset=["log_SFR_Ha"])
```

```
df = df.dropna(axis=0, thresh=int(0.8 * df.shape[1]))
df.fillna(df.median(numeric_only=True), inplace=True)

# Step 1: Cosmo-Rank Construction
gz_df = df.copy()
morph_cols = ['P(CD)', 'P(E)', 'P(S0)', 'P(Sa)', 'P(Sab)', 'P(Sb)', 'P(Sbc)',
              'P(Sc)', 'P(Scd)', 'P(Sd)', 'P(Sdm)', 'P(Sm)', 'P(Irr)']
gz_df['morph_sum'] = gz_df[morph_cols].sum(axis=1)
gz_df['Num_w'] = gz_df['conf_prob']
scaler_rank = MinMaxScaler()
gz_df[['morph_sum_norm', 'Num_w_norm', 'nsa_z_norm']] = scaler_rank.fit_transform(
    gz_df[['morph_sum', 'Num_w', 'nsa_z']]
)
gz_df['cosmo_rank'] = (0.5 * gz_df['Num_w_norm'] + 0.3 * gz_df['morph_sum_norm'] + 0.2 *
gz_df['nsa_z_norm'])
df['cosmo_rank'] = gz_df['cosmo_rank']

# Step 2: L_cosmo(s) Construction
alpha = -1.5
M_star = df['log_Mass_gas'].median()
df['a_n'] = (10 ** df['log_Mass_gas'])**(1 + alpha) * np.exp(-10 ** df['log_Mass_gas'] / (10 ** M_star))
bins = pd.qcut(df['Re_kpc'], q=20, labels=False, duplicates='drop') + 1
df['z_bin'] = bins
n_bins = df['z_bin'].nunique()
print(f"Number of unique bins after qcut: {n_bins}")
print("Bin distribution for Re_kpc (after qcut):")
print(df['z_bin'].value_counts().sort_index())

s_vals = [0.5, 1.0, 1.5, 2.0]
s_range = np.linspace(0.5, 2, 50)
for s in s_vals:
    col_name = f'L_cosmo_s_{s:.1f}'
    df[col_name] = df.groupby('z_bin')['a_n'].transform('mean') / (df['z_bin'] ** s)

print("Columns after creating L_cosmo_s*:")
print(df.columns.tolist())

print("Checking for NaN/infinite values in cosmo_rank and L_cosmo_s_1.0:")
print("cosmo_rank NaN count:", df['cosmo_rank'].isna().sum())
print("L_cosmo_s_1.0 NaN count:", df['L_cosmo_s_1.0'].isna().sum())
print("cosmo_rank infinite count:", np.isinf(df['cosmo_rank']).sum())
print("L_cosmo_s_1.0 infinite count:", np.isinf(df['L_cosmo_s_1.0']).sum())

df['cosmo_rank_L'] = df['cosmo_rank'] * df['L_cosmo_s_1.0']
df['cosmo_rank_L'] = df['cosmo_rank_L'].replace([np.inf, -np.inf], np.nan).fillna(0)
print("cosmo_rank_L created, NaN count:", df['cosmo_rank_L'].isna().sum())

# Plot L_cosmo(s) vs s
grouped_means = df.groupby('z_bin')['a_n'].mean().reindex(range(1, n_bins + 1), fill_value=0)
```

```
L_vals = [grouped_means / (np.arange(1, n_bins + 1) ** s) for s in s_range]
L_vals = np.array([sum(l) for l in L_vals])
plt.plot(s_range, L_vals)
plt.xlabel("s")
plt.ylabel("L_cosmo(s)")
plt.title("L_cosmo(s) Behavior")
plt.savefig("L_cosmo_curve.png")
plt.close()

# Scatter plot of L_cosmo_s_1.0 vs log_SFR_Ha
plt.scatter(df['L_cosmo_s_1.0'], df['log_SFR_Ha'], alpha=0.5)
plt.xlabel("L_cosmo_s_1.0")
plt.ylabel("log_SFR_Ha")
plt.title("L_cosmo_s_1.0 vs log_SFR_Ha")
plt.savefig("L_cosmo_s1_vs_SFR.png")
plt.close()

# Step 3: Define features and target
target = "log_SFR_Ha"
base_features = [
    "log_Mass_gas", "log_Mass", "Av_gas_Re", "Av_ssp_Re",
    "OH_O3N2_cen", "OH_T04_cen", "OH_dop_cen", "Age_LW_Re_fit",
    "ZH_LW_Re_fit", "Re_kpc", "vel_disp_Ha_cen", "Lambda_Re",
    "Sigma_Mass_Re", "EW_Ha_cen", "Ha_Hb_cen"
]
df["mass_metallicity"] = df["log_Mass"] * df["OH_O3N2_cen"]
df["dust_metallicity"] = df["Av_gas_Re"] * df["OH_T04_cen"]
df["disp_mass_ratio"] = df["vel_disp_Ha_cen"] / (df["Sigma_Mass_Re"] + 1e-5)
df["age_metallicity"] = df["Age_LW_Re_fit"] * df["ZH_LW_Re_fit"]
df["sqrt_Re_kpc"] = np.sqrt(df['Re_kpc'] + 1e-5)
df["log_mass"] = np.log(df["log_Mass_gas"] + 1e-5)
df["BSD_likelihood"] = (
    df["log_Mass_gas"] * df["OH_O3N2_cen"] /
    (df["Av_gas_Re"] + 1e-5 + df["Age_LW_Re_fit"])
)
df["cosmo_rank_mass"] = df["cosmo_rank"] * df["log_Mass"]
df["L_cosmo_s1_mass"] = df["L_cosmo_s_1.0"] * df["log_Mass"]
df["cosmo_rank_EW"] = df["cosmo_rank"] * df["EW_Ha_cen"]
df["L_cosmo_s1_EW"] = df["L_cosmo_s_1.0"] * df["EW_Ha_cen"]
df["cosmo_rank_scaled"] = df["cosmo_rank"] * 20
df["L_cosmo_s1_scaled"] = df["L_cosmo_s_1.0"] * 20
df["cosmo_rank_L_mass"] = df["cosmo_rank_L"] * df["log_Mass"]
df["L_cosmo_s1_metallicity"] = df["L_cosmo_s_1.0"] * df["OH_O3N2_cen"]
features = base_features + [
    "mass_metallicity", "dust_metallicity", "disp_mass_ratio",
    "age_metallicity", "sqrt_Re_kpc", "log_mass", "BSD_likelihood",
    "cosmo_rank", "L_cosmo_s_0.5", "L_cosmo_s_1.5", "L_cosmo_s_2.0",
    "cosmo_rank_L", "cosmo_rank_mass", "L_cosmo_s1_mass",
    "cosmo_rank_EW", "L_cosmo_s1_EW", "cosmo_rank_scaled", "L_cosmo_s1_scaled",
```

```
    "cosmo_rank_L_mass", "L_cosmo_s1_metallicity",
    "log_flux_Ha", "log_flux_Hb", "log_flux_OIII_5007", "log_flux_NII_6584",
    "log_e_flux_Ha", "log_SFR_Ha_raw", "OH_O3N2_raw"
]

# Add color features if available
if 'color_ug' in df.columns:
    features.extend(["color_ug", "color_gr", "color_ri", "color_iz", "e_color_ug", "e_color_gr"])
if 'color_JK' in df.columns:
    features.extend(["color_JK", "e_color_JK"])

X = df[features]
y = df[target]

# Train/test split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Normalize features
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# Optuna optimization for Random Forest
def objective_rf(trial):
    params = {
        'n_estimators': trial.suggest_int('n_estimators', 100, 500),
        'max_depth': trial.suggest_int('max_depth', 5, 20),
        'min_samples_split': trial.suggest_int('min_samples_split', 2, 10),
        'random_state': 42
    }
    model = RandomForestRegressor(**params)
    cv = KFold(n_splits=5, shuffle=True, random_state=42)
    scores = cross_val_score(model, X_train_scaled, y_train, cv=cv, scoring='r2')
    return scores.mean()

study_rf = optuna.create_study(direction='maximize')
study_rf.optimize(objective_rf, n_trials=20)
best_params_rf = study_rf.best_params
print("Best Random Forest Parameters:", best_params_rf)
print("Best Random Forest CV R²:", study_rf.best_value)

# Train Random Forest with best parameters
rf = RandomForestRegressor(**best_params_rf, random_state=42)
rf.fit(X_train_scaled, y_train)
y_pred_rf = rf.predict(X_test_scaled)
r2_rf = r2_score(y_test, y_pred_rf)

# Cross-validation for final RF model
cv_scores_rf = cross_val_score(rf, X_train_scaled, y_train, cv=5, scoring='r2')
```

```
print(f"Random Forest 5-Fold CV R²: Mean = {cv_scores_rf.mean():.4f}, Std = {cv_scores_rf.std():.4f}")

# Optuna optimization for Gradient Boosting
def objective_gb(trial):
    params = {
        'n_estimators': trial.suggest_int('n_estimators', 100, 500),
        'learning_rate': trial.suggest_float('learning_rate', 0.01, 0.2, log=True),
        'max_depth': trial.suggest_int('max_depth', 3, 10),
        'random_state': 42
    }
    model = GradientBoostingRegressor(**params)
    cv = KFold(n_splits=5, shuffle=True, random_state=42)
    scores = cross_val_score(model, X_train_scaled, y_train, cv=cv, scoring='r2')
    return scores.mean()

study_gb = optuna.create_study(direction='maximize')
study_gb.optimize(objective_gb, n_trials=20)
best_params_gb = study_gb.best_params
print("Best Gradient Boosting Parameters:", best_params_gb)
print("Best Gradient Boosting CV R²:", study_gb.best_value)

# Train Gradient Boosting with best parameters
gb = GradientBoostingRegressor(**best_params_gb, random_state=42)
gb.fit(X_train_scaled, y_train)
y_pred_gb = gb.predict(X_test_scaled)
r2_gb = r2_score(y_test, y_pred_gb)

# Cross-validation for final GB model
cv_scores_gb = cross_val_score(gb, X_train_scaled, y_train, cv=5, scoring='r2')
print(f"Gradient Boosting 5-Fold CV R²: Mean = {cv_scores_gb.mean():.4f}, Std =
{cv_scores_gb.std():.4f}")

print(f"R^2 Score (Random Forest, Test Set): {r2_rf:.4f}")
print(f"R^2 Score (Gradient Boosting, Test Set): {r2_gb:.4f}")

# SHAP for feature selection
explainer_rf = shap.Explainer(rf, X_train_scaled)
shap_values_rf = explainer_rf(X_test_scaled)
explainer_gb = shap.Explainer(gb, X_train_scaled)
shap_values_gb = explainer_gb(X_test_scaled)
shap.summary_plot(shap_values_rf, X_test_scaled, plot_type="bar", show=False)
plt.title("SHAP Summary - Random Forest")
plt.tight_layout()
plt.savefig("shap_rf_summary.png")
plt.clf()
shap.summary_plot(shap_values_gb, X_test_scaled, plot_type="bar", show=False)
plt.title("SHAP Summary - Gradient Boosting")
plt.tight_layout()
plt.savefig("shap_gb_summary.png")
```

```
plt.clf()

# Feature selection based on SHAP
shap_values_rf_mean = np.abs(shap_values_rf.values).mean(axis=0)
feature_importance = pd.DataFrame({"feature": features, "importance": shap_values_rf_mean})
feature_importance = feature_importance.sort_values("importance", ascending=False)
top_features = feature_importance["feature"].head(10).tolist()
bsd_features = ["cosmo_rank", "L_cosmo_s_1.0", "cosmo_rank_L", "cosmo_rank_scaled",
"L_cosmo_s1_scaled", "cosmo_rank_L_mass", "L_cosmo_s1_metallicity"]
for bsd_feature in bsd_features:
    if bsd_feature not in top_features:
        top_features.append(bsd_feature)
X_selected = df[top_features]
X_train_selected, X_test_selected, _, _ = train_test_split(X_selected, y, test_size=0.2, random_state=42)
X_train_selected_scaled = scaler.fit_transform(X_train_selected)
X_test_selected_scaled = scaler.transform(X_test_selected)

# Symbolic Regression with gplearn
symbolic_model = SymbolicRegressor(
    population_size=3000,
    generations=150,
    stopping_criteria=0.01,
    p_crossover=0.7,
    p_subtree_mutation=0.1,
    p_hoist_mutation=0.05,
    p_point_mutation=0.1,
    max_samples=0.9,
    verbose=1,
    parsimony_coefficient=0.0001,
    random_state=42,
    function_set=('add', 'sub', 'mul', 'div', 'sqrt', 'log', 'sin', 'cos')
)
symbolic_model.fit(X_train_selected_scaled, y_train)
y_pred_sym = symbolic_model.predict(X_test_selected_scaled)
r2_sym = r2_score(y_test, y_pred_sym)
print(f"gplearn R²: {r2_sym:.4f}")
print("Symbolic Expression:")
print(symbolic_model._program)

# Cross-validation for symbolic model
cv_scores_sym = cross_val_score(symbolic_model, X_train_selected_scaled, y_train, cv=5, scoring='r2')
print(f"Symbolic Regression 5-Fold CV R²: Mean = {cv_scores_sym.mean():.4f}, Std =
{cv_scores_sym.std():.4f}")

# Save symbolic expression as polynomial fit plot and print the equation
x = np.linspace(min(y_test), max(y_test), 500)
y_expr = symbolic_model.predict(scaler.transform(np.tile(X_test_selected.mean().values, (500,1))))
p = Polynomial.fit(y_test, y_pred_sym, deg=3)
# Extract polynomial coefficients
```

```
coeffs = p.coef
print("Polynomial Fit Equation (degree 3):")
print(f"y = {coeffs[0]:.4f} + {coeffs[1]:.4f}*x + {coeffs[2]:.4f}*x^2 + {coeffs[3]:.4f}*x^3")
# Plot the polynomial fit
plt.plot(*p.linspace(), label="Polynomial Fit")
plt.scatter(y_test, y_pred_sym, s=10, alpha=0.5, label="Symbolic Predictions")
plt.xlabel("True log_SFR_Ha")
plt.ylabel("Predicted log_SFR_Ha")
plt.title("Symbolic Regression Fit with Polynomial")
plt.legend()
plt.tight_layout()
plt.savefig("gplearn_expression_plot.png")
plt.clf()

# Plot predictions vs true
plt.scatter(y_test, y_pred_rf, alpha=0.5, label="RF", marker="o")
plt.scatter(y_test, y_pred_gb, alpha=0.5, label="GB", marker="s")
plt.scatter(y_test, y_pred_sym, alpha=0.5, label="Symbolic", marker="^")
plt.plot([y.min(), y.max()], [y.min(), y.max()], "k--")
plt.xlabel("True log_SFR_Ha")
plt.ylabel("Predicted")
plt.title("Model Predictions vs True Values")
plt.legend()
plt.tight_layout()
plt.savefig("model_comparison.png")
plt.clf()

# Residuals and Correlation Analysis
residuals = pd.DataFrame({
    "True": y_test,
    "RF": y_pred_rf - y_test,
    "GB": y_pred_gb - y_test,
    "Symbolic": y_pred_sym - y_test,
    "cosmo_rank": X_test["cosmo_rank"],
    "L_cosmo_s1": X_test["L_cosmo_s_1.0"]
})
sns.scatterplot(data=residuals, x="True", y="Symbolic", hue="cosmo_rank", size="L_cosmo_s1",
alpha=0.6)
plt.axhline(0, color="black", linestyle="--")
plt.title("Symbolic Residuals vs True log_SFR_Ha")
plt.savefig("residuals_sym.png")
plt.clf()

print("Correlation of BSD Features with Residuals:")
print(residuals[["RF", "GB", "Symbolic", "cosmo_rank", "L_cosmo_s1"]].corr()[["RF", "GB", "Symbolic"]])
```

Key Changes Made

1. **Updated `cross_match_in_chunks` to Use Specific Columns**:
    - Added `usecols=[ra_col2, dec_col2] + columns_to_keep` to `pd.read_csv`, ensuring only the necessary columns (`RAJ2000`, `DEJ2000`, `Jmag`, `e_Jmag`, `Hmag`, `e_Hmag`, `Kmag`, `e_Kmag`) are loaded from `2Mass.csv`.
    - This reduces the risk of parsing errors by ignoring columns that might have formatting issues (e.g., `Qflg`, `Rflg`, etc.).
2. **Added `on_bad_lines='warn'`**:
    - This ensures that rows with parsing issues (e.g., too many or too few fields) are skipped, and a warning is printed, allowing the script to continue running.
3. **Removed Unnecessary Debugging for `2Mass.csv` Columns**:
    - Since we're now loading only the necessary columns, we don't need to print all columns of `2Mass.csv` for debugging.
4. **Kept the Rest of the Script Intact**:
    - The SDSS merging, cross-matching with other datasets, and machine learning steps remain unchanged.

What to Expect When Running the Script

1. **No More ParserError**:
    - By loading only the necessary columns and skipping bad lines, the `ParserError` should be resolved.
    - If any rows are skipped, you'll see warnings like:
    - Skipping line 10050: expected 15 fields, saw 17
2. **Cross-Matching Output**:
    - The script will print the number of matches found after each cross-match step (SDSS, 2MASS, GAMA).
    - If no matches are found with SDSS, it will fall back to using `nsa_z` for redshift.
3. **Debugging Information**:
    - The script will print RA/Dec ranges for `merged_data.csv` and `sdss_merged` to help diagnose why cross-matching might be failing.
    - It will also print sample matches (if any) with their coordinates and distances.

Next Steps

1. **Run the Script**:
    - Save the updated script as `STAR.py` and run it:
    - `bash`
    - python STAR.py
    - Check the output for any warnings about skipped lines in `2Mass.csv`.
2. **Check Cross-Matching Results**:
    - Look at the number of matches after each cross-match step:

After SDSS cross-match: X galaxies
After II/246 cross-match: Y galaxies

    - After II/356 cross-match: Z galaxies
    - If the SDSS cross-match still results in 0 matches, share the RA/Dec ranges printed by the script, and we can investigate further.
3. **Inspect Skipped Lines (Optional)**:
    - If you see warnings about skipped lines in `2Mass.csv`, you can inspect those lines to understand the formatting issues (e.g., using `sed` or a text editor) and clean the file if needed.

Additional Notes

- **Performance**: Loading only the necessary columns from `2Mass.csv` should improve performance slightly, as `pandas` doesn't need to parse the entire file.
- **Data Loss**: Skipping bad lines means you might lose some data from `2Mass.csv`. If this is a concern, you can clean the file manually or write a script to fix the formatting issues, as discussed in the previous response.
- **Cross-Matching**: The script still uses a 10-arcsecond radius for cross-matching. If you're still getting 0 matches, we can increase this further or investigate coordinate mismatches.